

PLUGIN YOUR GAMES 2.0
ПОДКЛЮЧАЙТЕ СВОИ ИГРЫ

Оглавление

1. Начало работы
2. Часто Задаваемые Вопросы
3. Шаблон
4. Переход с PluginYG 1.x на 2.0
5. Платформы
6. Платформа Yandex Games
7. Платформа Crazy Games
8. Платформа Game Monetize
9. Платформа Welwise Games
10. Appodeal
11. Платформа GamePix
12. Полноэкранная реклама
13. Реклама за вознаграждение
14. Sticky-баннеры
15. Баннерная реклама
16. Данные окружения
17. Авторизация
18. Сохранения
19. Таблица лидеров
20. Локализация
21. Внутриигровые покупки
22. Метрика
23. Ссылки на игры и разработчика
24. Флаги (удалённая конфигурация)
25. Порт игры на TV
26. Серверное время
27. Оценка игры
28. Ярлык игры на рабочий стол
29. Полный экран (Fullscreen модуль)
30. Буфер обмена (Clipboard модуль)
31. Событие при выходе из игры (Quit Game Event модуль)
32. Создание своего модуля

33. Android Wi-Fi ADB Connect (ADB Build)

Ссылки

- Скачать с Unity Asset Store
- Канал в Telegram - Новости об обновлениях, полезная и важная информация.
- Чат поддержки в Telegram - Задать вопрос или найти уже готовые ответы.

Ниже предоставлены видеоматериалы.

Рекомендуется всё же ориентироваться на документацию, т.к. она содержит более подробную и всегда актуальную информацию.
Демонстрация и базовый функционал
Плейлист "Курс по плагину Your Games 2.0"

Оглавление раздела:

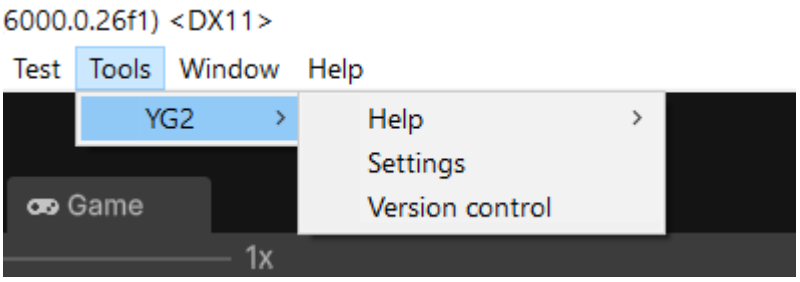
1. Начало работы
2. Модульная система
3. Работа с кодом
4. Демо материалы
5. Компонент "EventsYG2"
6. Сборка билда

Начало работы

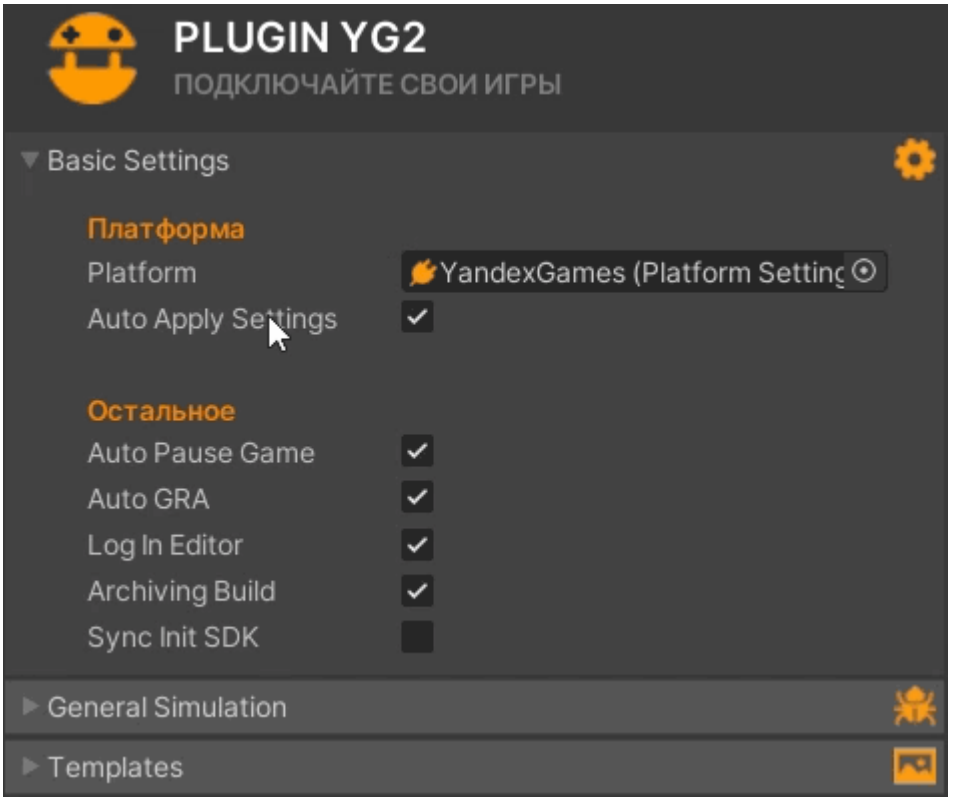
⚠ Обратите внимание !

- **Поддерживаемые версии Unity: 2021.3.18+, 2022.3+, 2023.2+, Unity 6.**
- **Не перемещайте файлы плагина в проекте.**
- **Перед импортом плагина убедитесь, что в проекте отсутствуют ошибки компиляции.**

Скачайте PluginYG2 с Unity Asset Store и импортируйте его в проект.
Вас встретит приветственное окно. В нём дублируются кнопки из контекстного меню:



Откройте **настройки** плагина — это фундаментальное окно. Обратите внимание, многие поля имеют всплывающие подсказки!



Рекомендации

- Рекомендуется ознакомиться со всеми параметрами из раздела **Basic Settings**

Если при импорте плагина вы отказались от установки оптимальных настроек для стандартной платформы Яндекс Игры

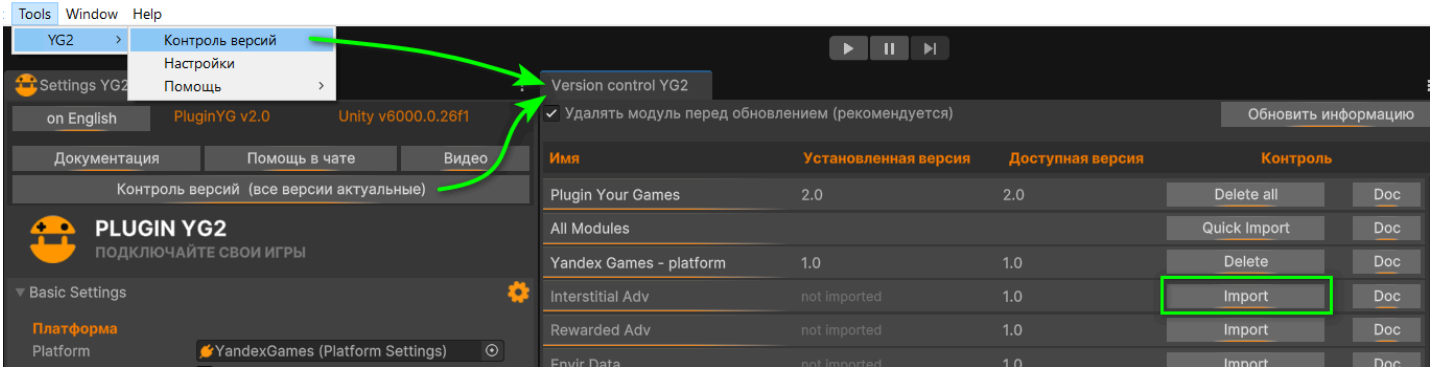
- В Basic Settings → Platform, необходимо выбрать платформу. Для портирования на WebGL, требуется выбрать соответствующий шаблон в Player Settings → Resolution and Presentation → WebGL Template. Вместо выбора шаблона, вы можете нажать галку Auto Apply Settings под полем Platform. В таком случае, шаблон применится автоматически, но вместе с ним применятся и другие настройки, которые предлагает плагин. Вы можете отключить автоматический выбор настроек проекта, это изменяется в свойствах платформы (поле Platform).

Локальный хост и размещение на своём сервере

- Тестирование игры на локальном хосте с помощью Build And Run - работает. При этом инициализация SDK (получение данных платформы) будет игнорироваться.
- При размещении игры на своём сервере для Яндекс Игр, включите опцию Own Host, которая находится в настройках платформы (поле Platform в разделе Basic Settings). Это добавит соответствующую строку подключения в index.html.
- Обратите внимание, ваш хост может не поддерживать некоторые настройки проекта. Например, компрессию Brotli. Если при импорте плагина вы соглашались на установку оптимальных настроек, то PluginYG2 сам меняет некоторые параметры проекта. Для контроля над этим процессом, перейдите в настройки платформы и внесите изменения в соответствии с вашими предпочтениями.
- Для локального тестирования игры с подключением к SDK, в Index.html измените функцию IsLocalHost под свои нужды. Данный файл можно корректировать в папке WebGLTemplates/YandexGames.

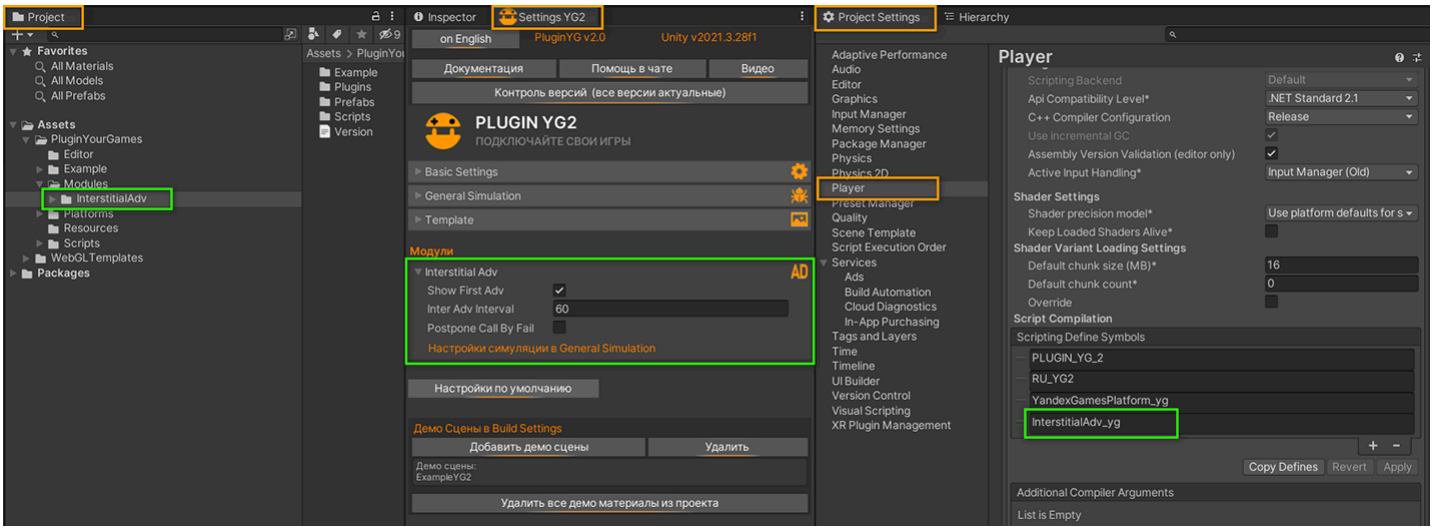
Модульная система

Все функции плагина разделены на отдельные модули.
Откройте инструмент контроля версий. Давайте скачаем и импортируем для примера модуль полноэкранной рекламы



Когда модуль импортирован:

1. Он попадает в папку **Modules**. Любая папка в директории Modules будет определена как модуль плагина.
2. Если модуль имеет глобальные настройки, то они отобразятся в настройках плагина.
3. Для всех модулей будет создан свой дефайн (также они называются директивами препроцессора). Вы можете использовать её в своём коде.



У дефайнов модулей плагина в конце названия есть постфикс yg. Дефайны платформ имеют следующую структуру именования НазваниеПлатформыPlatform_yg (например YandexGamesPlatform_yg. При возникновении проблем, вы можете проверить все ли дефайны правильно установлены и скорректировать их. Ещё можно попробовать обновить данные о модулях в проекте — удалив файл PluginYourGames/Editor/ModulesListYG2.txt

Если вы обнаружили явные проблемы с плагином

Пожалуйста, сообщите об этом в чат поддержки плагина в Telegram. Там вы сможете найти обсуждения уже с готовыми решениями, общаться по тематике разработки игр. Также вы можете следить за обновлениями плагина и другой важной информацией в телеграм канале PluginYG2 на связи.

Работа с кодом

PluginYG находится в пространстве имён YG (`using YG`), добавьте его в начало вашего скрипта, если используемая вами среда разработки не сделала это.
Практически все функции находятся в классе YG2. Так, например, мы можем найти в данном классе метод вызова рекламы: `YG2.InterstitialAdvShow();`
Как работать с полноэкранной рекламой и другими модулями смотрите в соответствующих разделах.

Другие объекты, которые содержатся в PluginYG2 без отдельных модулей:

Основные

`platform` — пример значения, которое может содержать поле platform: `YandexGames` , `CrazyGames` .
`onGetSDKData` — событие, сигнализирующее о том, что SDK платформы инициализирован (может пригодиться при использовании опции Sync Init).
`isSDKEnabled` — поле равно true — когда SDK инициализирован.
`isFirstGameSession` — поле, информирующее о том, первая ли сейчас игровая сессия.

Если первая — будет равно true всю текущую игровую сессию. Но обратите внимание! Данное поле не сохраняется в облачное хранилище.

Пауза игры

`onPauseGame` — событие паузы игры. Возвращает bool. Подробнее в разделе описания модуля полноэкранной рекламы.
`PauseGame` — метод для остановки игры. Принимает параметр bool: `true` - установить паузу, `false` - продолжить игру.
Метод PauseGame имеет и другие параметры, чтобы при включении паузы можно было выбрать: контролировать ли звук, время, курсор мыши, Event System.
`PauseGameNoEditEventSystem` — метод паузы без остановки Event System. Его удобно выполнять через EventsYG2 (о нём ниже). Контроль над Event System можно полностью отключить с помощью опции Edit Event System (настройки плагина → Basic Settings → Pause Game).
`isPauseGame` — поле равняется `true` когда игра остановлена.

Фокус игры

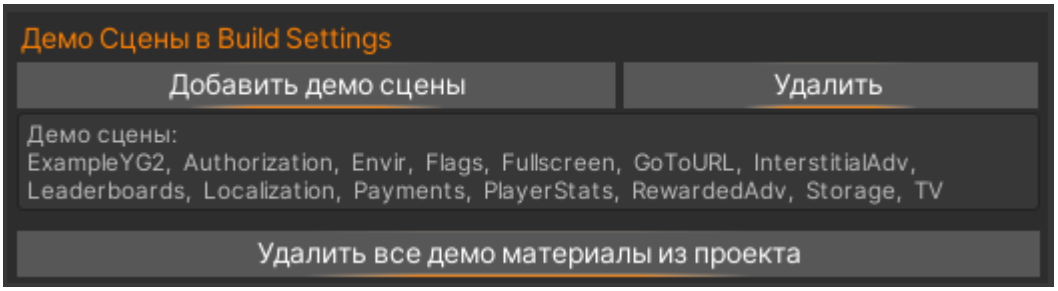
`onShowWindowGame` — событие срабатывает, когда игра становится в фокусе.
`onHideWindowGame` — выход из фокуса.
`onFocusWindowGame` — возвращает bool значение.
`isFocusWindowGame` — с помощью этого поля можно узнать в фокусе ли сейчас игра.

Разметка геймплея

- `GameReadyAPI` — выполняйте метод, когда игра загрузилась и пользователь может приступить к геймплею. Необходимо использовать, если в Basic Settings отключена галка Auto GRA (подробнее о Game Ready). По умолчанию метод GameReadyAPI выполняется автоматически после загрузки игры, потому что опция AutoGRA в настройках плагина включена. Если в вашей игре после её загрузки есть ещё дополнительные задержки - выключите AutoGRA и вызывайте метод вручную.
- `GameplayStart` — используйте метод, чтобы дать сигнал платформе о начале геймплея.
- `GameplayStop` — используйте метод, чтобы дать сигнал платформе об остановке геймплея.
- При открытии и закрытии рекламы или других паузах - выполняются методы `GameplayStart` и `GameplayStop`. Но только если раньше в игре был вызван метод `GameplayStart`.
- То есть, если в коде использовать методы разметки, то они будут автоматически выполняться при просмотре рекламы и других схожих паузах. А после паузы будет установлено состояние геймплея такое, какое было до паузы.

- Методы разметки геймплея необязательны, но их внедрение приветствуется.
- `isGameplaying` — поле, информирующее о активности геймплея.
- `YG2.optionalPlatform.HappyTime` — счастливое время.

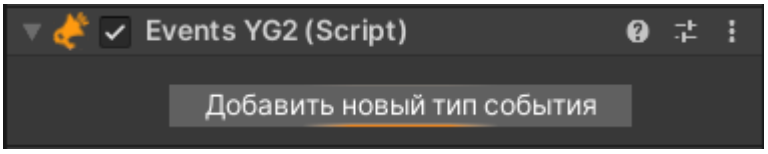
Демо материалы



Все демонстрационные материалы находятся в папках **Example**. Откройте главную демо-сцену: PluginYourGames/Example/Scenes/**ExampleYG2**. Главная сцена отображает демо-объекты всех модулей. Если у модуля есть сцена с демо-объектом, то он отобразится. Но сначала нужно нажать на кнопку "**Добавить демо сцены**". Тогда сцены модулей добавятся в Build Settings. Перед релизом игры или по завершению изучения плагина можно удалить демо сцены из списка сцен в Build Settings нажав на кнопку "**Удалить**" или выполнить удаление всех демо материалов из проекта.

> ⚠ **Важно**
>
> При удалении какого-либо модуля — сцена модуля останется в Build Settings и будет неопределённой. Перед удалением модуля желательно в разделе демо сцен нажать на кнопку "Удалить".

Компонент "EventsYG2"



- Компонент EventsYG2 хранит в себе большинство доступных методов плагина. С его помощью без кода можно, например, назначить на кнопку метод вызова рекламы.
- С помощью компонента EventsYG2 можно выполнять методы по событию. Для этого нажмите на кнопку «**Добавить новый тип события**» и выбирайте событие, при котором необходимо выполнить какой-либо метод. Это работает также, как вы привязываете методы на клик по кнопке в компоненте **Button**.

Сборка билда

Теперь вы знаете основу работы с **Plugin Your Games 2.0**. С этими знаниями уже можно собирать рабочий билд. Но есть ещё пару интересных моментов. Далее о номере билда и немного о правильной настройке проекта под WebGL.

Номер билда

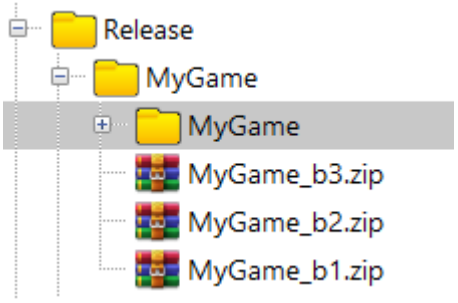
PluginYG2 хранит информацию о билде в файле PluginYourGames/Editor/**BuildLogYG2.txt**. Туда записывается номер билда. Например, если вы собираете проект при импортированном PluginYG третий раз, то номер билда будет = 3.

Номер билда отображается в следующих местах:

- В логе Unity Editor после сборки проекта
- В названии zip архива билда (приписка, например «b3»)
- В консоли браузера (в первых логах)
- В углу экрана игры при тестировании билда (подробннее далее)

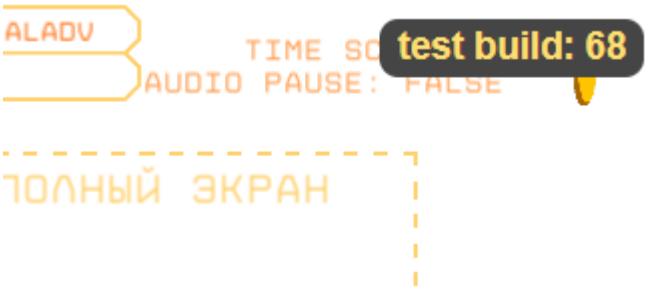
Для обнуления информации о билде можно удалять файл **BuildLogYG2.txt**.
Разрешается удалять и всю папку PluginYourGames/**Editor** для отчищения кеша плагина.

После сборки проекта, если включена галка **Arhiving Build** в настройках плагина в разделе **Basic Settings**, то билд игры будет автоматически заархивирован в формате **zip**, чтобы каждый раз не архивировать вручную. PluginYG не удаляет оригинальный билд и не помещает внутрь него архив, т.к. плагин не нарушает какую-либо базовую логику Unity при сборке билда. По этому, расположение созданного архива может показаться странным. Для удобства — внутри папки, в которую собираетесь построить билд — создайте ещё одну папку с таким же названием игры. И уже внутрь неё собирайте билд игры:



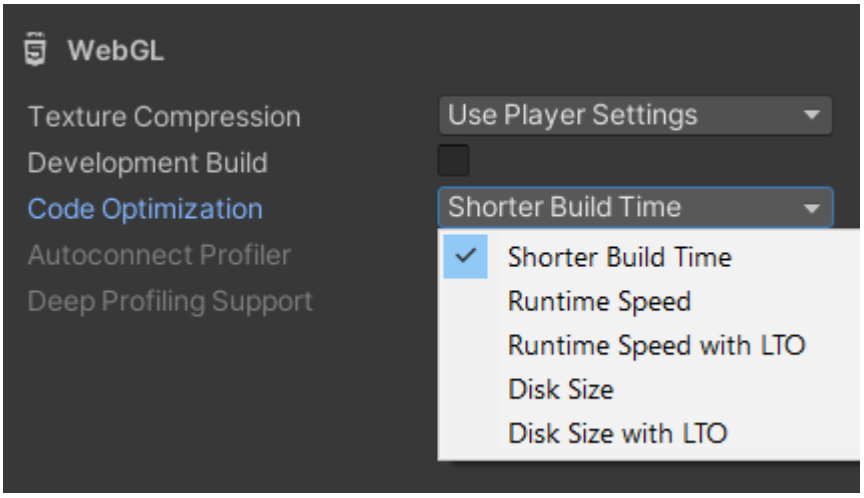
Настройки проекта

В настройках плагина, в разделе **Template** опция **Developer Build** поможет вам при тестировании билда игры. При использовании данной опции, в сборку попадут скрипты отображающие номер билда в углу экрана игры. Это даёт возможность мгновенно убедиться, что на сервере платформы актуальный билд.
Также, отображение такой плашки сигнализирует о том, что данный билд является не финальным, что поможет вспомнить о изменении настроек проекта под финальный билд.



На плашку с номером билда можно нажать, чтобы она исчезла.
Перед финальным билдом необходимо выключить опцию **Developer Build**. Также в **Build**

Settings для WebGL есть опция **Code Optimization**:



Для экономии времени при сборке тестового билда установите параметр **Shorter Build Time**. Для релизного билда **Runtime Speed with LTO**.

> **Важно !**
>
> Выбор сборки с LTO оптимизациями увеличивает время сборки в несколько раз. В некоторых случаях в десятки раз.
> Так что используйте этот режим только для финальной сборки проекта.
> Подробнее смотрите в официальной документации.

Другие основные настройки проекта под WebGL плагин устанавливает автоматически, если вы это ему позволили (смотрите подробнее в разделе Начало работы).
По умолчанию не применяется опция Color Space. Для WebGL рекомендуется использовать значение **Gamma**. Игры для IOS и некоторые другие устройства поддерживают только Gamma режим. Это же касается и для опции **Auto Graphic API** (рекомендуется включить).

Из основных рекомендаций ещё можно отметить опцию **DSP Buffer Size** для звуков — настройка оптимизации звука, которая ещё может помочь убрать проблему с посторонними шумами, если такая имеется. А в настройках самих звуков для WebGL параметр Load Type необходимо выставлять на **Decompress On Load**. Иначе на мобильных устройствах в шторке будет отображаться Media Player, будто звуки вашей игры — это прослушивание музыки.

Ещё проблема у новичков — не отображается шрифт в билде на WebGL. Нельзя использовать стандартный Unity шрифт, замените его. При возникновении трудностей читайте раздел «Часто задаваемые вопросы» (ещё не готов).

Советуем прочитать отличную статью о настройке и оптимизации проекта.

Часто Задаваемые Вопросы (F.A.Q)

> **Установил плагин, но пункт Tools в меню не появился. Что делать?**

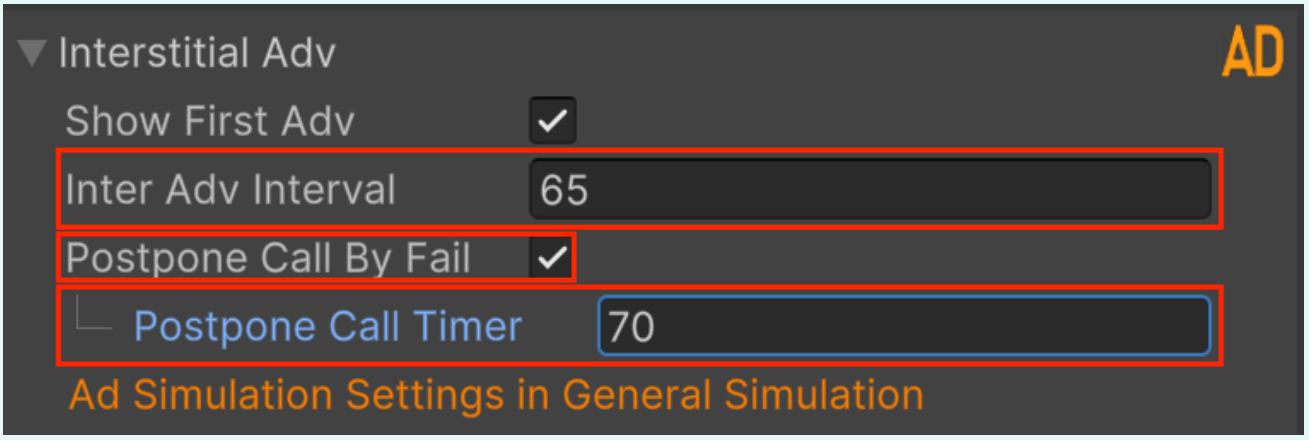
Устраните все ошибки в консоли чтобы Unity могла скомпилировать скрипты.

Зацикливается таймер перед показом рекламы. Как исправить?

Попробуйте выставить следующие значения в разделе "Interstitial Adv" настроек плагина:

- **Inter Adv Interval:** 65
- **Postpone Call By Fail:** true
- **Postpone Call Timer:** 70

👇 Скриншот настроек



Как установить слой логотип или задний фон для игры?

Следуйте этой инструкции:

1. Создайте папку "Images" внутри папки "WebGLTemplates/YandexGames";
2. В созданную папку "WebGLTemplates/YandexGames/Images" поместите необходимые изображения;
3. Изображения должны иметь названия "logo" для логотипа и "background" для заднего фона;
4. В настройках плагина, в разделе "Templates" укажите используемые форматы изображений для логотипа и заднего фона.

Хочу зафиксировать соотношение сторон в игре. Как это сделать?

Следуйте этой инструкции:

1. В разделе настроек плагина "Templates" установите флаг на пункте "Fixed Aspect Ratio"
2. В пункте Aspect Ratio установите необходимое соотношение сторон. Например для альбомной ориентации это "16/9", а для портретной "9/16"
3. (Опционально) Если требуется фиксированное соотношение сторон и на мобильных устройствах, то снимите флаг с пункта "Disable For Mobile"
4. (Опционально) Если требуется заполнить фон вне фиксированной части игры - поставьте флаг Fill Background и выберите между заполнением изображением или градиентом.

Пришло замечание от модерации "В текущей версии игры не реализовано автоматическое определение языка через SDK, что нарушает п. 2.14 требований". Что делать?

Установите модули "Localization" и "Auto Translate Langs".

Я использую движок Construct\Unreal\Godot\Cocos и у меня возникли трудности с ЯндексSDK. Сможете помочь?

PluginYG написан для игрового движка Unity и мы не знаем чем вам помочь.

Как проверить смену языка в редакторе?

В настройках плагина, в разделе "General Simulation", впишите нужный язык в поле "Language". Например "en" для английского или "tr" для турецкого языка.

Как проверить смену языка в черновике Яндекс Игр?

Вот как это сделать:

1. Запустите черновик с debug-панелью;
2. В debug-панели нажмите кнопку SDK mocks;
3. Выберите нужный язык в списке "Choose Game Lang"

Загрузил архив с игрой в черновик и при переходе по ссылке Яндекс пишет что такой игры нет. Как запустить игру?

Заполните все поля черновика. **Скриншоты и видео заполнять необязательно.**

Нужно-ли, сейчас, при сборке активировать флаг "*Decompression Fallback*"?

Для Яндекс Игр эта опция больше не нужна;
Для остальных платформ уточняйте этот момент в документации.

Опубликовал игру, но в ней нет стики баннеров. Как их включить?

Ждите. Баннеры появляются в течении нескольких часов, после публикации игры.

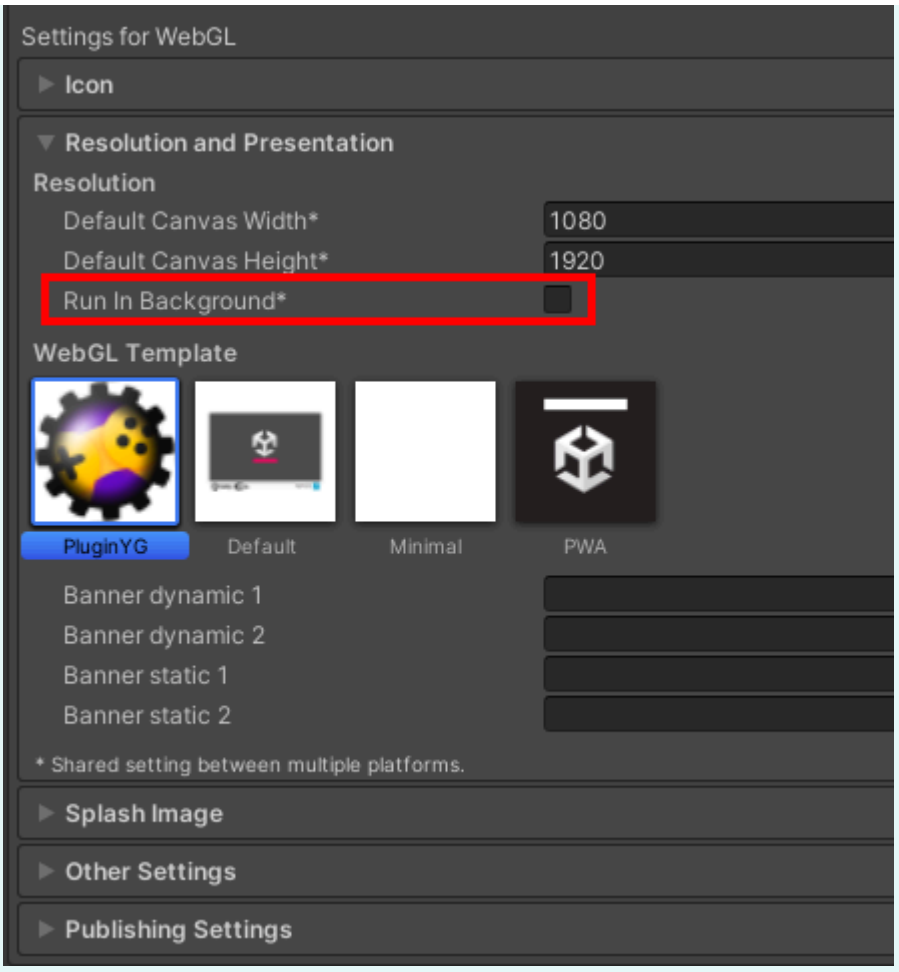
Установил плагин и в консоли появилась ошибка связанная с методом "FindAnyObjectByType()". Как исправить?

Это связано с использованием устаревшей версии Unity где этого метода ещё нет.
Полный список поддерживаемых плагином версий Unity:

- 2021.3.18+
- 2022.3+
- 2023.2+
- Unity 6 (6000.0)

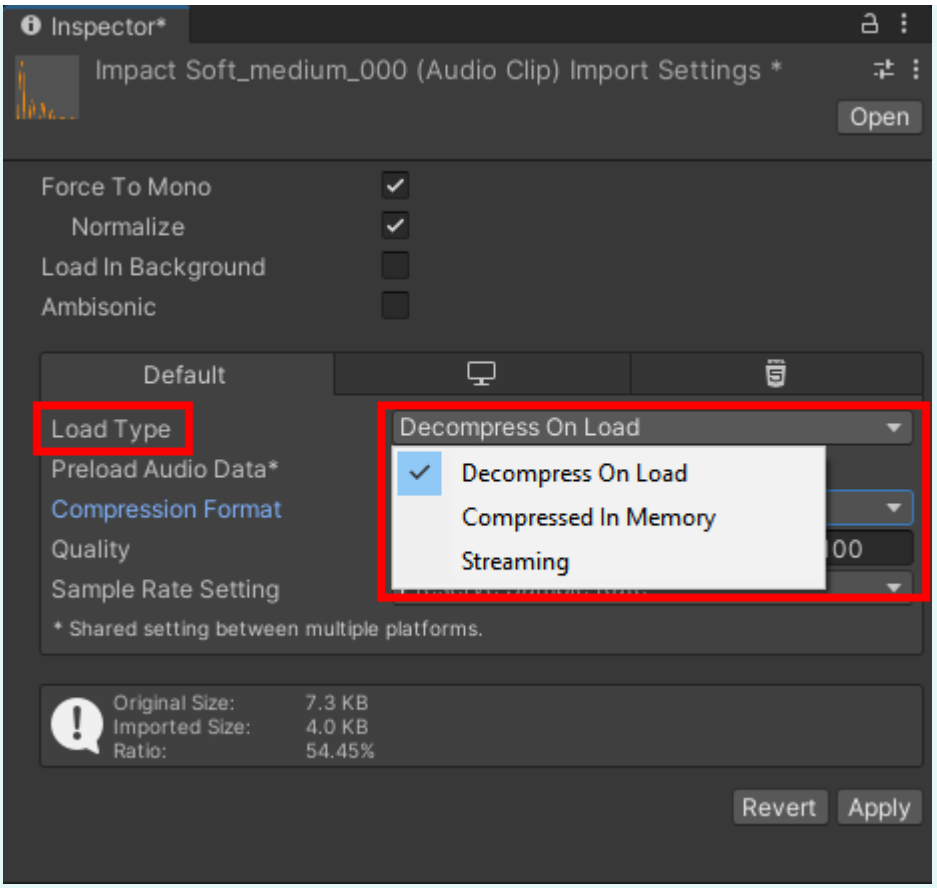
При сворачивании\переключении вкладки с игрой, звук продолжает воспроизводиться. Как исправить?

Убрать галочку на пункте "Run In Background", в "Player Settings\Resolution and Presentation".



Плеер (звуковой), из игры запущенной в браузере, отображается в панели уведомления (шторке) телефона. Как исправить?

У звуковых файлов (звуков), в Unity, установить параметр "*Load Type*" -> "*Decompress On Load*".



Нужно ли мне что то прописывать в index.html?

Нет. Для работы плагина Вам не нужно заходить в html и css скрипты. Но для собственного расширения функционала, конечно, вы можете что то изменять в любых скриптах плагина.

Игра в черновике становится "мыльной". Как исправить?

Откройте файл "index.html" и удалите в нём строку "config.devicePixelRatio".



Не отображается шрифт в WebGL билде. Как исправить?

Если вы используете Text Legacy, то в нём нельзя использовать дефолтный Unity шрифт, который по умолчанию назначен в компоненте текста. Нужно использовать какой-либо сторонний шрифт, который находится в проекте как файл, и назначить шрифт в компоненты текста.

Я собрал билд игры, загрузил на Яндекс Игры, но она не запускается. Что делать?

- Проверьте что вы не используете кириллицу в пути к папке в которую собирается проект;
- Убедитесь, что Вы заливаете именно тот билд, что планировали.
- Откройте консоль браузера (Chrome: F12 или CTRL+SHIFT+I. Вкладка "Console") и посмотрите внимательно на строки выделенные красным цветом и изучите их. Одна из этих строк, ваша ошибка и если вам не хватает опыта, то покажите эти ошибки в официальной группе

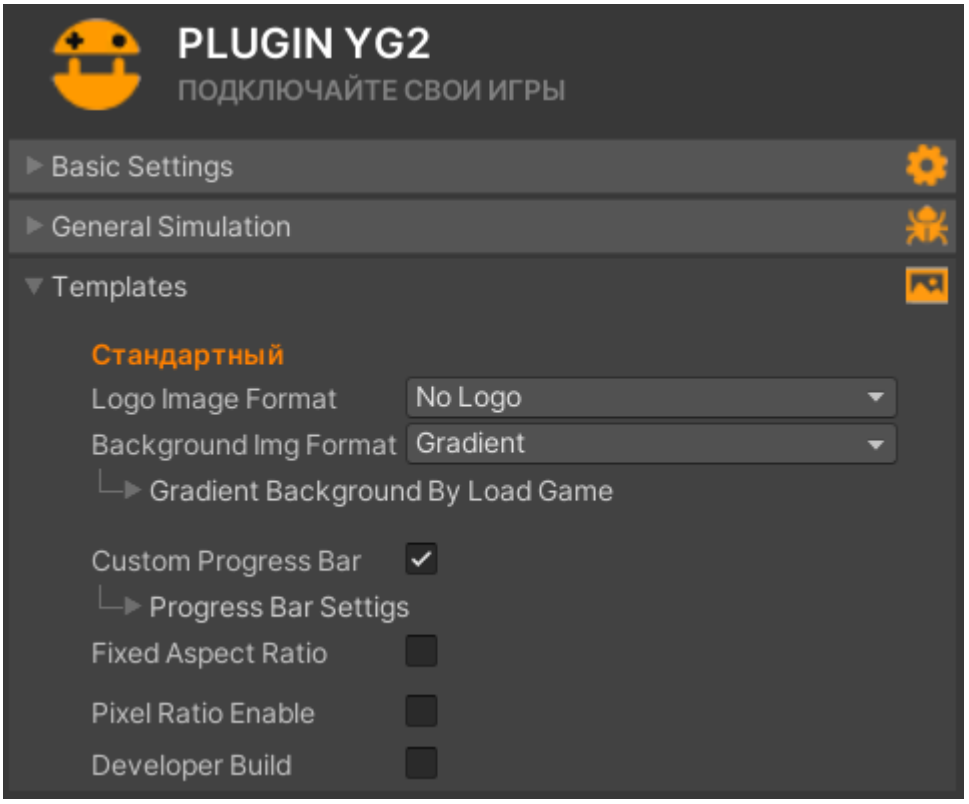
Оглавление раздела:

1. Шаблон
2. Загрузка игры (Logo и Background)
3. Фиксирование экрана
4. Создание своей реализации
5. Примеры

Шаблон



Для платформы WebGL плагин редактирует файлы index.html и style.css. В настройках PluginYG2 есть раздел **Templates**. Он предназначен для хранения опций различных шаблонов. Все опции стандартного темплейта применимы к платформе Yandex Games.



У вас есть возможность редактировать стандартный шаблон:

- **Дизайн загрузочного экрана**. Меняйте внешний вид полосы загрузки, добавляйте лого игры, изображение на задний фон или градиент.
- **Фиксирование соотношения сторон**. Например, для игры с портретной ориентацией экрана. Или если динамическое изменение экрана вызывает проблемы, можно его статично зафиксировать.
- **Оптимизация игры для мобильных устройств**, путём снижения разрешения (опция Pixel Ratio).

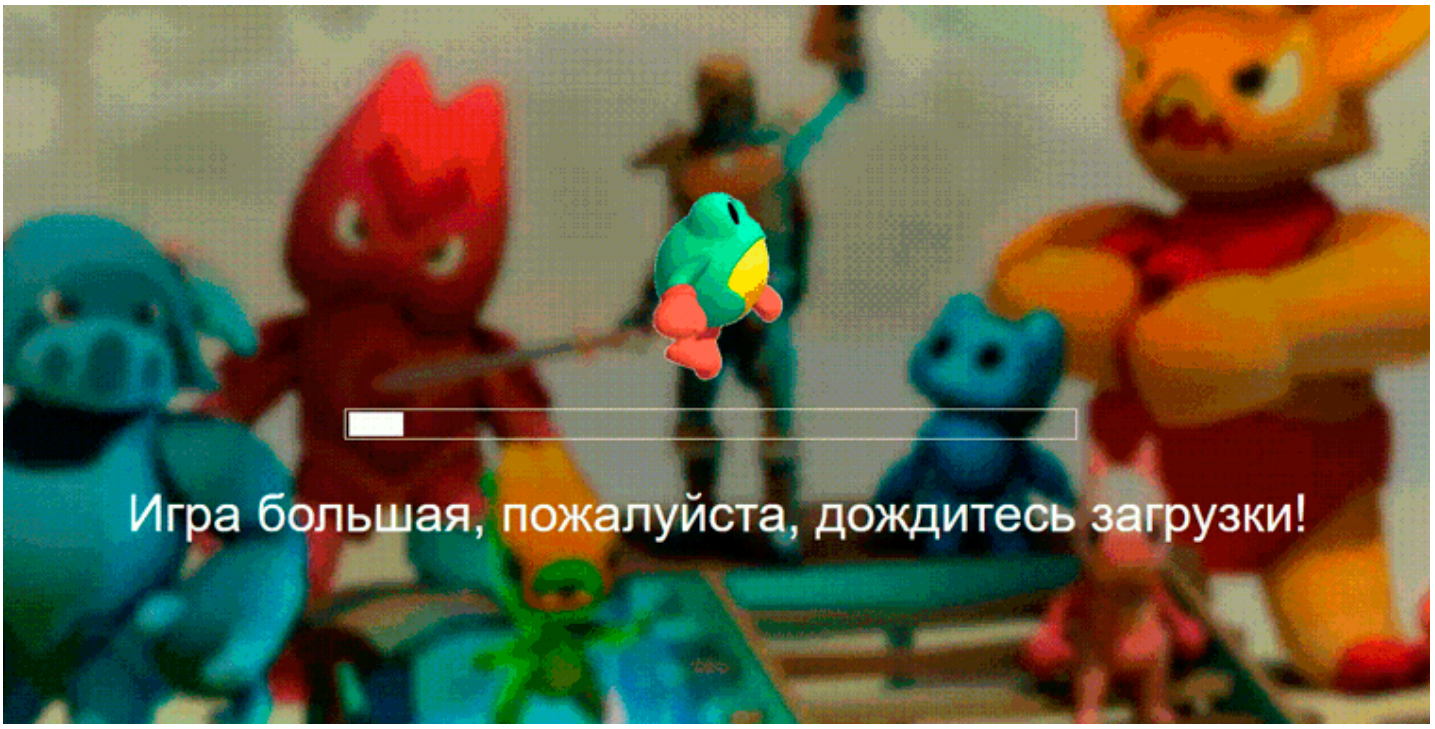
- **Отслеживание номера билда на сервере платформы**, чтобы убедиться в актуальности загруженной сборки. За это отвечает опция Developer Build и также она сигнализирует о том, что загруженный билд является тестовым, и перед загрузкой финального билда необходимо выбрать настройки проекта подходящие для релиза.

Также у вас есть возможность создать собственные настройки и реализацию для редактирования html и css. Или просто опции отображаемые в настройках PluginYG, которые можно использовать для чего угодно.

Ниже мы рассмотрим основные популярные настройки шаблона. С остальными параметрами вы сможете ознакомиться из всплывающих подсказок.

Загрузка игры (Logo и Background)

При загрузке игры можно отобразить изображения лого и заднего фона. Пример:



Для отображения logo или background изображений:

1. Создайте папку `Images` внутри **WebGLTemplates/YandexGames**.
2. В созданную папку WebGLTemplates/YandexGames/**Images** поместите изображения.
3. Изображение лого — должно называться **logo**, а задний фон — **background**.
4. В настройках **Templates** укажите формат изображений.

Фиксирование экрана

Опция **Fixed Aspect Ratio** — фиксирует соотношение сторон экрана в зависимости от того, как вы укажете параметр **Aspect Ratio**.
Например:
16/9 — фиксировать экран игры **горизонтально**.
9/16 — фиксировать экран игры **вертикально**.

> **Обратите внимание**

Всё что написано в этом разделе ниже - второстепенно.

Создание своей реализации

Создайте папку со своей реализацией внутри папки PluginYourGames/**Modules**. Тогда PluginYG2 определит вашу папку как новый модуль.
Работа с шаблоном зачастую касается только Unity Editor. Чтобы лишний код и файлы не попали в сборку игры, помещайте файлы в созданную внутри вашего модуля папку **Editor** или по обстоятельствам используйте директиву препроцессора `#IF UNITY_EDITOR` в C# скриптах.

Создание параметров

Чтобы создать параметры в разделе **Templates** в настройках плагина — создайте новый скрипт и вставьте в него следующий код:

```
namespace YG
{
    public partial class InfoYG
    {
        public partial class TemplateSettings
        {
            // Заголовок:
            [HeaderYG("My template")]

            // Ваши параметры. Например:
            public bool myParam;

            // Можете создать свой класс шаблона для отделения параметров
        }
    }
}
```

Теперь к параметру **myParam** можно обращаться в коде:
`YG2.infoYG.Templates.myParam`

Файл для копирования

Можно создать файл из которого будет копироваться текст и вставляться в index.html или style.css. Название и формат такого файла может быть каким угодно. Но если вы создавали модуль, файл нужно поместить внутри папки с модулем в следующую директорию: **Scripts/Editor/CopyCode**. То есть внутри модуля создаём эти папки, и внутри папки **CopyCode** уже помещаем файл для копирования. Это нужно для метода `FileTextCopy`, который ищет папку модуля по его названию и далее по директории с папкой CopyCode.

Чтобы удобно редактировать скрипт из файла копирования в каком-либо редакторе кода — можно указать соответствующий скрипту формат файла. Например, если в файле будет код на JS, лучше формат файла выбрать js.

Код для изменения шаблона

Создайте новый скрипт, вставьте и отредактируйте этот код:

```
#if YandexGamesPlatform_yg // По желанию определите платформу
namespace YG.EditorScr.BuildModify // Скрипт компилируется только в Unity Editor
{
    public partial class ModifyBuild
    {
        // Если создали модуль, назовите метод именем вашего модуля. Иначе имя метода может быть любое.
        public static void MyMethod()
        {
            // В этом методе редактируем index.html или style.css файлы

            // Пример использования созданного параметра в настройках Templates
            if (YG2.infoYG.Templates.myParam)
            {
                // Копирование текст из файла

                // Если создавали модуль, используйте этот метод копирования. Укажите название файла копирования.
                string copyCode = FileTextCopy("Scripts/Editor/MyCopyFile.js");

                // Если НЕ создавали модуль, используйте этот метод. Укажите путь к файлу копирования внутри папки Assets.
                string copyCode = ManualFileTextCopy("MyFolder/Scripts/Editor/MyCopyFile.js");

                // Можете заменять какие то значения в скопированном тексте
                copyCode = copyCode.Replace("___TEXT_REPLACE___", YG2.infoYG.Templates.myReplaceText);

                // Вставляем текст в файл index.html
                AddIndexCode(copyCode, CodeType.JS);
            }
        }
    }
}
#endif // Конец определения платформы
```

В примере используется `CodeType.JS` — это означает, что код вставиться в index.html внутрь тега `<script>`.
CodeType можно указать `Head`, `Body` и другие, тогда код вставиться в соответствующее место в файле index.html.
Есть ещё CodeType-ы инициализации. Очерёдность выполнения инициализации: `Init0, Init1, Init2, Init, Start`.
Вышеописанные CodeType работают для WebGL шаблона YandexGames. Место для вставки нового кода в index.html определяется с помощью комментариев внутри файла index.html:

```
<!-- Additional head modules -->
<!-- Additional body modules -->
// Additional script modules
// Additional start modules
// Additional init0 modules
// Additional init1 modules
// Additional init2 modules
// Additional init modules
```

Для YandexGames работают все CodeType. Для сторонних шаблонов есть CodeType `HeadNative` и `BodyNative`. Они могут помочь вставить новый код без редактирования файла index.html. Эти CodeType ищут место для вставки не по комментариям, а по тегам html синтаксиса.

В скрипте для изменения шаблона, внутри класса `ModifyBuild` вы можете воспользоваться полями `indexFile` и `styleFile`. Поле `indexFile` — это весь текст файла index.html. `styleFile` — соответственно весь текст файла style.css. Если нужно напрямую отредактировать файл, просто редактируйте данные поля. После редактирования ничего применять не нужно. Например, вы можете удалить вообще всё, что есть в файле: `indexFile = "";`. И файл index.html станет пустым. Таким образом можно вставить новый код в файл style.css, например:

```
namespace YG.EditorScr.BuildModify
{
    public partial class ModifyBuild
    {
        string copyCode = YG2.infoYG.Templates.myCSS;
        styleFile += $"\n{copyCode}";
    }
}
```

Примеры

Примерами являются скрипты из платформы Яндекс Игры. Они используют опции стандартного Template. Данные скрипты находятся в папке

PluginYourGames/Platforms/YandexGames/Scripts/Editor. Эту папку можно даже удалить, тогда изменения шаблона не применятся.

Оглавление раздела:

1. Переход с PluginYG 1.x на 2.0

Переход с PluginYG 1.x на 2.0

Переведите проект с PluginYG первых версий на новый Plugin Your Games 2.0. Переход будет состоять из нескольких этапов.

При переходе префаб **YandexGame** на сценах останется и все ссылки сохранятся. Но для второй версии данного префаба уже нет. Подробнее читайте в разделе Начало работы.

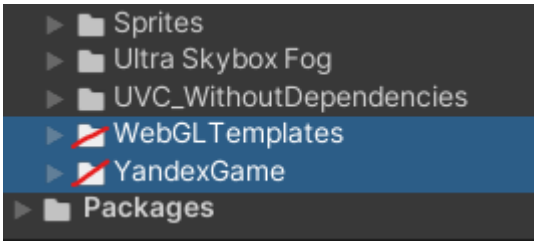
1. Бэкап

Внимание

- Рекомендуется сделать резервную копию проекта перед обновление плагина с версии 1.x до 2.x. (можете его просто заархивировать).
- Также можно сразу обновить проект до последней актуальной и стабильной версии Unity.

2. Удаление старого плагина

Откройте новый проект (копию) и удалите полностью старый плагин:

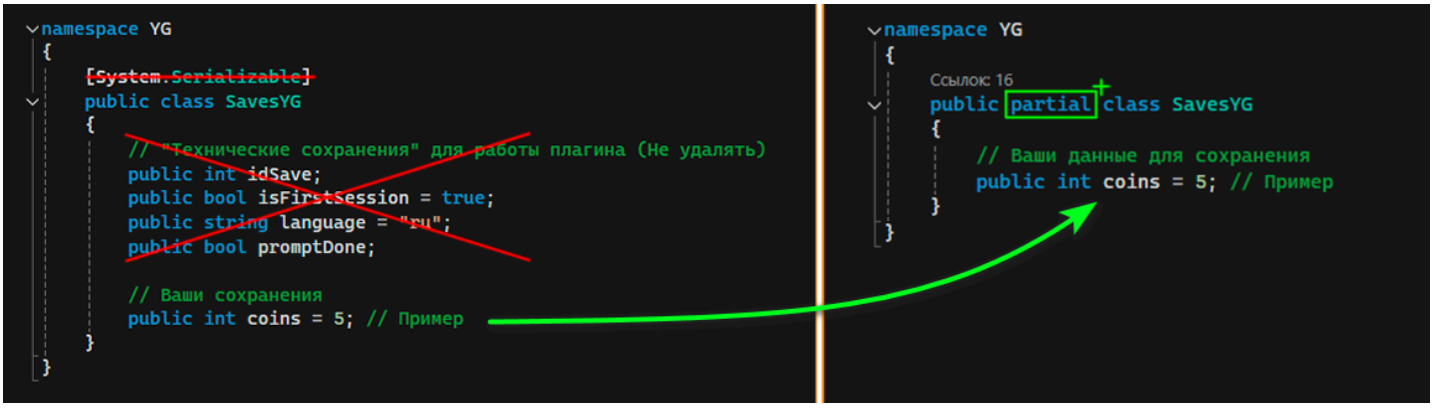


3. Импортируйте пакет для перехода

Скачайте и импортируйте пакет.
Не обращайте внимания на ошибки, мы устраним их позже.

4. Интеграция сохранений

Если в проекте использовался функционал сохранений от PluginYG, то из бэкапа проекта достаньте скрипт **SavesYG.cs**, который находиться в папке "**YandexGame/WorkingData**". Поместите его куда угодно в вашем проекте, название тоже может быть любое. Код скрипта нужно немного скорректировать:



Красное — удалить. Зелёное — вставить.
Подробнее об изменениях модуля сохранений можете узнать в разделе Storage.

5. Скройте ошибки

Если на данном этапе вы всё ещё наблюдаете ошибки — временно закомментируйте их. Не закрывайте скрипты с закомментированным кодом, чтобы потом можно было легко их найти и отредактировать. Библиотеку `UnityToolbag` можно смело удалять, атрибут `ConditionallyVisible` заменить на `YG.NestedYG`. Он не будет вызывать ошибок после импорта второй версии плагина.

6. Импорт плагина v2.0

Убедившись в отсутствии ошибок в проекте, импортируйте PluginYourGames2.

> **Напоминаем**😉
>
> Актуальную версию можно найти в нашей группе поддержки, в Telegram .

7. Восстановление файлов шаблона

Если использовали изображения **icon** и **background**, возьмите их из резервной копии проекта и поместите в папку WebGLTemplates/YandexGames/**Images**.

> **Обратите внимание**🤔
>
> Папку "*Images*" предварительно нужно создать.
> Далее в настройках плагина есть раздел ***Template***, в нём настройте формат изображений *icon* и *background*.

8. Импорт модулей

Импортируйте все необходимые для вашего проекта модули.
Пояснения к основным модулям:

- InterstitialAdv и RewardedAdv - Модули **межстраничной рекламы** и **рекламы за вознаграждение**.
- EnvirData - Данные окружения пользователя (например данные об устройстве на котором запущена игра, данные о языке пользователя и т.п.).
- Authorization - Данные игрока, такие как его ник или аватар. Также он необходим для облачных сохранений, таблиц лидеров и внутриигровых покупок.
- Storage - Необходим если используется функционал сохранений.
- Localization и Auto Translate Langs — Необходим если в проекте используется функционал локализации от PluginYG. В частности инструменты автоматического перевода текста.
- Если не понятно по остальным модулям, почитайте о них в соответствующих разделах документации.

Если не удаётся открыть окно контроля версий и импортировать модули из-за ошибок, вы можете скачать пакет содержащий сразу все доступные модули.

9. Устранение ошибок и настройка

Если ранее были ошибки, которые мы комментировали — расскомментируйте их и замените код на тот что соответствует новой версии плагина

Если вы использовали компонент для паузы игры - **Viewing Ads YG**:
В новом плагине замена этому компоненту - опция **Auto Pause**. В настройках плагина в разделе **Basic Settings** отключите этот параметр, чтобы избежать конфликтов.

Опции вызывающей рекламу при переключении сцены больше нет по определённым причинам. Необходимо самостоятельно разметить места для вызова рекламы при смене сцены. Метод рекомендуется выполнять при нажатии на кнопку, т.к. платформа Яндекс Игры требует, чтобы рекламы была вызвана во время пользовательского действия.

На сценах и в префабах не должно ничего слететь, кроме внутриигровых покупок. Проверьте их и ознакомьтесь с новой документацией. Там произошли изменения, которые не удалось адаптировать.

Настройте плагин в соответствии с проектом. Большинство опций остались прежними, что и в первой итерации PluginYG.

Оглавление раздела:

1. Платформы
2. Настройки платформ
3. Компонент PlatformEventsYG2
4. Работа с кодом

5. Опциональные методы
6. Создание опций
7. Создание платформы

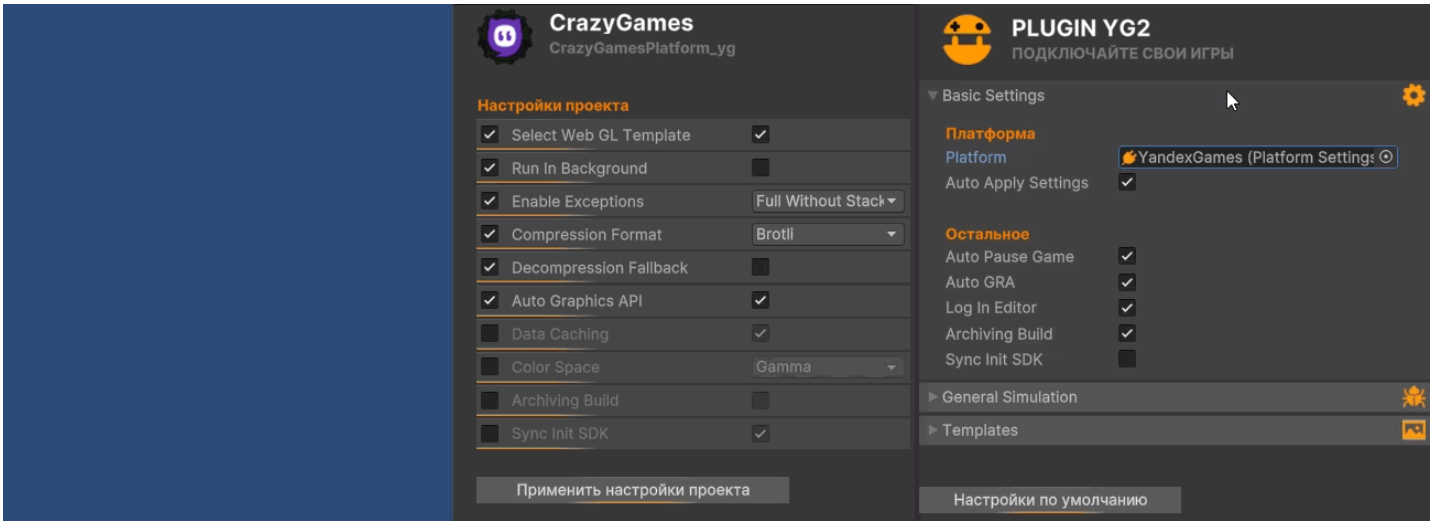
Платформы



Plugin Your Games 2.0 — мультиплатформенный. Переключаются в два клика!
При переключении платформы устанавливаются соответствующие настройки проекта. Всё гибко настраивается и можно даже создавать свои опции, которые будут вносить изменения в настройки проекта. Или опции могут просто меняться в зависимости от платформы, чтобы в коде получать разные значения для разных платформ.
Если вы можете интегрировать SDK какой-либо площадки в свою игру, то можете попробовать создать свой модуль платформы для PluginYG2.
Бесплатно предоставляется сам PluginYG2 и платформа **YandexGames**. Остальные готовые платформы я публикую на своём на Boosty всего за **200р**.

Настройки платформ

В настройках *Plugin Your Games* → ***Basic Settings*** есть поле ***Platform*** — это выбор платформы и ссылка на её настройки.
Настройки платформы удобнее всего открыть в отдельном окне — для этого нажмите правой кнопкой мыши по ссылке → ***Properties...***.



В данном gif примере 👆 вы видите настройки платформы.

Сверху есть название платформы (например **YandexGames**), которое можно получить из поля `YG2.platform`.
Под ним название дефайна платформы (например **YandexGamesPlatform_yg**) — используется для Scripting Define Symbols.

Настройки платформы необходимы, чтобы выбрать определённые настройки проекта для каждой платформы. Это как пресет настроек проекта. В этом окне дублируются некоторые

параметры проекта, для которых может потребоваться установить разные значения для разных платформ. Например, для платформы Яндекс Игры мы показываем рекламу при старте игры, а для платформы CrazyGames не показываем.

Получается, параметры в данном окне переопределяют параметры в проекте. Но переопределяться они будут, только если поле опции в настройках платформы активно. За это отвечает флаг слева от поля. Когда поле деактивировано, оно становится серым и не интерактивным. Деактивированное поле не будет переопределять значения оригинального параметра и никак не повлияет на проект.

Если нажать на кнопку «**Применить настройки проекта**», то значения оригинальных полей будут переопределены значениями из настроек платформы. Вам не нужно нажимать эту кнопку перед сборкой каждый раз. Настройки проекта будут применяться автоматически перед сборкой билда и при переключении платформы в настройках PluginYG (Если стоит флаг **Auto Apply Settings**, по умолчанию включёно).

> ⚠ **Сборка проекта под Android (если не используются модуль платформы для мобильных устройств)**
>
> Удалите платформу из поля "*Platform*" в разделе "*Basic Settings*" настроек плагина. В таком случае, практически никакой код плагина не попадёт в билд игры, только малая доля, которая может пригодится и для мобильных устройств. Например, локальные сохранения продолжат свою работу.
> Ниже будет рассмотрено, как создать свою платформу. Таким образом, вы можете как минимум подключить к плагину монетизацию, которую вы испротзуете для мобильных устройств. Тогда можно будет менять платформу в два клика и собирать игру под Android и под WebGL без корректировок проекта.

Компонент PlatformEventsYG2

Данный компонент может выборочно выполнять различные методы в зависимости от текущей платформы. Таким образом, можно скрыть некоторые элементы для определённого списка платформ:



В этом примере для **Yandex Games** видно все элементы, а для **Crazy Games** они скрываются, т.к. в **CG** эти модули не работают из-за отсутствия такого функционала у площадки.

Чтобы сделать также, выберите платформы для которых будут выполняться методы и привяжите какие угодно методы. В примере используется деактивация игрового объекта, такой метод привязывается по умолчанию.
Затем, если есть потребность, можно выровнять элементы интерфейса, чтобы они адаптировались под изменения. Например, с помощью компонента **Grid Layout Group**.

Работа с кодом

Нечто аналогичное как с компонентом `Platform Events YG2` можно делать и в коде, используя вышеупомянутые поле `YG2.platform` и дефайны. Например:

```
using UnityEngine;
using YG;

public class MyClass : MonoBehaviour
{
    public void MyMethod()
    {
        if (YG2.platform == "CrazyGames")
        {
            gameObject.SetActive(false);
        }

#if CrazyGamesPlatform_yg // В билд попадёт только один из SetActive()
        gameObject.SetActive(false);
#else
        gameObject.SetActive(true);
#endif
    }
}
```

Опциональные методы

Есть методы с разной реализацией для разных платформ, что позволяет использовать один метод, который будет вести себя по разному для разных платформ.

Метод `FirstInterAdvShow` - например, **CrazyGames** требует, чтобы самый первый показ рекламы при открытии первого уровня/локации проигнорировался. Метод `FirstInterAdvShow` проигнорирует первый показ рекламы для **CrazyGames**, а для **YandexGames** покажет рекламу в любом случае. После того, как реклама будет показана с помощью данного метода, информация об этом сохранится и реклама будет показана в любом случае, даже при после перезагрузки страницы. Чтобы сбросить эти сохранения, в Unity Editor необходимо отчистить Player Prefs. Для тестирования в вебе откройте игру в режиме инкогнито.

Метод `OtherInterAdvShow` покажет рекламу на **YandexGames**, для **CrazyGames** проигнорирует. В CrazyGames запрещается показывать рекламу при переходе в главное меню

игры и в некоторых других местах. Для таких моментов можно использовать метод `OtherInterAdvShow`, чтобы использовать разные его реализации для разных платформ.

Метод `LoadInterAdv` будет подгружать рекламу для платформ, в которых это необходимо. Для **YandexGames** и **CrazyGames** такого нет, по этому метод у них пустой. Такой же метод есть для рекламы за вознаграждение `LoadRewardedAdv`.

Все опциональные методы содержатся в классе `OptionalPlatform`. Пример вызова такого метода:
`YG2.optionalPlatform.FirstInterAdvShow();`

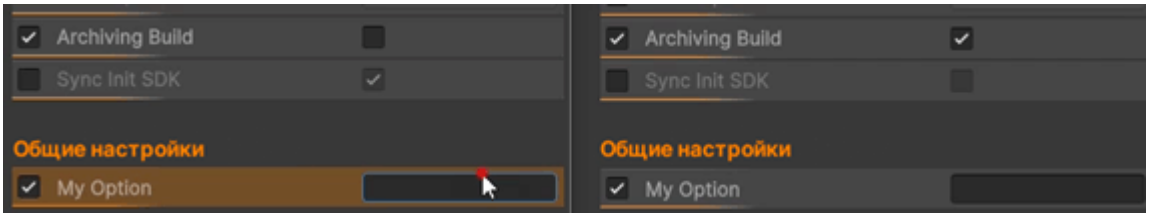
Опциональные методы также можно выполнять без кода через компонент EventsYG2.

> **Обратите внимание**
>
> Всё что написано в этом разделе ниже - второстепенно.

Создание опций

Можно создать новые опции для двух разделов:

1. Такие же опции как мы рассматривали ранее. Их можно сделать с флагами активности и без. Это опции, которые содержат разные значения для разных платформ.
2. Общие настройки, которые размещаются в отдельном разделе. У них будет одинаковое для всех платформ.



И в первом и во втором случае нужно создать скрипт, файл которого можно назвать как угодно и таких файлов сделать можно сколько угодно и к ним применимы атрибуты: `[ApplySettings]`, `[SelectPlatform]`, `[DeletePlatform]`.

> **Создание опций**
>
> Пример без флага активности.
> Создание поля с разными значениями для разных платформ:

```
namespace YG.Insides
{
    public partial class ProjectSettings
    {
        public string myOption; // Добавление опции
    }
}
```

Обращение к созданному полю:
`YG2.infoYG.platformOptions.myOption`

Также создаётся поле общих настроек, только класс другой:

```
namespace YG.Insides
{
    public partial class CommonOptions
    {
        public string myCommonOption; // Добавление опции
    }
}
```

Обращение к созданному полю:
`YG2.infoYG.common.myCommonOption`

Следующий пример — с добавлением флага активности:

```
namespace YG.Insides
{
    public partial class ProjectSettings // Или CommonOptions
    {
        public string myOption; // Добавление опции

        &#91ApplySettings&#93
        public void MyApplySettings()
        {
            // Сработает при применении настроек платформы. Пример:

            // Получаем ссылку на флаги опций
            PlatformToggles toggles = YG2.infoYG.platformToggles;

            // Если флаг активный - применяем опцию
            if (toggles.myOption == true)
                myOption = "apply";
        }

        &#91SelectPlatform&#93
        public void MySelectPlatform()
        {
            // Сработает при выборе платформы
        }

        &#91DeletePlatform&#93
        public void MyDeletePlatform()
        {
```

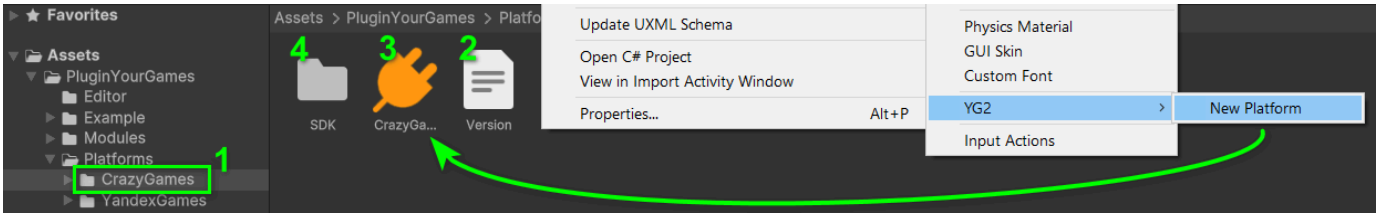
```
            // Сработает перед удаление платформы из проекта
        }
    }

    // Добавьте флаг активности
    public partial class PlatformToggles
    {
        public bool myOption; // Флаг должен называться также как опция
    }
}
```

Создание платформы

Для примера возьмём платформу **Crazy Games**.

1. Создайте папку с именем платформы в *PluginYourGames/**Platforms***. Название нужно подобрать тщательно, оно много где будет использоваться.
2. Создайте текстовый документ `Version.txt`, в нём запишите версию модуля в таком формате: `v1.0`.
3. Создайте файл настроек платформы: ***ЛКМ → Create → YG2 → New Platform***. Настройте по желанию.
4. Создайте папку `SDK` — она будет игнорироваться для компилятора, если платформа не активна. В данную папку поместите SDK площадки, если таковой имеется. И все скрипты платформы содержите в папке SDK.



Если вы создаёте WebGL платформу, то шаблон нужно назвать также как назвали платформу. Шаблон — это "index.html" и другие файлы для WebGL билда. Шаблон должен находиться в папке WebGLTemplates. Сделайте по аналогии с YandexGames.

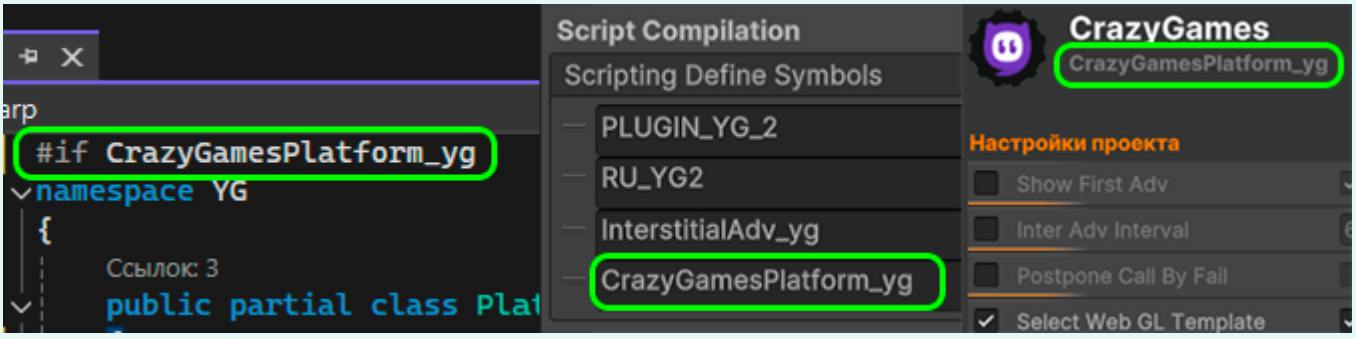
В настройках PluginYG2 переключитесь на созданную платформу.
В SDK можно создать папку ScriptsYG, чтобы обозначить скрипты для PluginYG. В этой папке создайте базовый скрипт, в котором будет логика инициализации. Я назову скрипт CG_BasicAPI. Вставьте в него шаблон:

```
#if CrazyGamesPlatform_yg
namespace YG
{
    public partial class PlatformYG2 : IPlatformsYG2
    {
        public void InitAwake() { }
```

```
        public void InitStart() { }
        public void InitComplete() { }
        public void GameplayStart() { }
        public void GameplayStop() { }
        public void HappyTime() { }
    }
}
#endif
```

Измените дефайн

`CrazyGamesPlatform_yg` — это дефайн (директива препроцессора). Его можно встретить в следующих местах:



Измените его название на то, что указано в настройках платформы. Когда название верное, код внутри `#if код #endif` не серый. Серый он становится тогда, когда дефайна нет в списке Scripting Define Symbols. Это ещё от IDE (редактор кода) зависит, конечно. Если в списке он есть и название верное, но код серый — перезагрузите IDE.

О интерфейсе и методах

Скрипт выше реализует интерфейс IPlatformsYG2.
Представленные методы переопределяют стандартную реализацию. Для всех методов в интерфейсе IPlatformsYG2 есть стандартная реализация. У представленных методов реализация пустая (кроме `InitAwake`, его пока не трогать). По этому их можно удалить или оставить так, ничего не изменится. В примере предоставлены самые важные методы, которые есть в PluginYG без отдельных модулей.

По простому, это работает так:
Например, нам нужно выполнить метод SDK платформы `GameplayStart`. В одноименном представленном методе внутри фигурных скобок прописываем реализацию:

```
public void GameplayStart()
{
    CrazySDK.Game.GameplayStart();
}
```

Можно сократить и прописать так:

```
public void GameplayStart() => CrazySDK.Game.GameplayStart();
```

Инициализация

Чаще всего SDK предоставляет асинхронный метод инициализации. После того, как игра загрузилась — SDK ещё не готов и нельзя пользоваться его функциями, надо подождать пока SDK инициализируется.
Давайте рассмотрим два варианта:

1. Когда SDK при запуске игры сразу готов (как у платформы YandexGames на PluginYG. Она адаптирована под два варианта). В этом случае нужно только удалить метод `InitAwake`.
2. Второй случай, когда необходимо дождаться инициализации.
   Для второго варианта необходимо включить опцию Sync Init SDK в Basic Settings и настроить вытикающие параметры. Почитайте всплывающие подсказки.

Затем, в реализации `InitAwake` выполните метод SDK для инициализации. По завершению инициализации выполните метод `YG2.SyncInitialization()`, чтобы сигнализировать PluginYG о том, что SDK готов. Пример:

```
public void InitAwake()
{
    if (CrazySDK.IsAvailable) // Требования CrazyGames
    {
        CrazySDK.Init(() =>
        {
            YG2.SyncInitialization();
        });
    }
}
```

Реализация отдельных модулей

Для удобства скрипты под отдельные модули можно поместить в папку Modules.

Например мы делаем реализацию для модуля некого `MyModule`. Создадим скрипт с названием файла CG_MyModule. И вставим в скрипт тот же код, что был для инициализации удалив все методы.
Но вот что нам ещё нужно сделать для скипта использующего методы модуля — прописать дефайн:

```
#if CrazyGamesPlatform_yg && MyMethod_yg // + && MyMethod_yg
namespace YG
```

```
{
    public partial class PlatformYG2 : IPlatformsYG2
    {
        public void MyMethod()
        {
            // Реализация
        }
    }
}
#endif
```

Мы добавили дефайн модуля. Дефайн состоит из названия модуля и приписки `_yg`. Это нужно делать, потому что модуль может отсутствовать в проекте и тогда пойдут ошибки.

Инициализация модулей

Многие модули имеют методы для инициализации. Они выполняются после рассмотренного выше метода `InitAwake`, когда SDK уже готов. Например, у модуля EnvirData есть метод `InitEnvirData` — в нём данные вроде `device`, `language` из SDK записываются в поля плагина. И для новой платформы тоже нужно передавать таким образом данные. Вот как выглядит готовый скипт CG_EnvirData:

```
#if CrazyGamesPlatform_yg && EnvirData_yg
using CrazyGames;

namespace YG
{
    public partial class PlatformYG2 : IPlatformsYG2
    {
        public void InitEnirData()
        {
            if (CrazySDK.User.IsUserAccountAvailable) // Требование
платформы
            {
                YG2.envir.language =
CrazySDK.User.SystemInfo.countryCode.ToLower();
                YG2.envir.browser = CrazySDK.User.SystemInfo.browser.name;
                YG2.envir.platform = CrazySDK.User.SystemInfo.os.name;

                string device = CrazySDK.User.SystemInfo.device.type;
                YG2.envir.deviceType = device;

                switch (device)
                {
```

```
                    case "desktop":
                        YG2.envir.isDesktop = true;
                        break;
                    case "tablet":
                        YG2.envir.isTablet = true;
                        break;
                    case "mobile":
                        YG2.envir.isMobile = true;
                        break;
                }
            }
        }

        public void GetEnvirData()
        {
            InitEnirData();
            YG2.GetDataInvoke();
        }
    }
}
#endif
```

Метод `GetEnvirData` — обновляет данные в процессе игры. И в конце он выполняет метод `YG2.GetDataInvoke`, который вызовет событие `onGetSDKData` если PluginYG уже полностью инициализирован. Данное событие может вызвать некоторые обновления данных плагина на сцене.

Асинхронная инициализация модулей

Бывает такое, что асинхронные методы есть не только для инициализации. Например, для Crazy Games нужно асинхронно инициализировать ещё данные игрока. Все асинхронные инициализации необходимо выполнить в методе `InitAwake`. По этому в скрипте с базовыми реализациями мы инициализируем и данные игрока, но зададим условие — если импортирован модуль авторизации. Вот так выглядит итоговый скрипт CG_BasicAPI:

```
#if CrazyGamesPlatform_yg
using CrazyGames;

namespace YG
{
    public partial class PlatformYG2 : IPlatformsYG2
    {
```

```
#if Authorization_yg
        public PortalUser User;
#endif
        public void InitAwake()
        {
            if (CrazySDK.IsAvailable)
            {
                CrazySDK.Init(() =>
                {
#if Authorization_yg
                    if (CrazySDK.User.IsUserAccountAvailable)
                    {
                        CrazySDK.User.GetUser(user =>
                        {
                            User = user;
                            YG2.SyncInitialization();
                        });
                    }
                    else
                    {
                        YG2.SyncInitialization();
                    }
#else
                    YG2.SyncInitialization();
#endif
                });
            }
        }
    }
}
#endif
```

Сохраняем данные в поле `User`. Используем его для записи данных в методе инициализации модуля. Готовый скрипт CG_Authorization:

```
#if CrazyGamesPlatform_yg && Authorization_yg
using UnityEngine;
using CrazyGames;

namespace YG
{
    public partial class PlatformYG2 : IPlatformsYG2
    {
        public void InitAuth()
```

```
        {
            if (CrazySDK.User.IsUserAccountAvailable)
            {
                YG2.player.auth = true;
                YG2.player.name = User.username;
                YG2.player.photo = User.profilePictureUrl;
            }
        }

        public void GetAuth()
        {
            if (CrazySDK.User.IsUserAccountAvailable)
            {
                CrazySDK.User.GetUser(user =>
                {
                    User = user;
                    YG2.GetDataInvoke();
                });
            }
        }

        public void OpenAuthDialog()
        {
            CrazySDK.User.ShowAuthPrompt((error, user) =>
            {
                if (error != null)
                {
                    Debug.LogError("Show auth prompt error: " + error);
                    return;
                }

                User = user;
                InitAuth();
                YG2.GetDataInvoke();
            });
        }
    }
}
#endif
```

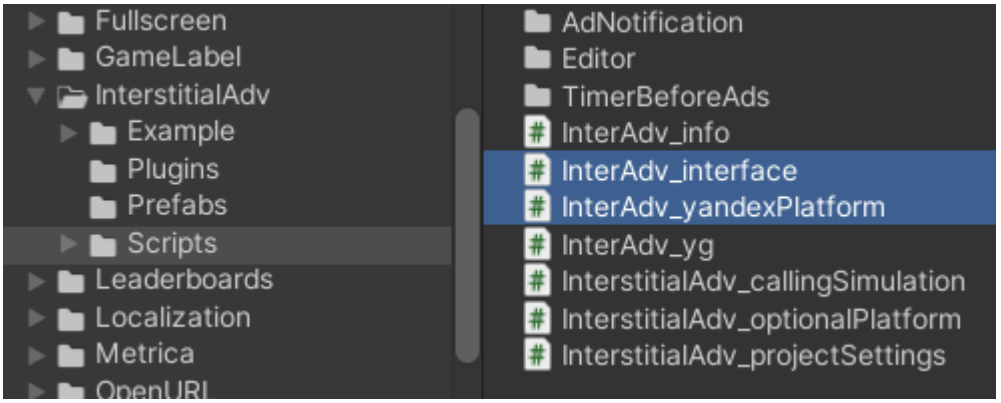
Итого:

1. Все реализации происходят в одном классе `PlatformYG2`.
2. Для реализации нужно переопределить существующий метод.

3. Сначала выполняется метод `InitAwake`, потом методы инициализации модулей. Потом метод `InitStart`, за ним `InitComplete`.
4. Если требуется, в методе `InitAwake` производим асинхронные инициализации. Если нужно, сохраняем некоторые данные для модулей.
5. Переопределяем реализации для модулей.

Реализация рекламы

Все методы в интерфейсе для модуля можно найти в папке модуля → **Scripts**. Скрипты интерфейсы помечены подписью `interface`. Там же можно посмотреть и стандартную реализацию.
Вместе со всеми модулями идёт реализация для стандартной платформы **YandexGames**. Содержится в скриптах с подписью `yandexPlatform`.
В скриптах с подписью `yg` содержится логика самого модуля. В `info` опции из настроек PluginYG.



Рассмотрим реализацию для Interstitial Adv:

```
#if CrazyGamesPlatform_yg && InterstitialAdv_yg
using CrazyGames;
using YG.Insides;

namespace YG
{
    public partial class PlatformYG2 : IPlatformsYG2
    {
        public void InterstitialAdvShow()
        {
            CrazySDK.Ad.RequestAd(CrazyAdType.Midgame, () =>
            {
                YGInsides.OpenInterAdv();
            }, (error) =>
            {
                YGInsides.ErrorInterAdv();
                YGInsides.CloseInterAdv();
            }, () =>
            {
```

```
                YGInsides.CloseInterAdv();
            });
        }

        public void FirstInterAdvShow()
        {
            OptionalPlatform.FirstInterAdvShow_RealizationSkip();
        }

        public void OtherInterAdvShow() { }
    }
}
#endif
```

Здесь мы переопределяем реализацию для метода `InterstitialAdvShow`. **CrazyGames** предоставляет коллбеки открытия, закрыти и т.д. рекламы.

В соответствующих коллбеках выполняются методы из внутреннего класса `YGInsides`, в нём содержится всё скрытое. Так мы выполняем, например, метод закрытия рекламы: `YGInsides.CloseInterAdv();`

А в `OptionalPlatform` содержатся готовые реализации для опциональных методов. О них сказано в разделе Опциональные методы.

Реализация локализации

Создайте метод `GetLanguage`, который должен вернуть язык. Пример:

```
#if CrazyGamesPlatform_yg && Localization_yg
using CrazyGames;

namespace YG
{
    public partial class PlatformYG2 : IPlatformsYG2
    {
        public string GetLanguage()
        {
            if (CrazySDK.User.IsUserAccountAvailable)
            {
                return CrazySDK.User.SystemInfo.countryCode.ToLower();
            }
            return "en";
        }
    }
}
#endif
```

Реализация облачных сохранений

Метод `LoadCloud` должен загружать сохранения. Для этого нужно передавать их в json строке методу `YGInsides.SetLoadSaves`. Стандартное значение указываем `Null`. Если передаваемое значение будет нулевым или это будет пустая строка, то сохранения сбросятся. Метод `SaveCloud` должен сохранять. Сохраняем поле `YG2.saves`. Пример:

```
#if CrazyGamesPlatform_yg && Storage_yg
using UnityEngine;
using YG.Insides;
#if NJSON_STORAGE_YG2
using Newtonsoft.Json;
#endif
using CrazyGames;

namespace YG
{
    public partial class PlatformYG2 : IPlatformsYG2
    {
        private const string KEY_SAVE = "CG_SavesYG";

        public void LoadCloud()
        {
            string jsonSaves = CrazySDK.Data.GetString(KEY_SAVE, null);
            YGInsides.SetLoadSaves(jsonSaves);
        }

        public void SaveCloud()
        {
#if NJSON_STORAGE_YG2
            CrazySDK.Data.SetString(KEY_SAVE,
JsonConvert.SerializeObject(YG2.saves));
#else
            CrazySDK.Data.SetString(KEY_SAVE, JsonUtility.ToJson(YG2.saves));
#endif
        }
    }
}
#endif
```

Локальные сохранения и сохранения для смуляции в Unity Editor уже реализованы в логике самого модуля.

Создали модуль?

Напишите разработчику плагина: https://t.me/MaxBornysov



Оглавление раздела:

1. Платформа Yandex Games

Платформа Yandex Games

Основная платформа Plugin Your Games это Яндекс Игры. Плагин позволяет **бесплатно использовать весь функционал SDK Яндекс Игр**. Все модули плагина по умолчанию работают с Яндекс Играми.

PluginYG - это **неофициальный плагин** Яндекс Игр! Но мы поддерживаем друг-друга. Таким образом, совместно с Яндекс Играми, мы сделали видеокурс по плагину :D



Оглавление раздела:

1. Платформа Crazy Games
2. О платформе (субъективно)
3. Правила (субъективно)
4. Готовые решения
5. Поддерживаемые функции
6. Рекомендуемые настройки
7. Дополнительная информация

Платформа Crazy Games

Чтобы понимать как работать с разными платформами в Plugin Your Games 2.0 — почитайте раздел "Платформы".
Скачать пакет CrazyGames можно на странице Boosty.

О платформе (субъективно)

С первой же игры делайте качественно и точно по правилам платформы. Обязательно прочтите требования для публикации. Чтобы приняли вашу игру и вас как разработчика, отнеситесь к Crazy Games с уважением. Иначе вы рискуете получить отказ к публикации игры без объяснений. И даже по почте могут не отвечать.

Модерация происходит быстро, в пару дней. Но она не работает в выходные дни.
Даже если всё будет по правилам, вас могут попросить внести индивидуальные правки. Но с ними можно договориться.

У платформы Crazy Games правила могут показаться излишне запутанными. Ниже постараюсь объяснить некоторые моменты.

Правила (субъективно)

Межстраничная реклама должна показываться только при открытии уровней или при их перезагрузке.
Есть моменты, при которых реклама показываться не должна:
При открытии первого уровня реклама показываться не должна. Но только при первом открытии уровня за игровую сессию. После того, как вы один раз не показали рекламу при открытии первого уровня – рекламу можно показывать при открытии всех уровней.
Общее правило межстраничной рекламы: **Реклама в середине игры должна запускаться только в тех местах, которые напрямую ведут к игровому прогрессу**.

После просмотра рекламы за вознаграждение не должна сразу показываться межстраничная реклама. Сделайте таймер или блокировку следующего вызова рекламы после получения вознаграждения.

В предложении просмотреть рекламы за вознаграждение обычно есть текст вроде «получить броню за просмотр рекламы». Может быть, вам захочется как-то отдельно выделить надпись «за рекламу». Так нельзя, весь текст должен быть однородным. С одинаковым шрифтом и цветом. Кроме этого, текст должен быть максимально правдивым. Например, для получения брони напишите уточнение «на одну игру. Получится: «Получить броню на одну игру за просмотр рекламы».

После получения вознаграждения за рекламу, вы должны заблокировать возможность снова получить это вознаграждение на 1 минуту. На месте кнопки для получения награды сделайте таймер вместо предложения просмотра рекламы.
Блокировку рекламы с таймером необходимо делать для той награды, что была уже получена, необязательно после просмотра рекламы за вознаграждение блокировать все награды.
Правило с таймером относится к многоразовым вознаграждениям вроде «получить монеты». Для получения, например, разных скинов — необязательно делать таймер.

В меню паузы игры у вас может быть кнопка для перезагрузки уровня. На английском она обязательно должна называться «**Restart**». А кнопка выхода в главное меню должна называться «**Main menu**» и никак иначе.

Когда в геймплее игроки умирают, вы можете вызвать рекламу при нажатии на кнопку «**Перезагрузить**», но при условии, что игрокам не предлагается возродить игрока за просмотр рекламы.

Игры с жестокостью не принимаются. Напрмиер: эффект крови при попадании в персонажа не должен быть окрашен в красный цвет.

Заранее сделайте ивенты разметки геймплея Start и Stop. Есть большая вероятность, что вас попросят внедрить эти ивенты.

Вы не можете рекламировать другие свои игры через текущую игру. Не размещайте никаких кнопок, переходящих по ссылке.

Готовые решения

PluginYG2 предоставляет автоматизированные решения всех вышеописанных задач!

- Для показа межстраничной рекламы под различные правила есть специальные методы. Смотрите раздел документации "Опциональные методы".
- Скрипт таймера после получения награды и другие объекты для рекламы за вознаграждение смотрите в разделе "Полноэкранной рекламы".
- За пропуск следующего показа межстраничной рекламы после получения вознаграждения отвечает параметр **Skip Inter Adv After Reward** в разделе модуля **Rewarded Adv** (в настройках плагина).
- Методы разметки геймплея `GameplayStart` и `GameplayStop`. Также работает метод `HappyTime`.

Поддерживаемые функции

Модуль платформы CrazyGames имеет явные реализации следующих модулей:

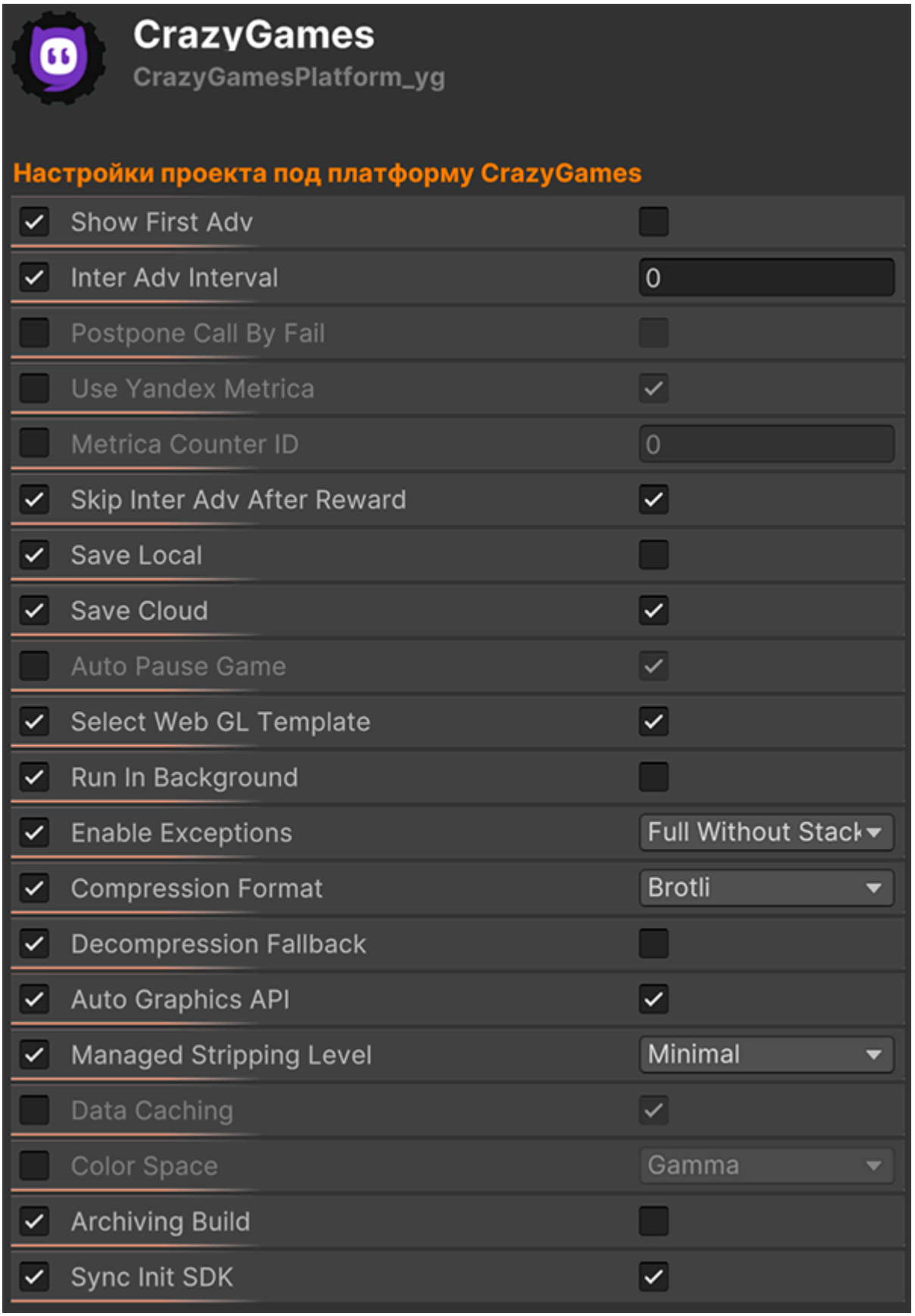
- Полноэкранная реклама
- Реклама за вознаграждение
- Данные окружения
- Авторизация
- Сохранения
- Локализация

⚠ Модуль платформы CrazyGames НЕ ИМЕЕТ реализации следующих модулей:

- Таблицы лидеров
- Внутриигровые покупки *(планируется)*
- Ссылки на игры и разработчика
- Отзывы

- Ярлык игры на рабочий стол

Рекомендуемые настройки



Show First Adv = false — не вызывайте рекламу при старте игры, это запрещено в Crazy Games.
Skip Inter Adv After Reward = true — после получения вознаграждения следующий показ межстраничной рекламы пропустится.

Select WebGLTemplate = true — выбрать шаблон платформы.
Archiving Build = false — в Crazy Games надо загружать папку билда, а не архив. Поэтому архивация не нужна.

> **Важно !**
>
> ***Sync Init SDK = true*** — этот параметр необходимо ставить, когда SDK инициализируется с задержкой, как у Crazy Games. Если игра адаптирована под синхронную инициализацию, то этот параметр лучше включить для всех платформ. То есть, можно не устанавливать разное значение для разных платформ, а сразу включить опцию **Sync Init SDK** в **Basic Settings**. Почитайте всплывающие подсказки и настройте вытикающие параметры.

> **Важно !**
>
> При использовании модуля **Storage**, желательно отключить локальные сохранения (опция **Save Local** в настройках модуля), т.к. в консоли разработчика, Crazy Games предлагает выбрать лишь одну из систем сохранения. При настройке игры на сайте и при использовании облачных сохранений (**Save Cloud**), выберите пункт с упоминанием **Data Module**.
> Вы можете настроить локальные и облачные сохранения отдельно для каждой платформы. В CrazyGames нет сильных ограничений на частоту сохранений игры, поэтому можно без ущерба выключить локальные сохранения оставив только облачные.

Дополнительная информация

PluginYG2 и его модули самостоятельно определяют важные данные окружения, такие как язык и устройство пользователя. На платформе могут верно определяться и другие поля, и корректно работать некоторые другие функции. А функций в PluginYG2 очень много, вести список о работоспособности каждой, для конкретной платформы — сложно. Пожалуйста, тестируйте необходимый вам функционал. Обычно, всё, что разработчик плагина мог сделать рабочим без предоставления API от платформы, или что можно симулировать — работает.

PluginYG2 не предоставляет возможность вынести некоторые функции, например, таблицы лидеров, на отдельные сервера. Он работает полностью на локальном подключении к вашему проекту. Это даёт надёжность и независимость. Используя PluginYG2, грубо говоря, вы используете только те функции, которые предоставляет та или иная платформа. При таком подходе, необходимо отдельно для каждой платформы внутри игры скрывать функционал, который не поддерживается. PluginYG2 предоставляет для этого мощный и удобный функционал. Подробнее смотрите в разделе Платформы".

Оглавление раздела:

1. Платформа Game Monetize
2. Работа с платформой
3. Тестирование игры
4. Поддерживаемые функции

Платформа Game Monetize

Чтобы понимать как работать с разными платформами в Plugin Your Games 2.0 — почитайте раздел Платформы.
Скачать пакет GameMonetize можно на странице Boosty (Требуется версия PluginYG 2.003 и выше).

Работа с платформой

- Зарегистрируйте аккаунт разработчика на сайте gamemonetize.com.
- Добавьте игру, и скопируйте её ID.
- В Unity, в настройках платформы (поле Platform, раздел настроек плагина Basic Settings) есть поле **Game Monetize ID**, запишите в него скопированный ID.
- Настройки платформы можно оставить такие же, как для Яндекс Игр.
- Работают все опции стандартного шаблона (стандартный шаблон используется для Яндекс Игр). Папка **Images** с изображениями **logo** и **background** копируется в билд из папки шаблона **YandexGames**. Таким образом, экран загрузки игры и т.д. будет такой же как и на платформе Яндекс Игры.
- Весь функционал, который предоставляет платформа - это только обычная межстраничная реклама. Подробнее в разделе ниже «Поддерживаемые функции».
- У площадки отсутствует реклама за вознаграждение. Скройте в игре возможность открыть ревард рекламу с помощью компонента PlatformEventsYG2.
- Реклама при старте не работает (опция Show First Adv).

Тестирование игры

Для отправки игры на модерацию, требуется выполнить тестирование рекламы. То есть, реклама в игре должна быть успешно показана. Для этого:

1. Обязательно выключите расширения вроде AdBlock, блокирующие рекламу на сайтах.
2. Скорее всего потребуется VPN (лучше США), чтобы реклама была показана.
3. SDK платформы и соответственно вызов рекламы полностью работает только если открыть игру нажав на кнопку **Verify Game**.

4. Если после загрузке нового билда, запускается старый, то в настройках **Player** → **Publishing Settings** включите опциию **Name Files As Hashes**. Также можно отчищать кеш в настройках браузера.

Поддерживаемые функции

PluginYG2 и его модули **EnvirData** и **Localization** самостоятельно определяют важные данные окружения, такие как язык и устройство пользователя. На платформе могут верно определяться и другие поля, и корректно работать некоторые другие функции. А функций в PluginYG2 очень много, вести список о работоспособности каждой, для конкретной платформы — сложно. Пожалуйста, тестируйте необходимый вам функционал. Обычно, всё, что разработчик плагина мог сделать рабочим без предоставления API от платформы, или что можно симулировать — работает.

PluginYG2 не предоставляет возможность вынести некоторые функции, например, таблицы лидеров, на отдельные сервера. Он работает полностью на локальном подключении к вашему проекту. Это даёт надёжность и независимость. Используя PluginYG2, грубо говоря, вы используете только те функции, которые предоставляет та или иная платформа. При таком подходе, необходимо отдельно для каждой платформы внутри игры скрывать функционал, который не поддерживается. PluginYG2 предоставляет для этого мощный и удобный функционал. Подробнее смотрите в разделе платформы.

Оглавление раздела:

Платформа Welwise Games
О платформе
Правила и публикация
Технические детали
Поддерживаемые функции

Платформа Welwise Games

Чтобы понимать как работать с разными платформами в Plugin Your Games 2.0 — почитайте раздел Платформы.
Скачать пакет для поддержки платформы WelwiseGames. (Требуется версия PluginYG 2.004 и выше)

О платформе

Welwise Games — веб-платформа для размещения HTML5-игр, ориентированных на детскую аудиторию. 2D, 3D, одиночные, мультиплеерные — любой игре будут рады на Welwise Games, если она интересна аудитории и сделана с душой. Приоритет в каталоге отдается мультиплеерным проектам. Присутствует возможность объединять свои игры в единую метавселенную с бесшовными переходами между играми и общими сохранениями.

Правила и публикация

На момент запуска первой версии платформы формальных ограничений и сложных требований к публикации нет. Платформа открыта к экспериментам и различным форматам сотрудничества. Публикация происходит следующим образом: необходимо связаться с командой через почту или Telegram, интегрировать SDK, протестировать игру в окружении Welwise Games и отправить на модерацию. Все заявки рассматриваются индивидуально, команда Welwise Games готова сопровождать разработчика на каждом этапе.

> **Ссылки:**
>
> Telegram-канал поддержки
> Telegram-канал с новостями платформы
> Почта: welwise.games@welwise.ru

Технические детали

Модуль платформы WelwiseGames имеет явные реализации следующих модулей:

- InterstitialAdv (полноэкранная реклама)
- RewardedAdv (реклама за вознаграждение)
- EnvirData (данные окружения)
- Storage (облачные сохранения)
- ServerTime (серверное время)

Также вы можете воспользоваться методом для перехода в другую игру:

```
#if WelwiseGamesPlatform_yg
    YG2.GoToWelwiseGame(int id);
#endif
```

Необходимо передать идентификатор игры и обернуть метод в директиву препроцессора `WelwiseGamesPlatform_yg`, так как этот метод существует только при выбранной платформе WelwiseGames. Перед выполнением метода будет произведено сохранение игры (будет вызван метод `SaveProgress`).

Поддерживаемые функции

PluginYG2 и его модули самостоятельно определяют большинство необходимых данных, даже если платформа их не предоставляет. Функций в PluginYG2 очень много, вести список о

работоспособности каждой, для конкретной платформы — сложно. Пожалуйста, тестируйте необходимый вам функционал. Обычно, всё, что разработчик плагина мог сделать рабочим без предоставления API от платформы, или что можно симулировать — работает.

PluginYG2 не предоставляет возможность вынести некоторые функции, например, таблицы лидеров, на отдельные сервера. Он работает полностью на локальном подключении к вашему проекту. Это даёт надёжность и независимость. Используя PluginYG2, грубо говоря, вы используете только те функции, которые предоставляет та или иная платформа. При таком подходе, необходимо отдельно для каждой платформы внутри игры скрывать функционал, который не поддерживается. PluginYG2 предоставляет для этого мощный и удобный функционал. Подробнее смотрите в разделе Платформы.



Оглавление раздела:

1. Appodeal
2. Помощь в установке SDK Appodeal
3. Настройки модуля
4. Рекомендуемые настройки
5. Дополнительная информация

Appodeal

Чтобы понимать, как работать с разными платформами в Plugin Your Games 2.0 — почитайте раздел Платформы.
Скачать пакет Appodeal можно на странице Boosty.

Appodeal - это платформа монетизации мобильных приложений, которая автоматически управляет рекламными сетями, оптимизируя доход. Она позволяет использовать в приложении такие виды рекламы как: межстраничные объявления, видео за вознаграждение, баннеры. С помощью алгоритмов аукциона в реальном времени выбирается наиболее выгодное объявление для показа пользователю.

Appodeal можно использовать на Android и IOS устройствах, в Google Play, Ru Store и в других аналогичных сторах. Чтобы настроить модуль плагина вам нужно только лишь вставить ключ приложения в соответствующее поле. Остальное — это установить сам SDK Appodel, установить пакет EDM и настроить проект в соответствии с документацией Appodeal. Ниже вы сможете найти короткое разъяснение как это сделать!

Помощь в установке SDK Appodeal

Помимо прочтения этой документации, также внимательно прочтите документацию от Appodeal!

Чтобы не наткнуться на ошибку во время импорта необходимых пакетов, проверьте через Unity Package Manager не установлен ли в вашем проекте пакет **Advertisement/Advertisement Legacy Package Manager**. Если установлен — удалите.

Сначала необходимо установить **EDM** (External Dependency Manager) — это инструмент для Unity, который управляет зависимостями плагинов и автоматически загружает нужные библиотеки для Android и IOS. Установите EDM именно **.tgz** формата и версию как указано в документации Appodeal.

В документации Appodeal мы видим два раздела: **UPM дистрибьюция** и **Manual** дистрибьюция. Обязательно выберите вкладку **UPM дистрибьюция**, которая содержит ссылку на установку новейшей версии SDK Appodeal. Версия Manual отличается и PluginYG её не поддерживает.

Если на вашем компьютере не установлен Git, скачайте его, установите и перезапустите компьютер. После чего вы сможете установить пакет по ссылке, следуя документации Appodeal.

Далее из важного, необходимо изменить версию gradle с помощью корректировки файла, настроить параметры gradle в Player Settings, настроить EDM, для IOS включить разрешение NSAppTransportSecurity.

Чтобы билд собрался и заработал, обязательно требуется либо сразу получить ID от AdMob и заполнить соответствующие поля в настройках Appodeal. Либо отключить AdMob в проекте. Это вполне сгодится, чтобы протестировать приложение. Зайдите в **Dependency Manager**, отключите **Google AdMob** и **Google Ad Manager** и нажмите кнопку **Generate**. После этого, вы наконец сможете собрать рабочий билд! Но перед началом сборки не соглашайтесь на предложение обновить gradle. Если случайно согласитесь, то снова скорректируйте файл формирующий версию gradle.

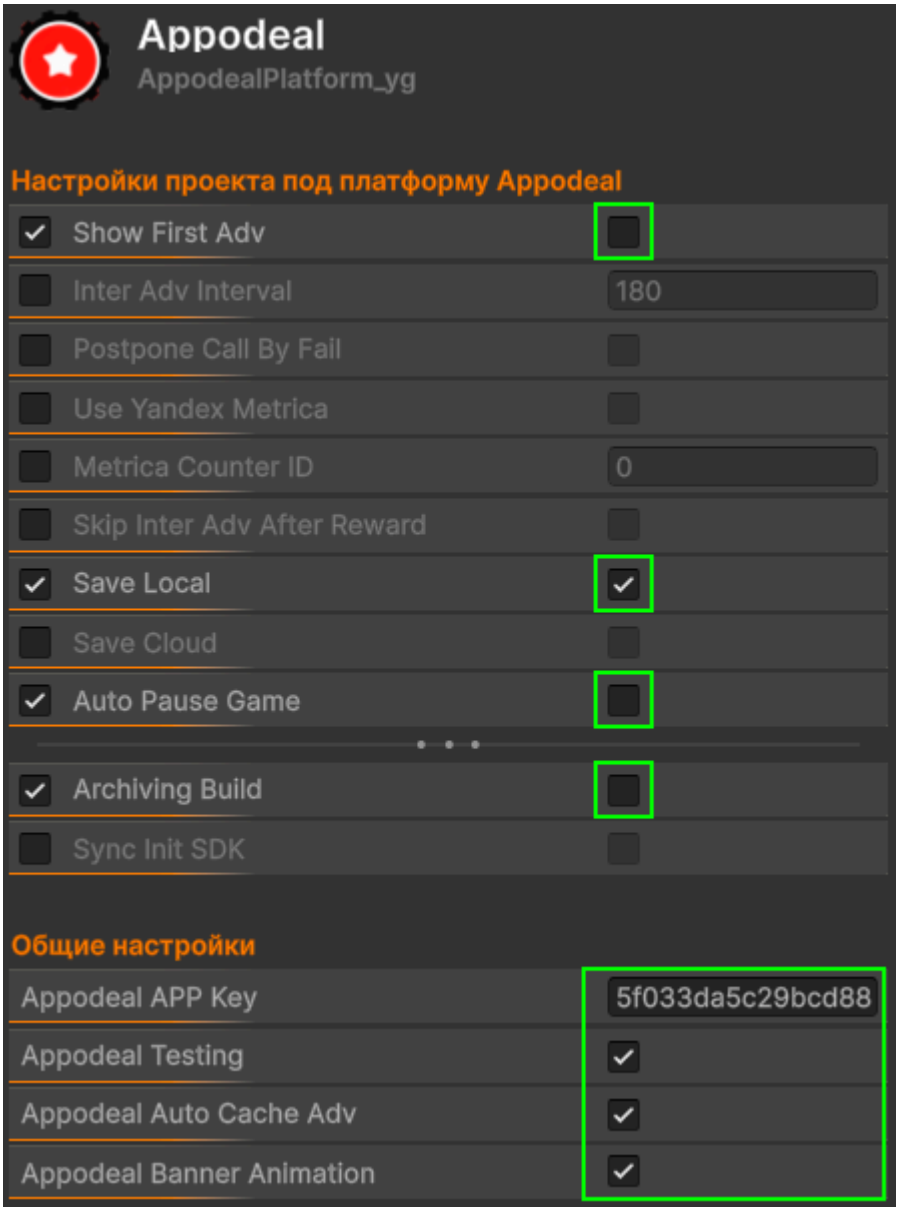
Чтобы быстро и максимально удобно собирать билды, которые сразу запустятся на вашем телефоне без перекидывания на него apk и даже подключения смартфона к компьютеру через кабель, я предлагаю воспользоваться Android Wi-Fi EasyConnect (ADB Build) соединения Unity и Android устройства по Wi-Fi, который я сделал специально для вас!😀

А чтобы видеть все логи, ошибки и другие сообщения прямо в билде на Android вы можете воспользоваться моей с любовью созданной — командной консолью! Которую я делал специально для себя😄 По этому она очень удобная, в использовании и внешнем виде простая, но внутри многофункциональная. С помощью неё вы сможете не только отслеживать логи, но и создавать свои команды, выполнять свои игровые методы. Можно изменять размер консоли и её положение на экране с сохранением состояния и даже можно поменять внешний вид консоли, создать свои пресеты дизайна. Данная командная консоль будет позже доступна на Boosty, как и модуль Appodeal. Спасибо всем, кто поддерживает нас и тем самым помогает в развитии плагина и других инструментов для разработки игр!

У Appodeal есть ещё настройка рекламных сетей, водопадов и прочего, рекомендую вам самостоятельно разобраться в этой теме и качественно настроить проект в конечном итоге.

Настройки модуля

Чтобы интегрировать модуль **Appodeal** в вашу игру, вам необходимо скопировать ключ приложения из личного кабинета Appodeal и вставить его в поле **Appodeal App Key** в настройках платформы. Это всё что требуется для интеграции самого модуля. Ниже изображение с данным полем, в котором вы сможете посмотреть для примера и другие настройки платформы:



Эти настройки идут по умолчанию. Но настройки по умолчанию не относятся к галочкам слева. Этими галочками можно активировать или деактивировать внесение изменения в ту или иную опцию в списке. Эти галочки имеют одно состояние для всех платформ! Чтобы была возможность выбрать разные параметры для разных платформ. Подробнее читайте в разделе Платформы.

Поля расположенные ниже поля ключа приложения:
Appodeal Testing — оставьте эту опцию включённой на время разработки и тестирования. Перед релизом уберите эту галку, убедитесь, что настоящие показы рекламы работают и тогда можно отправляться в релиз.
Appodeal Auto Cache Adv — автоматическая загрузка и кеширование рекламных объявлений.

Когда данная опция включена, вам достаточно только выполнять методы вызова рекламы, но в таком случае контроля над кешированием объявлений нет, что может хуже сказаться на оптимизации приложения. Если хотите ручной контроль, выполняйте метод загрузки рекламы и по желанию воспользуйтесь событиями завершения загрузки рекламного объявления:

```
YG2.optionalPlatform.LoadInterAdv()
YG2.optionalPlatform.LoadRewardedAdv()
YG2.optionalPlatform.onLoadedInterAdv
YG2.optionalPlatform.onLoadedRewardedAdv
```

Также есть события клика по рекламе:

```
YG2.optionalPlatform.onClickedInterAdv
YG2.optionalPlatform.onClickedRewardedAdv
```

Для использования рекламного баннера требуется импортировать модуль Баннерная реклама.
Этот модуль позволяет отображать рекламный баннер на экране снизу, сверху, слева или справа. Для Appodeal достаточно выполнить метод для открытия баннера с передачей параметра, который указывает в какой части экрана расположить объявление. Метод загрузки и удаления баннера можно не использовать.

Рекомендуемые настройки

Show First Adv = false — не желательно вызывать рекламу при старте, т.к. она может долго грузиться и открыться в неподходящий момент. Но она будет работать даже при выключенном автоматическом кешировании рекламы.
Save Cloud = false — облачные сохранения выключаем.
Save Local = true — локальные сохранения можете включить, они работают!
Auto Pause Game = false — можно выключить, потому что Appodeal имеет собственную реализацию отключения звука и остановки игры во время просмотра рекламы.
Archiving Build = false — для Appodeal скорее всего мы создаём **apk** или **aap**, по этому архивация не нужна.

Дополнительная информация

PluginYG2 и его модули **EnvirData** и **Localization** самостоятельно определяют важные данные окружения, такие как язык и устройство пользователя. На платформе могут верно определяться и другие поля, и корректно работать некоторые другие функции. А функций в PluginYG2 очень много, вести список о работоспособности каждой, для конкретной платформы — сложно. Пожалуйста, тестируйте необходимый вам функционал. Обычно, всё, что разработчик плагина мог сделать рабочим без предоставления API от платформы, или что можно симулировать — работает. На Android и IOS устройствах работают локальные сохранения от модуля Storage!

PluginYG2 не предоставляет возможность вынести некоторые функции, например, таблицы лидеров, на отдельные сервера. Он работает полностью на локальном подключении к вашему проекту. Это даёт надёжность и независимость. Используя PluginYG2, грубо говоря, вы используете только те функции, которые предоставляет та или иная платформа. При таком подходе, необходимо отдельно для каждой платформы внутри игры скрывать функционал, который не поддерживается. PluginYG2 предоставляет для этого мощный и удобный функционал. Подробнее смотрите в разделе Платформы.



Оглавление раздела:

1. Платформа GamePix
2. Работа с платформой
3. Основные правила
4. Технические детали
5. Специфичные (необязательные) методы для GamePix
6. Тестирование игры
7. Поддерживаемые функции

Платформа GamePix

Чтобы понять как работать с разными платформами в Plugin Your Games 2.0 — почитайте раздел Платформы.
Скачать пакет GamePix можно на странице Boosty.

GamePix — платформа для распространения HTML5-игр. Связывает разработчиков с глобальной аудиторией через сеть партнерских сайтов. Предлагает легкую интеграцию SDK и монетизацию через рекламу с доходом 45% разработчику.

Работа с платформой

- Зарегистрируйте аккаунт разработчика на сайте gamepix.com.
- Создайте новую игру.
- В настройках PluginYG параметры платформы можно оставить такие же, как для Яндекс Игр.
- Работают все опции стандартного шаблона (стандартный шаблон используется для Яндекс Игр). Папка **Images** с изображениями **logo** и **background** копируется в билд из папки шаблона **YandexGames**. Таким образом, экран загрузки игры и т.д. будет такой же как и на платформе Яндекс Игры.

Основные правила

- Звук должен автоматически приостанавливаться во время рекламы.
- Показывайте рекламу только между уровнями, не во время геймплея.
- Запрещены внешние ссылки, аналитика и сторонние SDK.
- Игры должны корректно отображаться в формате 640x480px.
- Контент не должен содержать религиозную тематику или explicit-материалы.
- Название игры и ассеты должны быть согласованы.
- Ответственность за авторские права лежит на разработчике.
- Поддерживайте мультиязычность с английским по умолчанию.

Подробнее о правилах на сайте GamePix.

Технические детали

Модуль платформы GamePix имеет явные реализации следующих модулей:

- Полноэкранная реклама
- Реклама за вознаграждение
- Данные окружения
- Сохранения
- Локализация
- Метод `HappyTime` (Он же "*Happy Moment*" в терминологии SDK GamePix): отправьте в SDK информацию о каком-либо "***счастливом моменте***" в вашей игре. Например разблокировка особого предмета или запуск какого-либо интересного события в игре.

В консоли разработчика GamePix не все функции работают правильно. Не полагайтесь на корректное отображение опций показа реклама в списке для проверки функций. Также, тестирование паузы игры работает странно. И нельзя протестировать сохранения, при перезагрузки игры всё стирается.

Специфичные (необязательные) методы для GamePix

Обновление уровня — Вызывайте эту функцию каждый раз, когда пользователь переходит на новый уровень. Функция принимает только числа.

Пример использования:

```
#if GamePixPlatform_yg
    YG2.optionalPlatform.UpdateGamePixLevel(2);
#endif
```

Обновление счета — Вызывайте эту функцию каждый раз при обновлении счёта в игре. Функция принимает только числа.

Пример использования:

```
#if GamePixPlatform_yg
    YG2.optionalPlatform.UpdateGamePixScore(100);
#endif
```

Подробнее о методах на сайте GamePix

Тестирование игры

- В разделе «**Build**» перейдите по ссылке: «**Before uploading your build, try our Testing Toolkit!**» (Инструмент для тестирования игры).
- Выберите: «**Yes! No, I would like to upload my local Build.**»
- Запустите игру нажав на кнопку «**Load Game URL**». Предварительно можно выбрать язык, на котором запустится игра.
- Можно запустить игру в полном экране открыв её по ссылке, которая находится слева от кнопки загрузки игры.
- Билд игры для тестирования и для отправки на модерацию необходимо загружать отдельно.

Поддерживаемые функции

PluginYG2 и его модули **EnvirData** и **Localization** самостоятельно определяют важные данные окружения, такие как язык и устройство пользователя. На платформе могут верно определяться и другие поля, и корректно работать некоторые другие функции. А функций в PluginYG2 очень много, вести список о работоспособности каждой, для конкретной платформы — сложно. Пожалуйста, тестируйте необходимый вам функционал. Обычно, всё, что разработчик плагина мог сделать рабочим без предоставления API от платформы, или что можно симулировать — работает.

PluginYG2 не предоставляет возможность вынести некоторые функции, например, таблицы лидеров, на отдельные сервера. Он работает полностью на локальном подключении к вашему проекту. Это даёт надёжность и независимость. Используя PluginYG2, грубо говоря, вы используете только те функции, которые предоставляет та или иная платформа. При таком подходе, необходимо отдельно для каждой платформы внутри игры скрывать функционал, который не поддерживается. PluginYG2 предоставляет для этого мощный и удобный функционал. Подробнее смотрите в разделе Платформы.

AD

Оглавление раздела:

1. Полноэкранная реклама
2. Работа с кодом
3. Реклама в процессе игры
4. Нотификация перед рекламой

Полноэкранная реклама

(Реклама в середине игры)

- Необходимо импортировать модуль Полноэкранной рекламы".
- Вызывать рекламу можно без написания кода через компонент Начало работы > Компонент "EventsYG2".
- Правила показа рекламы на площадке **Yandex Games**.

> **Обратите внимание на параметр "*Pause Game*" в настройках плагина "*Basic Settings*" !**
>
> При просмотре рекламы и в других ситуациях, когда это необходимо — игра будет остановлена. Пауза контролирует глобальные параметры `Time.timeScale`, `AudioListener.pause`, `Cursor.visible` и `Cursor.lockState`, а также отключает EventSystem, потому что иногда с ним есть проблемы.
>
> Вы можете менять значения перечисленных выше параметров в своём коде независимо от того, остановил плагин игру или нет. После паузы параметры будут установлены на те значения, что были прописаны в вашем коде. Даже если во время паузы ваш код, например, устанавливает `timeScale` = 1, то плагин отловит этот момент и вернёт `timeScale` = 0. Но после того, как пауза будет закончена, плагин установит значение `timeScale` = 1, т.к. в вашем коде вы так прописали.
>
> Если всё же возникают трудности при использовании автоматической паузы, то есть ивент:
> `YG2.onPauseGame` (тип `Action<bool>`)

Опции рекламы можно найти в ***настройках плагина*** → ***Interstitial Adv***.
По умолчанию есть интервал между показом рекламы в 60 секунд контролируемый PluginYG2. Его можно отключить, установив параметр **Inter Adv Interval** в **0**. В таком случае, в билде вызов рекламы будет при каждом выполнении соответствующего метода. Но для Unity Editor есть отдельная настройка интервала в **настройках плагина** → **General Simulation**.
Разделение на две опции сделано для удобства тестирования рекламы в проекте. Для тестирования в Unity Editor плагин имеет симуляцию показа рекламы.

Работа с кодом

Метод вызова рекламы:
`InterstitialAdvShow`

Пример использования метода:

```
using YG; // Подключите пространство имён плагина

public class MyClass
{
    public void MyMethod()
    {
        YG2.InterstitialAdvShow();
    }
}
```

> **Обратите внимание**
>
> Всё что написано в этом разделе ниже - второстепенно

События:
`onOpenInterAdv` — открытие рекламы.
`onOpenAnyAdv` — открытие любой рекламы (interstitial или rewarded).
`onCloseInterAdv` — закрытие рекламы.
`onCloseAnyAdv` — закрытие любой рекламы (interstitial или rewarded).
`onCloseInterAdvWasShow` — закрытие рекламы, возвращает bool была ли показана реклама.
`onErrorInterAdv` — ошибка при открытии рекламы.
`onAdvNotification` — вызывается перед открытием любой рекламы.
`onPauseGame` — пауза игры. Возвращает bool.

Информативные поля:
`nowInterAdv` — открыта ли сейчас реклама.
`nowAdsShow` — открыта ли сейчас любая реклама (interstitial или rewarded).
`isTimerAdvCompleted` — сколько осталось до конца таймера интервала между показами рекламы.

Специфические методы для разных платформ:
`YG2.SkipNextInterAdCall();` — пропустить следующий показ рекламы
`YG2.optionalPlatform.FirstInterAdvShow();` — первый показ рекламы. Для Yandex Games в любом случае покажет рекламу. Для Crazy Games пропустит первый показ рекламы, затем, даже после перезагрузки страницы реклама сразу будет вызывать с данным методом.
`YG2.optionalPlatform.OtherInterAdvShow();` — специальная реализация под отдельные платформы. Для Yandex Games в любом случае покажет рекламу. Для Crazy Games не будет показывать рекламу.

Реклама в процессе игры

(Таймер перед рекламой)
Готовое решение для рекламы в процессе игры. Он вызывает рекламу тогда, когда заканчивается интервал между рекламой. Перед показом рекламы идёт остановка игры пока не пройдёт таймер (рекомендуется 2 сек.). Скрипт гибко настраивается.

Для рекламы в середине игры воспользуйтесь префабом **Timer Before Ads**. Его можно найти: *PluginYourGames/Modules/InterAdv/Prefabs/**Timer Before Ads***.
Для корректной работы скрипта необходимо включить опцию ***Postpone Call By Fail*** в настройках модуля рекламы.

> **Рекомендация**
>
> При использовании префабов PluginYG рекомендуется создавать свой префаб вариант для избежания проблем при обновлении плагина (что такое префаб вариант).

Поместите префаб (префаб вариант) на сцену, в которой хотите показывать рекламу в процессе игры. Настройте внешний вид UI элемента под стиль вашей игры.

Сообщение перед рекламой

В некоторых случаях нужно скрыть экран игры и поставить паузу, пока реклама загружается. В этом может помочь скрипт **Ad Notification YG** и префаб **Ad Notification**. Поместите префаб (префаб вариант) на сцену, которая загружается первой в проекте. Можете отредактировать цвет «заглушки».
Для корректной работы скрипта необходимо включить опцию **Postpone Call By Fail** в настройках модуля рекламы.



> **Оглавление раздела:**
>
> 1. Реклама за вознаграждение
> 2. Другие объекты
> 3. Таймер после вознаграждения

Реклама за вознаграждение

Метод вызова рекламы за вознаграждение:
`RewardedAdvShow` — имеет два параметра: (первый — **id** вознаграждения типа **string**), (второй необязательный — **callback** типа **Action**).

Событие для вознаграждения:
`onRewardAdv` — возвращает **id** вознаграждения.

Пример использования рекламы за вознаграждение с коллбэком:

```
using UnityEngine;
using YG;

public class MyClass : MonoBehaviour
{
    public string rewardID;
    public int coins;

    // Вызов рекламы за вознаграждение
    public void MyRewardAdvShow()
    {
        YG2.RewardedAdvShow(rewardID, () =>
        {
            // Получение вознаграждения
            coins++;

            // По желанию, воспользуйтесь ID вознаграждения
            if (rewardID == "AddCoin")
                coins++;
        });
    }
}
```

Ещё вариант использования с коллбэком:

```
public void ShowAdvReward()
{
    string id = "coin"; // Передача id требуется для внутренней работы плагина
    YG2.RewardedAdvShow(id, Reward);
}

public void Reward()
{
    // Выдаём вознаграждение
}
```

Пример использования рекламы за вознаграждение с подпиской на событие:

```
using UnityEngine;
using YG;
```

```
public class MyClass : MonoBehaviour
{
    public string rewardID;
    public int coins;

    // Когда объект с данным классом станет активным, метод OnReward подпишется
на событие вознаграждения
    private void OnEnable()
    {
        YG2.onRewardAdv += OnReward;
    }

    // Необходимо отписывать методы от событий при деактивации объекта
    private void OnDisable()
    {
        YG2.onRewardAdv -= OnReward;
    }

    // Вызов рекламы за вознаграждение
    public void MyRewardAdvShow(string id)
    {
        YG2.RewardedAdvShow(rewardID);
    }

    // Метод подписан на событие OnReward (ивент вознаграждения)
    private void OnReward(string id)
    {
        // Проверяем ID вознаграждения. Если совпадает с тем ID, с которым
вызывали рекламу, то вознаграждаем.
        if (id == rewardID)
        {
            // Получение вознаграждения
            coins++;
        }
    }
}
```

Important

Всё что написано в этом разделе ниже - второстепенно

Другие объекты

События:
`onOpenRewardedAdv` — открытие рекламы за вознаграждение.
`onOpenAnyAdv` — открытие любой рекламы (interstitial или rewarded).
`onCloseRewardedAdv` — закрытие рекламы за вознаграждение.
`onCloseAnyAdv` — закрытие любой рекламы (interstitial или rewarded).
`onErrorRewardedAdv` — ошибка при открытии рекламы за вознаграждение.
`onAdvNotification` — вызывается перед открытием любой рекламы.
`onPauseGame` — пауза игры. Возвращает bool.

Информативные поля:
`nowRewardAdv` — открыта ли сейчас реклама за вознаграждение.
`nowAdsShow` — открыта ли сейчас любая реклама (interstitial или rewarded).

Специфический метод для разных платформ:
`YG2.optionalPlatform.LoadRewardedAdv();` — предзагрузка рекламы за вознаграждение.
`YG2.SkipNextInterAdCall();` — пропустить следующий показ межстраничной рекламы.

Таймер после вознаграждения

В некоторых платформах вроде Crazy Games после вознаграждения требуется установить таймер с временной блокировкой просмотра новой рекламы за вознаграждение.
Для этого был создан готовый скрипт **Rewarded Adv Lock Timer** с префабом **Lock Timer**. Его можно найти в папке **Prefabs** внутри модуля. Пример использования данного префаба есть в демо сцене *PluginYourGames/Modules/RewardedAdv/Example/Scenes/**RewardedAdv***.

> **Tip**
>
> При использовании префабов PluginYG рекомендуется создавать свой вариант префаба для избежания проблем при обновлении плагина (что такое вариант префаба).



> **Оглавление раздела:**
>
> Sticky-баннеры

Sticky-баннеры

Это статичные рекламные баннеры в Яндекс Играх:

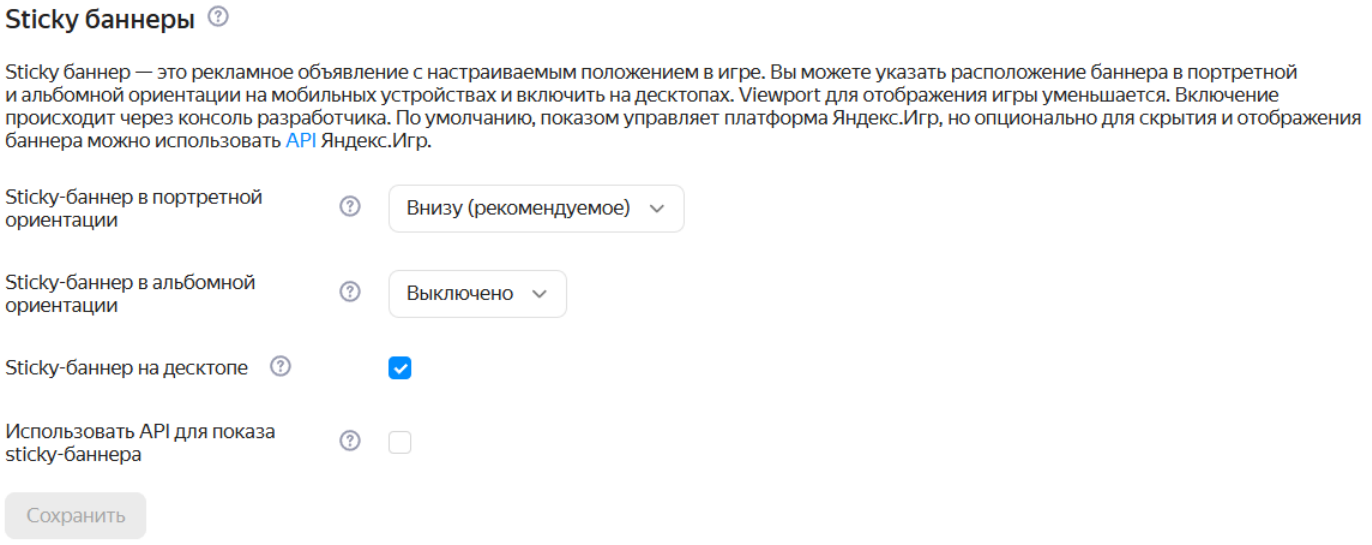


Так выглядят Sticky-баннеры.

⚠ Sticky-баннеры НЕ отображаются, если:

1. Размер экрана на ПК слишком маленький
2. Включён полноэкранный режим
3. Sticky-баннеры деактивированы в консоли игры
4. Вы используете API для скрытия баннеров

В консоле игры, зайдите в раздел **Реклама** и настройте баннеры под ваш проект:



⚠ Внимание

Необязательно импортировать модуль Sticky-баннеры, чтобы баннерная реклама работала. Модуль предназначен для динамического включения и отключения Sticky-

баннеров.
Для этого используется метод:

```
YG2.StickyAdActivity(bool activity);
```



Оглавление раздела:

1. Баннерная реклама
2. API
3. Симуляция в Unity Editor

Баннерная реклама

(Рекламное объявление на экране снизу, сверху, слева или справа)

Warning

Данный модуль не предназначен для платформы Яндекс Игры.
Для платформы Яндекс Игры смотрите раздел Sticky-баннеры.

Важно

- Необходимо импортировать этот модуль с помощью Начало работы > Модульная система.
- Вызывать рекламу можно без написания кода через компонент Компонент "EventsYG2".

Баннер предназначен для отображения на Android и IOS платформах. Он является агрегатором для реализации API других модулей рекламной сети, таких как Appodeal или Яндекс Реклама в мобильных приложениях (модуль в разработке). (Такие модули размещаются на Boosty)

API

- `YG2.SetBannerPosition(YG2.BannerPosition.Bottom)` — перед тем как вызвать метод для открытия баннера, сначала установите значение, которое определяет его расположение на экране. По умолчанию баннер появляется в верхней части.
- `YG2.LoadBanner()` — загрузить баннер в кеш. Не для всех платформ это обязательный метод. Смотрите документацию модуля рекламной сети, который используете.
- `YG2.ShowBanner()` — показать баннер.
- `YG2.HideBanner()` — скрыть баннер.
- `YG2.DestroyBanner()` — удалить баннер из кеша. Также, не для всех рекламных сетей данный метод необходимо выполнять.

Симуляция в Unity Editor

В настройках **Plugin YG** вы найдёте раздел **Banner Adv**, в котором можно задать размер высоты баннера в пикселях и цвет панели, которая будет симулировать объявление. Да, в эдиторе есть симуляция отображения баннера, при желании её можно отключить в тех же настройках.
Размер баннера может быть разный в зависимости от используемой рекламной сети. Настраивайте интерфейс в вашей игре в зависимости от того, какой рекламный баннер будет отображаться в билде приложения.



Оглавление раздела:

1. Данные окружения
2. Симуляция в Unity Editor
3. Payload

Данные окружения

Используйте **данные окружения** чтобы получить параметры от SDK платформы такие как устройство на котором запущена игра, язык и т.д.
Все данные содержатся внутри `YG2.envir`. Пример получения устройства различными способами:

```
using UnityEngine;
using YG;

public class MyClass : MonoBehaviour
```

```
{
    public void MyMethod()
    {
        // deviceType может возвращать: desktop, mobile, tablet, tv
        string deviceStr = YG2.envir.deviceType;

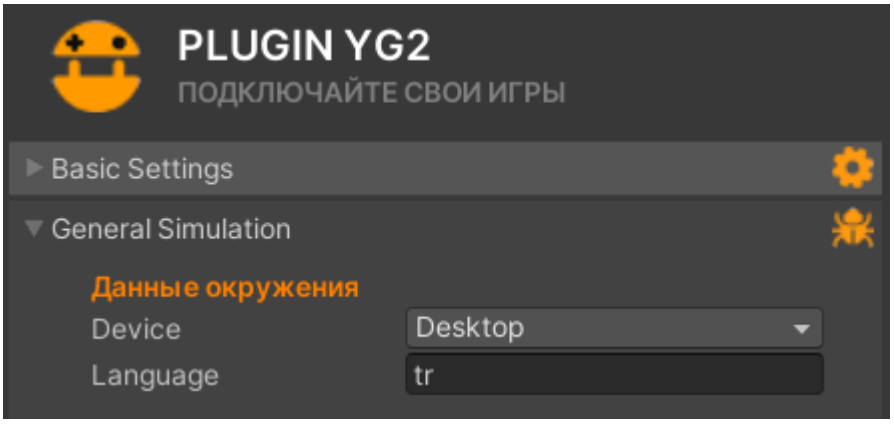
        // Есть bool поля: isDesktop, isMobile, isTablet, isTV
        bool deviceIsDesktop = YG2.envir.isDesktop;

        // Поле enum:
        YG2.Device device = YG2.envir.device;
    }
}
```

Остальные поля:
`language` — язык от SDK платформы в формате «**код страны**». Пример: `ru`, `en`
`domain` - пример: `ru`, `com`
`appID` - пример: `177895`
`browserLang` - язык браузера в формате «**код страны**»
`payload` - описание ниже
`platform` - операционная система, пример: `Win32` (подробнее)
`browser` - пример: `Yandex`, `Opera`, `Firefox`, `IE`, `Edge`, `Chrome`, `Safari`, `Other`

Симуляция в Unity Editor

В настройках плагина → **General Simulation** вы можете поменять поле девайса или языка:



Например, запишите `en` в поле **Language**, тогда в Unity Editor поле `YG2.envir.language` будет = `en`. Таким образом можно легко тестировать игру на разных языках.

Payload

Вы можете передавать какое-либо значение в игру через ссылку с помощью гипер-оператора. Таким образом можно выполнять какой то код в игре при переходе по определённой ссылке, например: открывать конкретный уровень в игре, давать бонус, запускать игру в режиме отладки.

Как передать значение в игру

Например, адрес Вашей игры: `https://yandex.ru/games/app/012345`
Допишите к адресу приписку: `?payload=`
После «=» напишите значение, которое хотите передать в игру. Допустим, значение будет: `debug123`
Мы получили ссылку: `https://yandex.ru/games/app/012345?payload=debug123`
Теперь при переходе по ссылке с припиской `?payload=debug123`, в игре поле `YG2.envir.payload` вернёт строку `debug123`.

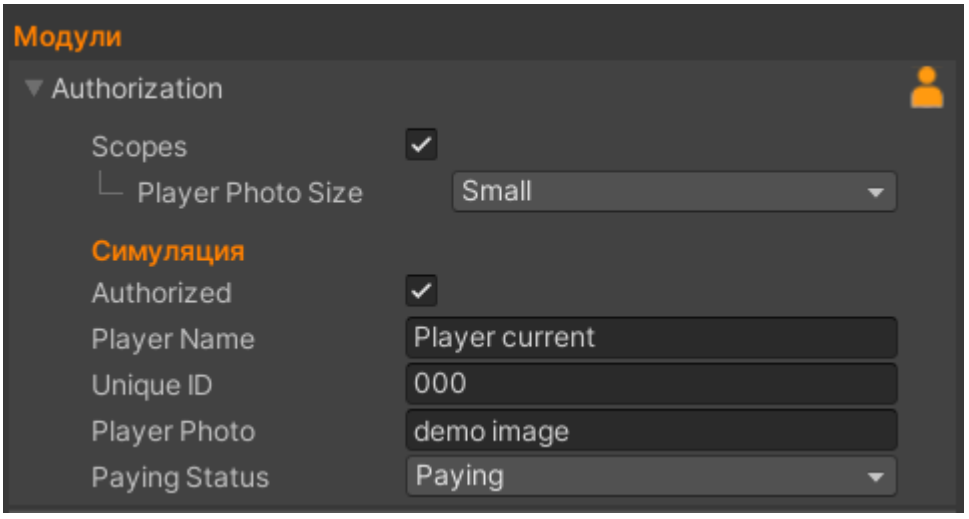
Оглавление раздела:

1. Авторизация
2. Быстрое отображение ника и аватара

Авторизация

С помощью модуля авторизации можно получить данные пользователя такие как, например, ник или аватарка. Также он необходим для некоторых других модулей таких как Leaderboards или Payments.
В настройках плагина можно найти опции для симуляции в Unity Editor:

Данные пользователя хранятся в `YG2.player`.
Например, так можно получить ник: `YG2.player.name`.
Если пользователь не авторизован, то ник будет `unauthorized`.
Если пользователь предпочёл остаться анонимным, то ник будет `anonymous`.



`YG2.player.auth` — авторизация пользователя (`true`, если авторизован).
`YG2.player.name` — ник пользователя.
`YG2.player.id` — идентификатор пользователя.
`YG2.player.photo` — ссылка на подкачку аватарки пользователя.

Также есть метод `YG2.OpenAuthDialog`, который открывает диалоговое окно авторизации пользователя. Окно откроется только если пользователь не авторизован! После успешной авторизации, сработает событие `onGetSDKData`, на которое можно подписаться для обновления данных.

Быстрое отображение ника и аватара

Есть префаб **Player Name & Photo**, с помощью которого можно быстро отобразить ник и аватар пользователя. Он находится в папке PluginYourGames/Modules/Authorization/**Prefabs**. Компонент **Get Player YG** содержит два поля для заполнения — необязательно заполнять все поля. Если не заполнить, они проигнорируются.

> ⚠ **Вариант префаба**
>
> Префаб необходимо поместить в Canvas объект.
> При использовании префабов PluginYG рекомендуется создавать свой вариант префаба для избежания проблем при обновлении плагина. (что такое вариант префаба).



> **Оглавление раздела:**
>
> 1. Сохранения
> 2. Основной модуль Storage
> 3. Redefine Player Prefs
> 4. Player Stats

Сохранения

В данном разделе будут описаны три с способа для сохранений:

1. Первый основной модуль **Storage** необходим для облачного сохранения. Он самый удобный и правильный, но если сохранения в проекте уже реализованы, например с Player Prefs — то придётся их переделывать.
2. Модуль **Redefine Player Prefs** — переопределяет стандартный класс **Player Prefs** на сохранения плагина. Если импортирован модуль Storage, то данные будут сохраняться в класс SavesYG, как сохраняются другие данные с помощью модуля Storage. Таким образом вы можете сохранить свои Player Prefs в облако. Если не переопределить Player Prefs, то в Яндекс Играх и в некоторых других платформах после обновления игры все сохранения сотрутся. Когда модуль Storage не импортирован, данные сохранятся в

локальное хранилище браузера. В таком случае данные не сотрутся, но это всё же это не такой надёжный способ, как облако.

3. Модуль **Player Stats** — быстрые сохранения в облако. Сохранять данные можно только типа int.

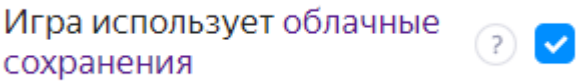
Основной модуль Storage

Рассмотрим настройки модуля **Storage**:



При использовании облачных сохранений (флаг **Save Cloud**) для Яндекс Игр необходимо:

1. Импортировать модуль Авторизация.
2. В консоли игры установите данный флаг:



Для Яндекс Игр рекомендуется использовать только облачные сохранения (флаг **Save Cloud**). При облачном сохранении независимо от того, авторизован пользователь или нет, данные будут сохранены! Если пользователь не авторизован, то данные будут сохранены в локальное хранилище Яндекс Игр.

Локальные сохранения плагина (флаг **Save Local**) используйте для других платформ, которые не имеют облачных сохранений. Можно настроить флаг **Save Local** для каждой конкретной платформы по-разному (смотрите раздел Платформы).

Рекомендуется отключить опцию **Flash**, чтобы сохранения ставились в очередь, тогда все данные точно отправятся на сервер. Иначе, при слишком частом сохранении прогресса (есть лимиты) отправка данных будет отклонена.

Интеграция

Создайте скрипт в любой папке проекта и с любым наименованием. Вставьте в него следующий код:

```
namespace YG
{
    public partial class SavesYG
    {
        // Ваши данные для сохранения
        public int coins = 5; // Пример
    }
}
```

Таких скриптов можно делать хоть сколько благодаря модификатору partial, чтобы разграничить сохранения разного назначения.
Внутри класса **SavesYG** будут храниться все ваши поля, которые необходимо сохранить. Как пример создано поле `coins` (монеты). Оно будет использоваться в проекте как глобальное поле для монет.
Чтобы получить данные или изменить, делайте это также, как с любыми другими полями в вашем коде. Хранилище сохранений находится в `YG2.saves`.

Пример

```
using UnityEngine;
using YG;

public class MyClass : MonoBehaviour
{
    public void MyMethod()
    {
        // Чтение
        Debug.log(YG2.saves.coins);

        // Запись
        YG2.saves.coins += 10;
    }
}
```

Хранилище сохранений статично. Изменения поля не сбросятся на протяжении всей игровой сессии.
Но данные необходимо сохранять, чтобы они не сбросились после закрытия игры. Для этого используйте метод:
`YG2.SaveProgress();`
Загружаются сохранения сами — в поле `YG2.saves` (класс SavesYG) при запуске игры. Не нужно вызывать метод загрузки, но всё же вы можете его использовать в специфических случаях: `YGInsides.LoadProgress();`
Метод `LoadProgress` требует подключения пространства имён `YG.Insides`. В этой библиотеке есть и другие внутренние методы PluginYG2.
Получение данных после выполнения метода `LoadProgress` идёт асинхронно. Поэтому после того как данные будут получены — вызовется ивент `onGetSDKData`.

Подытожим:

1. Необходимо создать класс и прописать в нём свои поля, которые нужно сохранять.
2. В своих скриптах пользоваться созданными полями, использовать их для чтения и записи.
3. Сохранять эти поля с помощью метода `SaveProgress` отправляя их в локальное или облачное хранилище.

4. При запуске игры, данные сами подтянутся в экземпляр класса `YG2.saves`.

Итого

1. Данные сохраняются в строку **json**.
2. Можно сохранять экземпляр класса — для этого необходимо к классу применить атрибут `[System.Serializable]`.
3. Можно использовать конструктор. В классе **SavesYG** может быть любая логика — методы и т.д.
4. Используйте **коллекцию Generic** вместо обычного массива. При попытке расширить массив сохранения будут загружаться с ошибкой.

Пример

```
using System;
using System.Collections.Generic;
using UnityEngine;

namespace YG
{
    public partial class SavesYG
    {
        public List&ltItemData&gt items = new List&ltItemData&gt();

        public ItemData GetItem(string name)
        {
            for (int i = 0; i &lt items.Count; i++)
            {
                if (items&#91i&#93.name == name)
                    return items&#91i&#93 ;
            }

            ItemData newItem = new ItemData
            {
                name = name,
                position = Vector3.zero
            };

            return newItem;
        }
    }
}

&#91Serializable&#93
public struct ItemData
```

```
{
    public string name;
    public Vector3 position;
}
```

Если требуется сохранять более сложные структуры, например двойной массив:

Есть вариант использования пакета Newtonsoft Json. Он мощнее стандартного класса JsonUtility, но с ним билд игры будет весить на 2 мб больше.
В настройках PluginYG2 есть кнопка для импорта пакета Newtonsoft Json. После импорта пакета, появится кнопка активации Newtonsoft Json для модуля Storage.

Первая игровая сессия и сброс сохранений

Событие `onDefaultSaves` — вызывается в старте игры при отсутствии сохранений, то есть когда пользователь первый раз зашёл в игру. Или если выполнить метод `SetDefaultSaves` — сброс сохранений. После сброса сохранений будут вызваны события `onDefaultSaves` и `onGetSDKData`. После сброса не произойдёт автоматического сохранения, то есть если игра не была сохранена, то при её перезагрузке сохранения будут отсутствовать. Выполняйте метод `SaveProgress` после сброса самостоятельно.

Удалить сохранения для Unity Editor

Зайдите в папку PluginYourGames/Editor и удалите файл SavesEditorYG2.json. Он отмечен значком сохранений.

Redefine Player Prefs

(Необходимо импортировать модуль RedefinePlayerPrefs)
Желательно использовать в связке с модулем Storage, чтобы данные сохранялись в облако. В Unity Editor, Android, Windows и т.д. сохранение будет происходить по средствам стандартного класса Player Prefs.

В скриптах вашего проекта переопределите стандартный класс **PlayerPrefs**. Вставьте следующую строку в начале скрипта:
`using PlayerPrefs = RedefineYG.PlayerPrefs;`
Чтобы быстро найти все скрипты в проетке использующие Player Prefs — можно воспользоваться ссответствующим инструментом вашего **IDE**. Например, в **Visual Studio** открыть поиск по всем скриптам проекта можно сочетанием клавиш **CTRL + SHIFT + F**.

Теперь остаётся только выполнять метод `SaveProgress` или `PlayerPrefs.Save` в ключевых местах.

Метод `PlayerPrefs.Save` — дублирует метод SaveProgress. Если он уже присутствует в нужных местах — отлично.
Когда мы используем методы записи данных, например `PlayerPrefs.SetString`, они меняются в текущей игровой сессии, но не сохраняются в локальное хранилище или облако. Ещё необходимо их сохранять с помощью описанных выше методов. Помните, что слишком часто выполнять методы сохранения не рекомендуется!

Player Stats

Это быстрые сохранения в облако с большим тайм-аутом со стороны платформы Яндекс Игры. Использование PlayerStats схоже с PlayerPrefs, но сохранить данные можно только **integer** типа.

> ⚠ **Внимание:**
>
> Необходимо импортировать модуль "***PlayerStats***".

> 🔥 **Методы:**
>
> - `YG2.SetState(название сейва, integer значение)` - Сохранение значения;
> - `YG2.GetState(название сейва)` - Получение значения;
> - `YG2.GetAllStats()` — вернёт `Dictionary<string, int>` - Получить список всех значений этого типа сохранения.

Изменить весь список сейвов можно с помощью метода `YG2.SetAllStats()`. Таким образом, можно удалить все сейвы.

Загрузку сейвов производит сам PluginYG при запуске игры. Но в специфических ситуациях вы можете воспользоваться методом `LoadStats`. Получение данных после выполнения метода `LoadStats` идёт асинхронно. Поэтому после того как данные будут получены — вызовется ивент `onGetSDKData`.



> 🔥 **Оглавление раздела:**

1. Таблица лидеров
2. Настройка для Яндекс Игр
3. Работа с кодом
4. Создание таблицы в Unity
5. Симуляция отображения таблицы в Unity Editor

6. Time тип

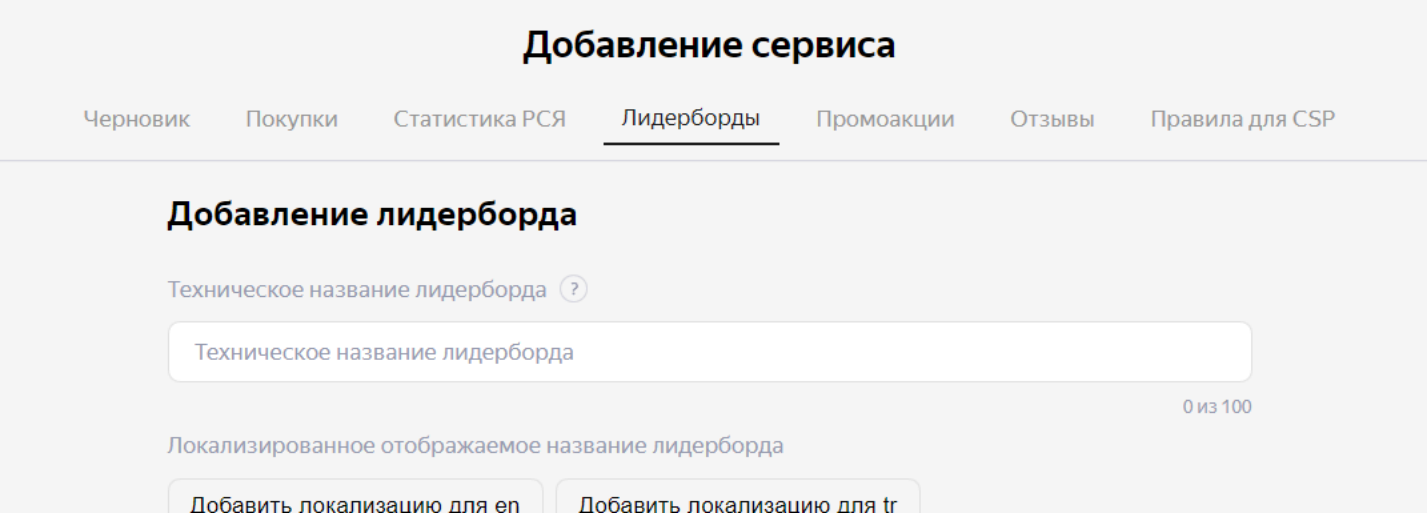
Таблица лидеров

> ⚠ **Обратите внимание:**
>
> Для работы этого модуля необходимы модули Таблица лидеров" и Авторизация.

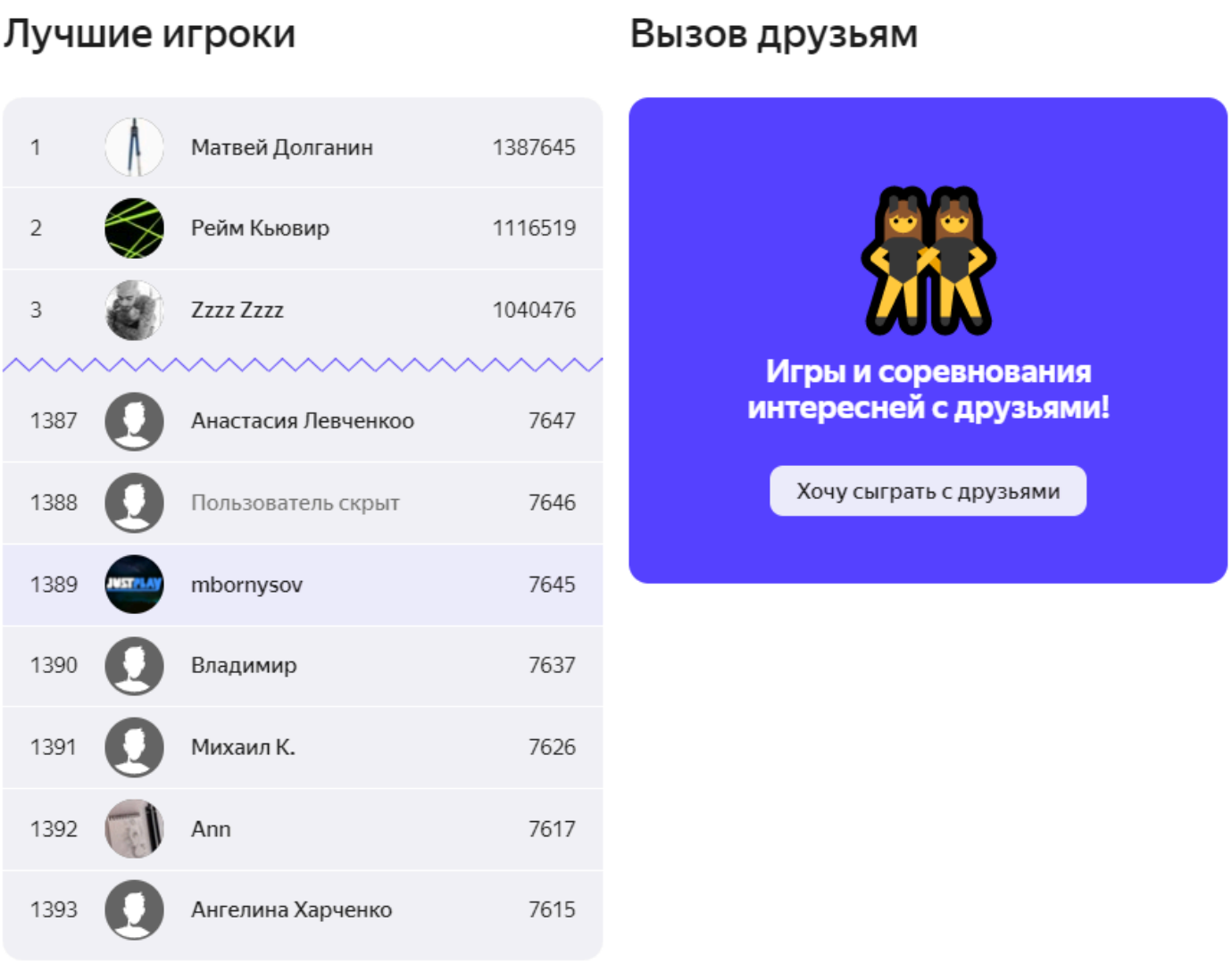
Для отображения таблицы лидеров на страничке игры в Яндекс Играх — достаточно лишь воспользоваться одним методом записи нового рекорда.
С помощью PluginYG2 у вас также есть возможность очень просто добавить таблицу прямо в игру используя один префаб.
И конечно, PluginYG2 предоставляет весь остальной API вплоть до того, чтобы создать кастомную реализацию таблицы внутри игры.

Настройка для Яндекс Игр

В консоли игры, перейдите в раздел «**Лидерборды**» и запишите «**Техническое название**» соревновательной таблицы. «**Локализация наименования**» нам не нужна, а остальные опции имеют пояснения.



После добавления метода записи нового рекорда в таблицу, она будет отображаться следующим образом:



Работа с кодом

Для записи нового рекорда используйте следующий метод:
`YG2.SetLeaderboard(string "техническое название таблицы", int новый рекорд);`
Он имеет ещё третью перегрузку string **extraData**. Можно передать строку, а потом при получении таблицы посмотреть поле её у каждого пользователя из списка.

Пример кода:

```
using YG; // Подключите пространство имён плагина

public class MyClass
{
    public void MyMethod()
    {
        YG2.SetLeaderboard("TechnoNameLB", 100);
    }
}
```

Caution

В таблицу запишется новое число, даже если оно меньше предыдущего. Необходимо самостоятельно выполнять сравнение рекорда перед новой записью.

- **Запрос можно отправлять не чаще, чем один раз в секунду**.
- **Запись рекорда типа long не поддерживается**.

Создание таблицы в Unity

Отображением таблицы занимается компонент **Leaderboard YG**. Все его опции имеют всплывающие подсказки. Обязательно заполните параметр **Name LB** — это техническое название таблицы, которое мы записывали в консоли разработчика.

В папке *PluginYourGames/Modules/Leaderboards/**Prefabs*** расположены примеры таблиц, возьмите их за основу.

Вариант префаба:

Префаб необходимо поместить в Canvas объект.

- При использовании префабов PluginYG рекомендуется создавать свой вариант префаба для избежания проблем при обновлении плагина (что такое вариант префаба).

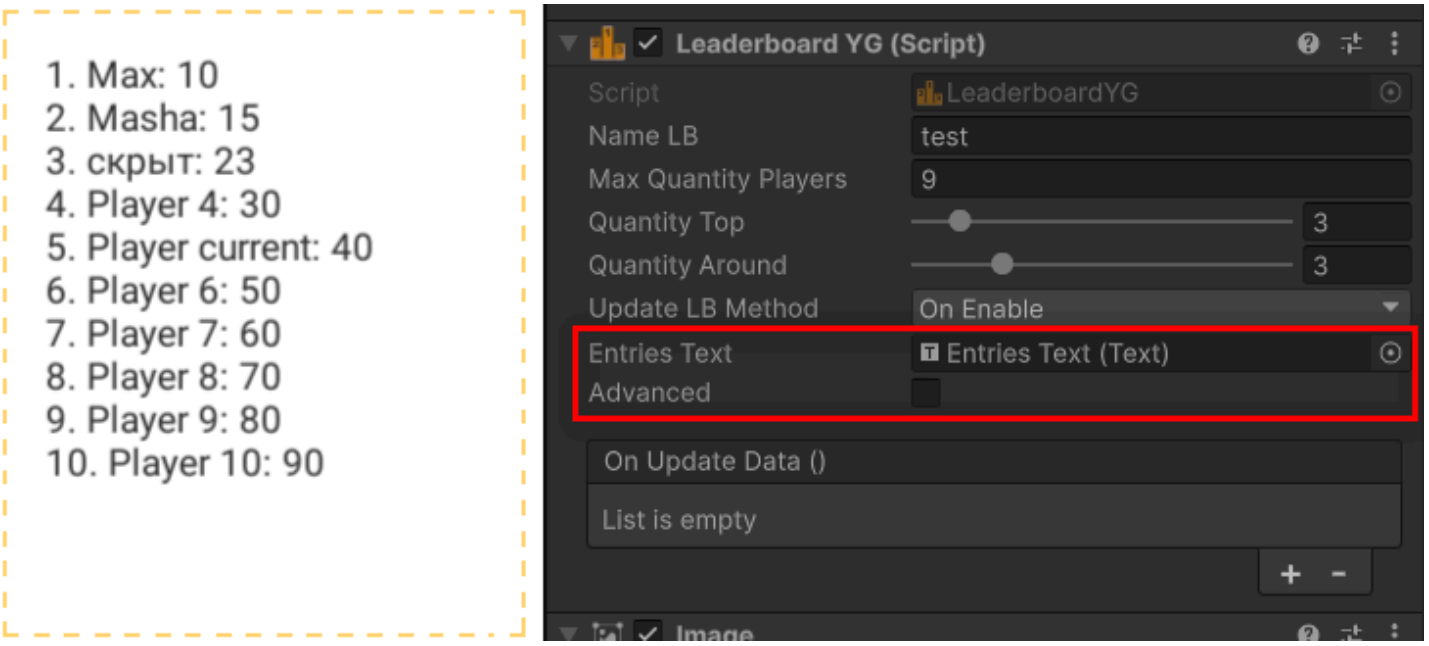
Important

Всё что написано в этом разделе ниже - второстепенно.

Простой вариант отображения таблицы

Для режима **простого** отображения рекордов достаточно указать в скрипте **Leaderboard YG** ссылку на компонент **Text** в опции **Entries Text,** галочка **Advanced** должна быть отключена

для простой таблицы. И скрипт сам будет записывать в него данные таблицы.



Продвинутый (Advanced) вариант отображения таблицы

Отдельными блоками отображает всех игроков. Их рейтинг, аватар, ник, рекорд. Позволяет выделить топ игроков и пользователя аккаунта.

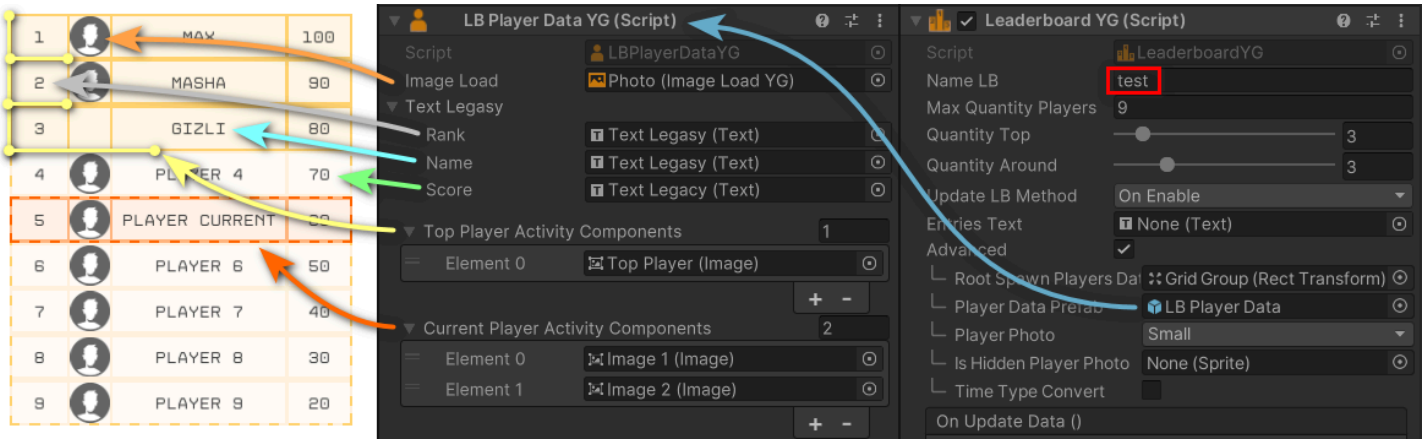
1. Включите галочку **Advanced**.
2. **Entries Text** заполнять необязательно.
3. Укажите **Root Spawn Players Data** — это объект в иерархии которого будут создаваться объекты с компонентом **LB Player Data YG.** Каждый такой объект с данным компонентом — это игрок в таблице (информация о пользователе).
4. Укажите **Player Data Prefab** — это должен быть префаб с компонентом **LB Player Data YG**. Создайте такой префаб по аналогии с готовыми примерами. Поля в **LB Player Data** заполняются по желанию.

У скрипта **Leaderboard YG** есть следующие публичные методы:

- `UpdateLB()` — обновление таблицы
- `SetLeaderboard(int score)` — запись нового рекорда
- `SetLBTimeConvert(float score)` — запись нового рекорда и конвертация в Time тип.

Префаб таблицы **Advanced** создаёт экземпляры префаба из поля **Player Data Prefab**. Объект, который создаётся, это пользователь отображаемый в таблице лидеров. Вы можете

отредактировать поля в скрипте **LBPlayerDataYG**:



> **Вариант префаба:**
>
> Префаб необходимо поместить в Canvas объект.
>
> - При использовании префабов PluginYG рекомендуется создавать свой вариант префаба для избежания проблем при обновлении плагина (что такое вариант префаба).

Симуляция отображения таблицы в Unity Editor

Для отображения данных в таблице в **Unity Editor**, необходимо создать эти данные следующим образом:

В настройках плагина → **Leaderboards** есть массив таблиц для симуляции. Элемент этого массива — это лидерборд со всеми параметрами, и все их настраивать необязательно. Не все данные повлияют на симуляцию. Все данные отображаются, потому что используют тот же класс, в котором хранится информация о реальной таблице. Благодаря этому можно посмотреть какие реальные данные существуют.

1. Создайте таблицу в массиве или измените уже имеющуюся.
2. Запишите **Techno Name.**
3. **Entries** — будет работать для не продвинутой таблицы.
4. **Is Invert Sort Order** — можете установить порядок расстановки по возрастанию или по убыванию. Также, как настраиваете в консоли Яндекс Игр.
5. Для отображения игроков в таблице создайте их, каждый элемент массива это игрок отображающийся в таблице. Заполните данные игрока по своему усмотрению:

- **Rank** — позиция в таблице (в симуляции выставляется автоматически).
- **Score** — рекорд игрока. По рекорду будет выставляться расстановка игроков.
- **Name** — имя игрока. Если назвать `anonymous`, то будет симуляция анонимного игрока.
- **Photo** — это ссылка на скачивание изображения для аватарки.
- **Unique ID** — пригодится для выделения пользователя в таблице, для этого **Unique ID** должен совпадать с одноимённым параметром в симуляции модуля авторизации.

Если не настроить симуляцию, лидерборд внутри **Unity Editor** будет отображать только надпись «Нет данных».

Time тип

1. В консоли разработчика выберите **Time** тип лидерборда.
2. Скорее всего, на первых местах таблицы вы захотите видеть рекорд игрока, который прошёл уровень за наименьшее время (быстрее всех). Для этого установите параметр **Направление сортировки** → **Сортировка по возрастанию**.
3. **Размер десятичной части счета** оставьте на 0.
4. В компоненте **Leaderboard YG** поставьте галочку напротив параметра **Time Type Converter**.
5. При записи нового рекорда вместо метода `SetLeaderboard` используйте метод `SetLBTimeConvert(string "техническое название таблицы", float новый рекорд);`

Новый рекорд вы передаёте в типе float, число может выглядеть, например, так: 180,135. Рекорд записывается в формате «секунды». Например, рекорд «3 минуты» должен записываться как число «180».

Код таймера в вашей игре может выглядеть следующим образом

```
using YG;

class MyClass: MonoBehaviour
{
    float timer;

    void Update()
    {
        timer += Time.deltaTime;
    }

    // Пример записи нового рекорда Time типа
    void NewRecord()
    {
        YG2.SetLBTimeConvert("nameLB", timer);
    }
}
```

Полный API

Есть метод получения данных лидерборда `YG2.GetLeaderboard`, он имеет различные перегрузки.

После выполнения метода `GetLeaderboard` вызовется событие `YG2.onGetLeaderboard`. Событие передаёт класс **LBData**. Для использования данного класса необходимо подключить библиотеку `YG.Utils.LB`. Класс LBData содержит следующую информацию о лидерборде:

- `technoName` — техническое название таблицы
- `entries` — описание таблицы в тексте. Это тот текст, который записывается в таблицу при выборе простого режима (**Advanced = false**).
- `isDefault` — является ли основным лидербордом
- `isInvertSortOrder` — сортировка. false = сортировка по убыванию. true = сортировка по возрастанию
- `decimalOffset` — размер десятичной части счёта. Это число определяет, сколько знаков из целого числа счета должны быть отображены после запятой. Например, значение 5712 в лидерборде при размере десятичной части равном 2 будет отображено как 57.12.
- `type` — тип таблицы: **numeric** или **time**
- `currentPlayer` — класс `LBCurrentPlayerData`. Это информация о текущем пользователе. Имеет параметры: `rank`, `score` и `extraData`.
- `players` — список игроков (`LBPlayerData[]`). Массив игроков в таблице лидеров. Элементы массива содержат информацию о пользователе.

Класс `LBPlayerData` содержит следующую информацию о игроке:

- `rank` — ранк игрока (позиция в таблице)
- `name` — ник пользователя
- `score` — рекорд пользователя
- `photo` — ссылка на аватар пользователя
- `uniqueID` — уникальный идентификатор

Пример кастомной таблицы можно найти в демо сцене **Leaderboards**.

Оглавление раздела:

1. Локализация
2. Auto Translate Langs
3. Шрифты

Локализация

Информация о языке, которую предоставляет SDK платформы, доступна через модуль Данные окружения.

Более расширенный вариант — модуль **Localization**. Он содержит следующие функции:

Опция **Set Language Mod** в настройках модуля — позволяет выбрать как менять язык игры при её запуске из трёх вариантов:

1. Менять язык игры в соответствии с тем, который предлагает SDK платформы — только при первом запуске игры.
2. При каждом запуске.
3. Не запрашивать язык у SDK платформы.

Для Яндекс Игр оставьте стандартное значение «При каждом запуске», такое требование у ЯИ.

В PluginYG2 используется язык в формате *код страны*.
Пример: `ru`, `en`, `tr`. В таком же формате передаётся язык из платформы Яндекс Игры.

`YG2.lang` — язык, который устанавливается в соответствии с SDK платформы при запуске игры. Данное поле может меняться при переключении языка внутри игры.

`YG2.SwitchLanguage(string язык)` — метод переключение языка. Его можно выполнять в меню игры при нажатии на кнопку выбора языка. Необходимо передать параметр `language` — этот язык будет установлен в игре.

После установки языка будет вызвано событие `onSwitchLang` передающее string `language`.

Ещё есть событие `onCorrectLang` также передающее string `language`. Оно вызывается при запросе языка платформы, то есть при запуске игры. Его можно использовать для того, чтобы адаптировать язык под свои нужды. Например, изменить код страны на прописной формат языка для какого-либо ассета локализации.

Пример корректирования языка полученного от SDK платформы:

```
using UnityEngine;
using YG;

public static class CorrectLang
{

&#91RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.BeforeSceneLoad)
&#93
    private static void Init()
    {
        YG2.onCorrectLang += OnChangeLang;
    }

    public static void OnChangeLang(string lang)
    {
        if (lang != "ru" && lang != "en" && lang != "tr")
```

```
        {
            YG2.lang = "en";
        }
    }
}
```

В демо сцене модуля локализации есть примеры, демо скрипты. На подобии скрипта **LanguageExample** вы можете сделать свой компонент для перевода текстов.

Auto Translate Langs

Готовое полное решение для локализации, с автоматическим переводом текстов с помощью API Google Translate. Модуль полностью функциональный, он мощный и может сэкономит много времени. Но данный модуль больше не будет поддерживаться, потому что он перекочевал из первых версий плагина и имеет проблемы. Планируется сделать новый подобный модуль под замену.

Для автоматического перевода с помощью API Google Translate необходимо импортировать пакет **Newtonsoft.NET Json**. В настройках плагина вы найдёте кнопку для быстрого импорта.

Для начала выберите языки, которые будут присутствовать в игре. Для этого в настройках модуля **AutoTranslateLangs** найдите массив **Languages** и выберите языки игры.

За перевод текста отвечает компонент **LanguageYG**. Располагайте его на объекте с компонентом **Text** или **TextMeshPro**. Если компонент текста на объекте не будет найден, то скрипт предложит его создать.

Есть инструмент для перевода всех текстов на всей сцене. Он находится в верхней вкладке **YG2 → Auto Translate Langs → Auto Localization Masse.**

Шрифты

В настройках модуля, в опции **Fonts** есть массивы шрифтов для каждого языка и дефолтный массив. Если вы заполните, допустим, нулевой элемент массива, то в игре шрифты заменятся на тот, что вы указали. Вы также можете закинуть шрифт, допустим, в первый элемент массива. Тогда, если в компоненте **LanguagesYG** вы укажите поле **Font Index** равное одному, то для этого текста будет устанавливаться шрифт из первого элемента массива.

В компоненте **LanguagesYG** есть поле **Unique Font** (уникальный шрифт). Если вы перетащите туда какой-либо шрифт, то данный текст всегда будет грузиться с указанным шрифтом.

Есть инструмент по замене шрифтов на всей сцене на дефолтный из списка шрифтов. Он находится в верхней вкладке **YG2 → Auto Translate Langs → Font Default Masse.**

В настройках модуля массив **Font Size Correct** поможет задать корректировку масштаба для каждого шрифта.

Обратите внимание на компонент **Lang YG Additional Text** — Добавляет строку к переведенному тексту. Через код можно менять поле **Additional Text** и всё корректно применится.



Оглавление раздела:

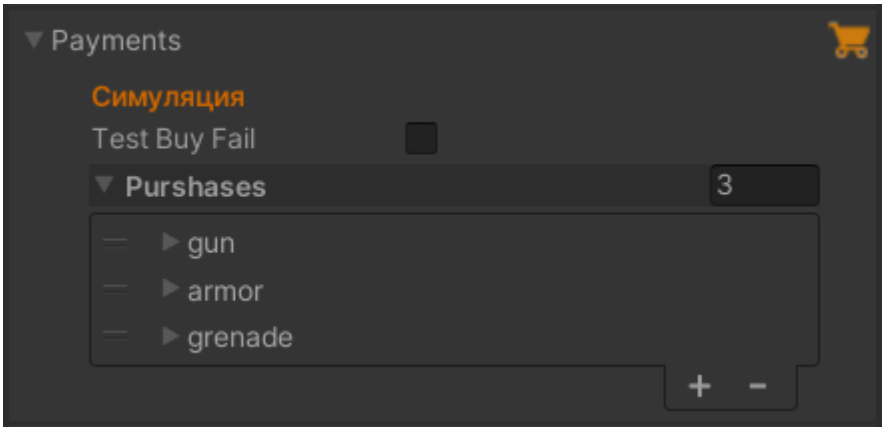
1. Внутриигровые покупки
2. Симуляция отображения покупок в Unity Editor
3. Готовое решение
4. Активация покупки
5. Консумирование (обязательно)
6. Остальное API

Внутриигровые покупки

Для платформы Яндекс Игры прочитайте раздел их документации Внутриигровые покупки.
Там необходимо отправить запрос о подключении и активировать тестовые покупки.
При использовании внутриигровых покупок на платформе Яндекс Игры необходимо сохранять прогресс в облако. Поэтому требуется использование модуль Сохранения или Сохранения > Player Stats]|Player Stats].
После добавления товаров в каталог, можно приступать к настройке инапов в Unity.

Симуляция отображения покупок в Unity Editor

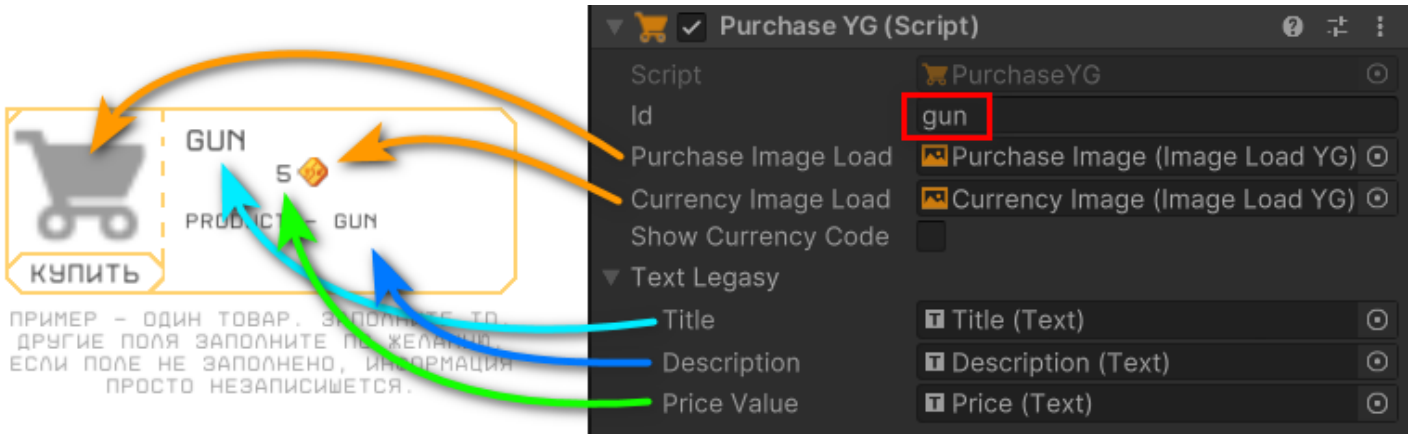
Чтобы плагин симулировал получение каталога товаров в **Unity Editor**, настройте эту симуляцию в свойствах модуля.
Purchases — это массив товаров для симуляции. Создайте или измените элементы массива и их параметры.



Готовое решение

Самый простой способ — использовать готовый префаб **Payments Catalog** из папки модуля **Prefabs**. Он автоматически отображает каталог всех товаров. Он создаёт экземпляры префаба из поля **Purchase Prefab**.
По умолчанию в данном поле ссылка на готовый префаб **Purchase**. Такой префаб должен иметь скрипт **PurchaseYG**

Скрипт **PurchaseYG** — отображает товар, все его данные. Можно использовать готовый префаб **Purchase** для отображения одного продукта. Для этого необходимо заполнить поле **ID**. Настройте остальное по предпочтениям.



Вариант префаба:

Префаб необходимо поместить в Canvas объект.

- При использовании префабов PluginYG рекомендуется создавать свой вариант префаба для избежания проблем при обновлении плагина (что такое вариант префаба).

Активация покупки

Чтобы выдать товар после его приобретения, подпишитесь на событие `YG2.onPurchaseSuccess`. Оно передаёт **ID** покупки.
Сохранять купленные товары рекомендуется с помощью модуля Сохранения > Player Stats]|Player Stats].

Есть событие неуспешной покупки `YG2.onFailedPurchased`, также передаёт **ID** покупки.

Обратите внимание на скрипт **Receiving Purchase Example**, он содержит следующий пример:

Обратите внимание на скрипт "*Receiving Purchase Example*", он содержит пример:

```
private void OnEnable()
{
    YG2.onPurchaseSuccess += SuccessPurchased;
    YG2.onPurchaseFailed += FailedPurchased;
}
```

```
private void OnDisable()
{
    YG2.onPurchaseSuccess -= SuccessPurchased;
    YG2.onPurchaseFailed -= FailedPurchased;
}

private void SuccessPurchased(string id)
{
    // Ваш код для обработки покупки, например:
    string coinsKey = "coins";
    int coins = YG2.GetState(coinsKey);

    if (id == "50")
        YG2.SetState(coinsKey, coins + 50);
    else if (id == "250")
        YG2.SetState(coinsKey, coins + 250);
    else if (id == "1500")
        YG2.SetState(coinsKey, coins + 1500);
}

private void FailedPurchased(string id)
{
    // Покупка не была совершена
}
```

Консумирование (обязательно)

Может произойти ошибка, при которой деньги будут заплачены, но в игре товар не будет добавлен. Это называется необработанная покупка и её нужно консумировать.
При запуске игры необходимо проверять: есть ли необработанные покупки. Если есть, то консумировать их.

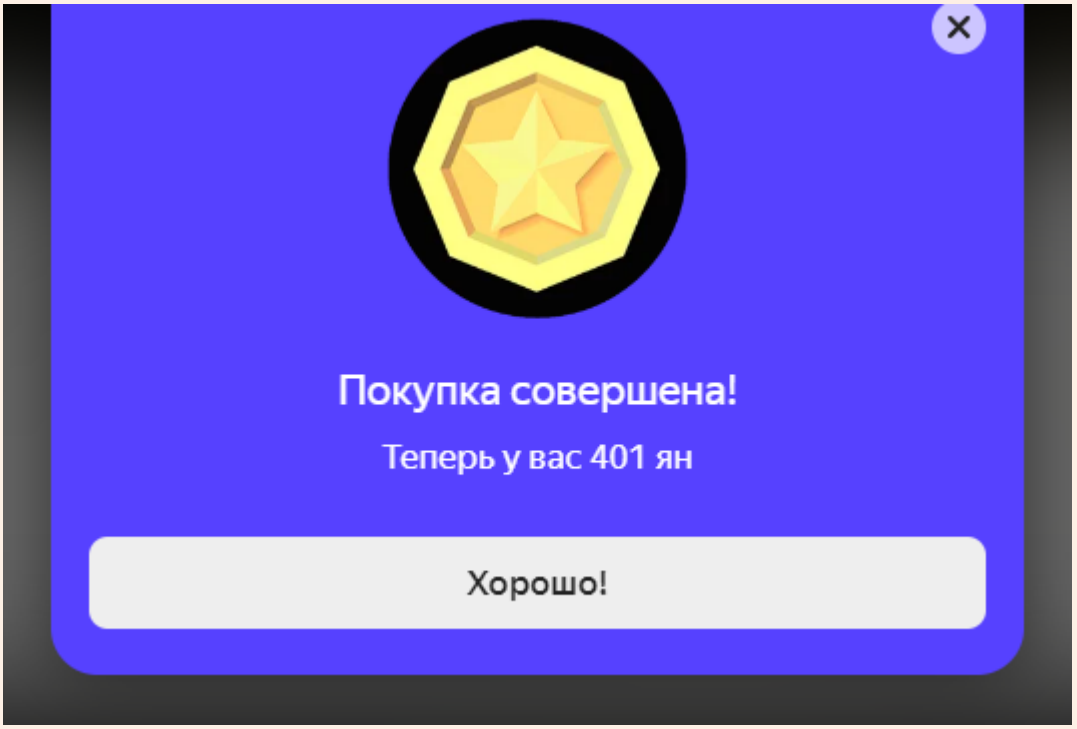
Консумировать — это значит выдать необработанное вознаграждение и сказать об этом платформе. Методы консумирования:
`YG2.ConsumePurchases` — консумировать все покупки
`YG2.ConsumePurchaseByID` — консумировать одну покупку, необходимо передать ID

Оба метода имеют перегрузку `onPurchaseSuccess`.
По умолчанию данный параметр равен **true** — значит для необработанных покупок будет вызвано событие успешной покупки (`onPurchaseSuccess`).
При значении **false** — событие успешной покупки не будет вызвано, в таком случае необходимо самостоятельно выдать продукт. Подробнее.
Если ваши скрипты в момент консумирования будут подписаны на метод получения вознаграждения, то оно будет получено.

Вы можете просто воспользоваться компонентом **ConsumePurchasesYG**. Установите его на какой-либо объект, и когда он будет активен, скрипт проверит и консумирует необработанные покупки, если они есть, и вызовет для них событие вознаграждения. По умолчанию на префабе **Payments Catalog** уже содержится скрипт **ConsumePurchasesYG**.

⚠ **Очень важно:**

Проверьте, действительно ли при запуске игры необработанные покупки выдаются!
В Яндекс Играх, в черновике игры — после совершения покупки не нажимайте на кнопку «Хорошо»:



Перезагрузите страницу. После перезагрузки необработанные покупки должны быть обработаны и выданы.

Если вас не устраивает обработка покупки таким способом, сделайте свою реализацию. PluginYG2 предоставляет список необработанных покупок, об этом ниже в главе **Остальное API**.

Остальное API

После получения данных о покупках от SDK платформы — выполняется метод `onGetPayments`. Он используется для обновления информации о покупках, вызывается при старте игры.

Метод `YG2.BuyPayments(string id)` — открыть окно, в котором можно совершить покупку.

Если подключить пространство имён `YG.Utils.Pay`, то появится доступ к классу `Purchase`. Он имеет следующие поля:
`id` — идентификатор товара.
`title` — название товара.
`description` — описание товара.
`imageURIimageURI` — **URL изображение** товара.

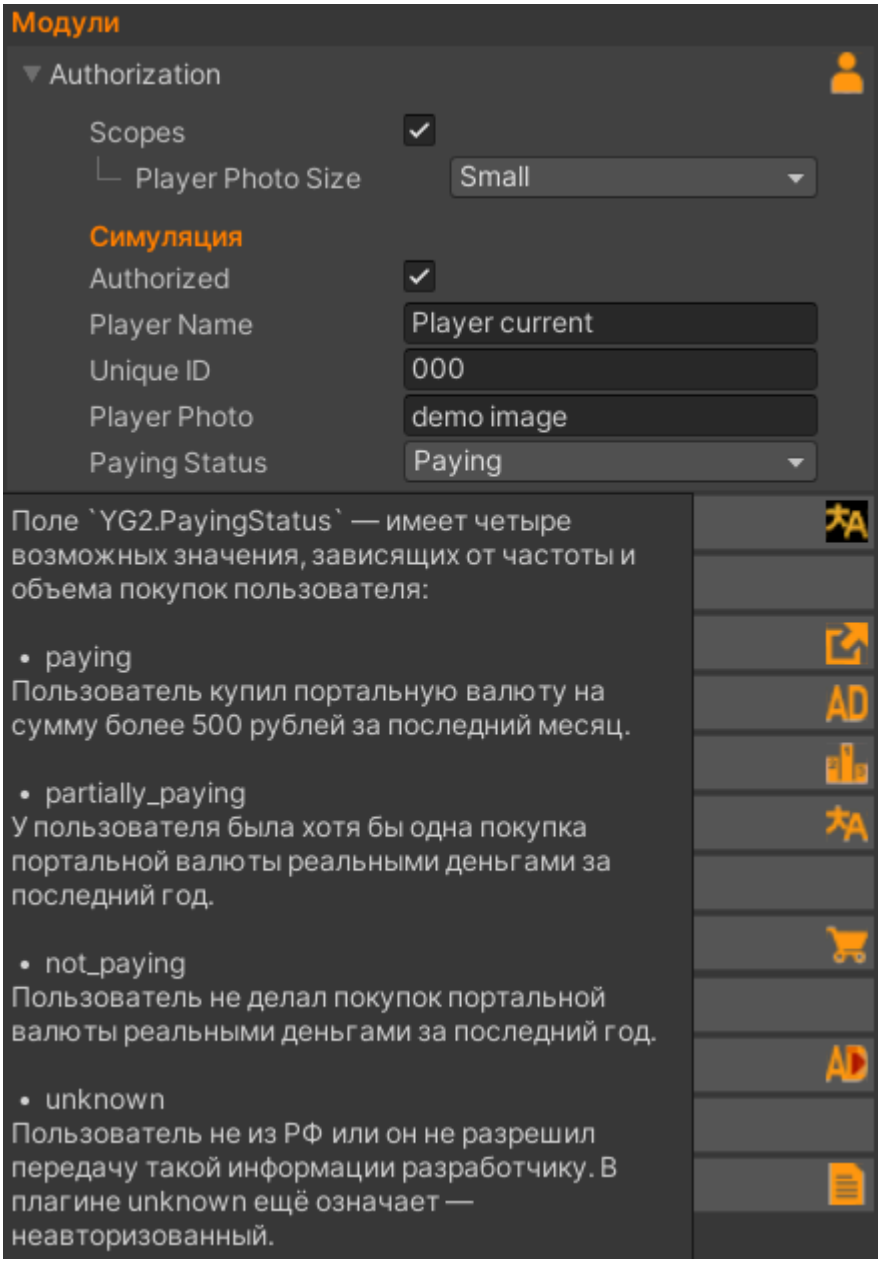
`price` — стоимость товара в формате `<цена> <код валюты>`.
`priceValue` — стоимость товара в формате `<цена>`.
`priceCurrencyCode` — код валюты.
`currencyImageURL` — адрес иконки валюты.
`consumed` — использован ли товар. `true` — использован, `false` — не использован.

Поле `YG2.purchases` — массив всех товаров, тип `Purchase[]`.

Метод `YG2.PurchaseByID` — принимает ID товара, по нему выдаёт информацию о продукте (возвращает экземпляр класса `Purchase`).

Поле `YG2.langPayments` — язык в формате «код страны».

В `YG2.player` (требуется модуль **Autorization**) имеется интересное поле `payingStatus`:





Оглавление раздела:

1. Метрика
2. Яндекс Метрика
3. Методы для отправки событий
4. Использование на разных платформах

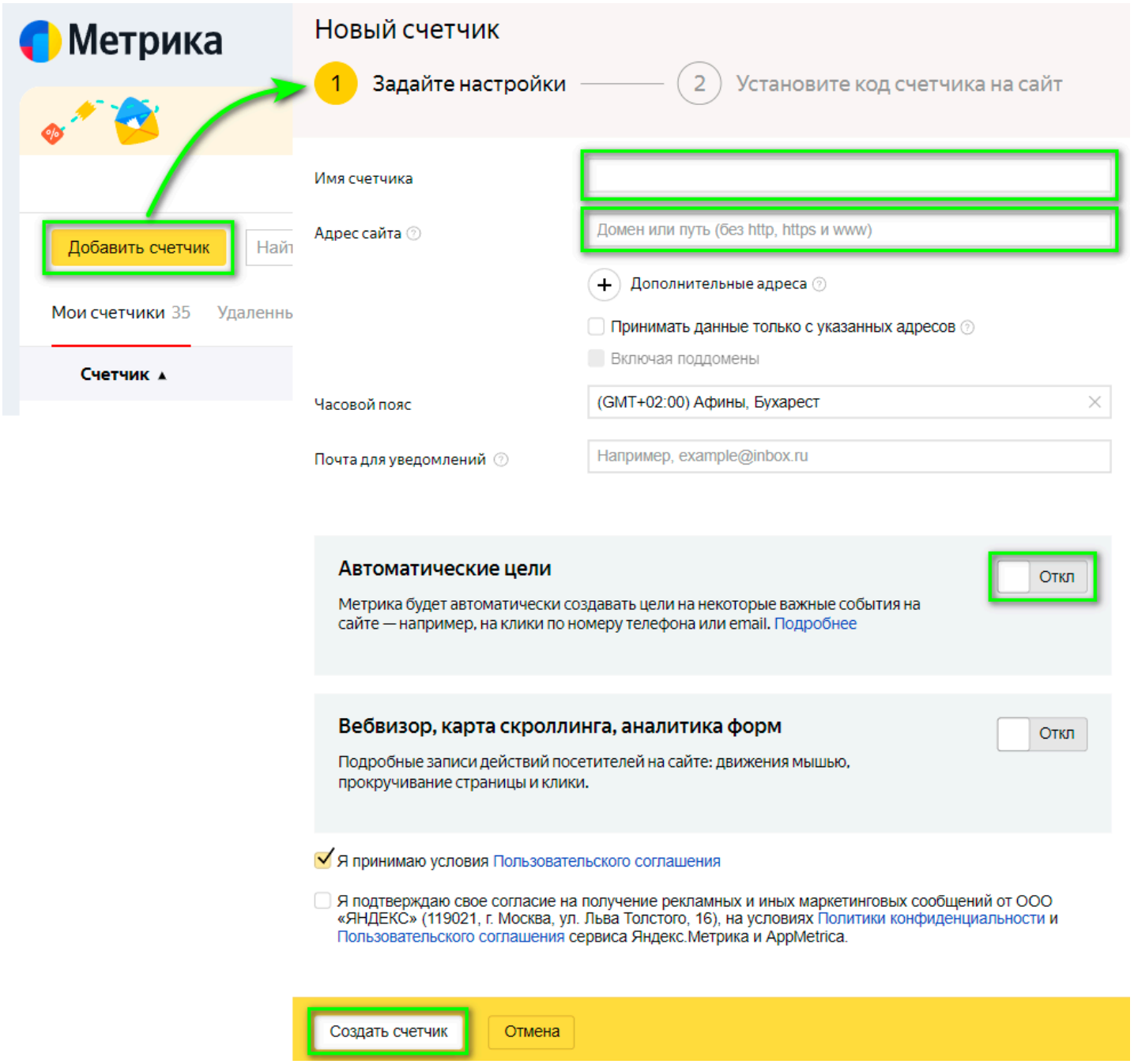
Метрика

Метрика — это инструмент для сбора и анализа данных о поведении пользователей в вашей игре. Она позволяет отслеживать ключевые события, такие как действия игроков, их достижения, поведение в интерфейсе и другие важные показатели, которые помогают понять, как игроки взаимодействуют с игрой, какие механики работают лучше всего, а какие требуют улучшения. С помощью метрики можно улучшить игровые механики, персонализировать контент, а также оптимизировать игровой процесс для разных групп пользователей.

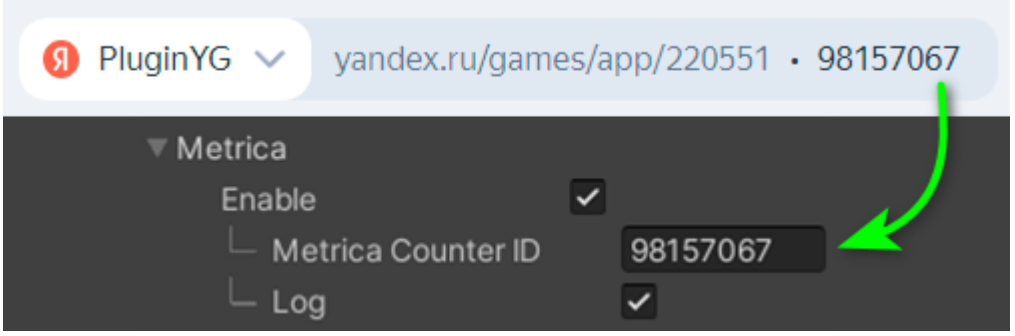
Яндекс Метрика

Чтобы интегрировать метрику в проект для Яндекс Игр, для начала на сайте Яндекс Метрика создайте счетчик.
Укажите имя счётчика (например название игры)

Адрес сайта запишите в формате `yandex.ru/games/app/012345`



Затем скопируйте ID счётчика и вставьте его в поле **Metrica Counter ID** (Настройки плагина, раздел Metrica):



На этом этапе ваша игра уже будет подключена к Яндекс Метрике. Теперь вы можете смотреть метрики игры, например, количество игроков.

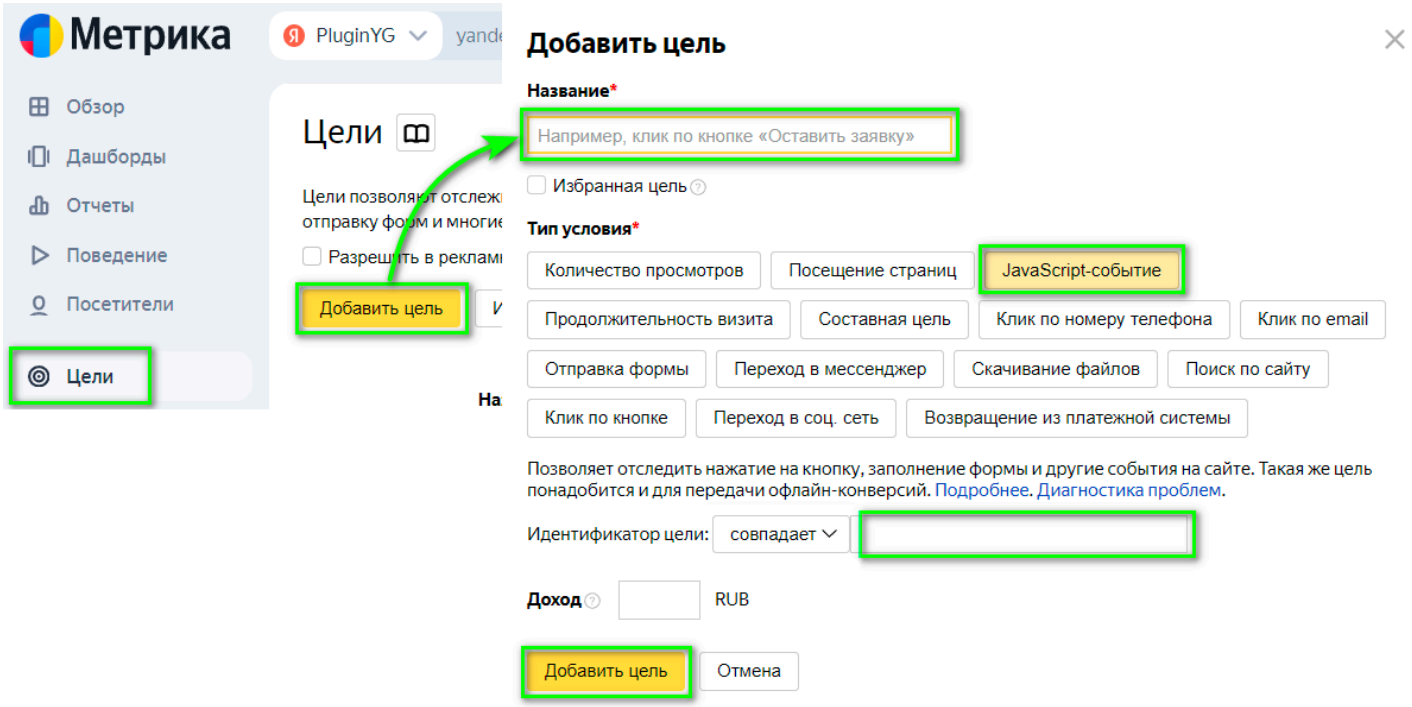
Обратите внимание!

Данные на сайте обновляются с задержкой!

Далее рассмотрим настройку целей, чтобы из игры можно было отправлять события и отслеживать активность любых действий в вашей игре.

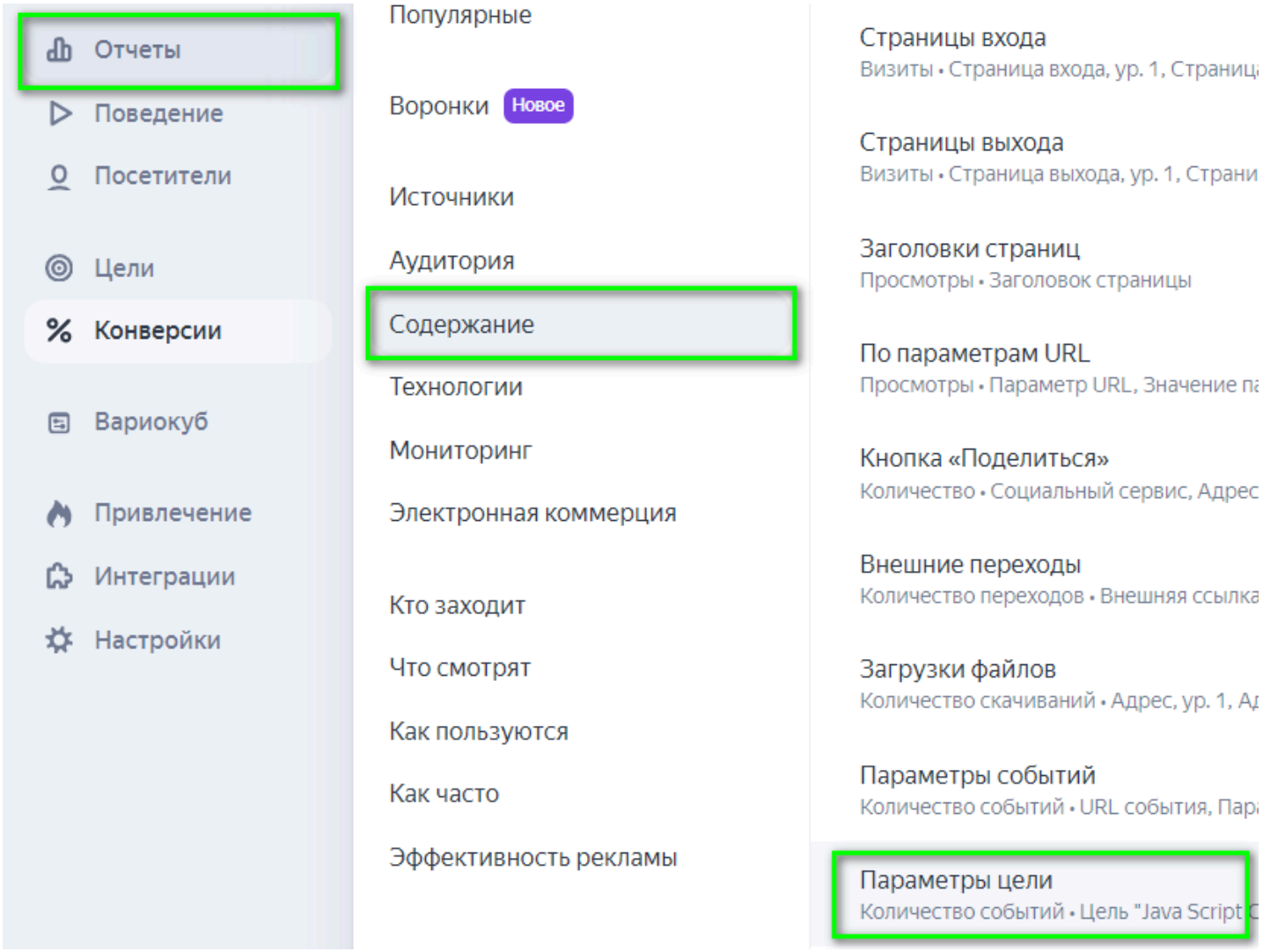
Создайте цель и настройте:

1. Запишите название цели.
2. Тип условия выберите JavaScript-Событие.
3. Идентификатор цели: «**совпадает**».
4. Заполните идентификатор цели (строго на английском языке и без пробелов).
5. Нажмите кнопку «**Добавить цель**».



После того, как из игры будут отправлены события через методы PluginYG2, вы сможете посмотреть их во вкладке **Конверсии**. Для просмотра вложенных параметров перейдите в

раздел **Параметры цели**:



Методы для отправки событий

Метод `MetricaSend(string eventName)`
Этот метод отправляет событие в Яндекс.Метрику, не передавая дополнительных данных. Он подходит для простых случаев, когда вам нужно только зафиксировать событие. Пример:
`YG2.MetricaSend("level_up");`
eventName: строка, представляющая название события (например, `level_up`).
Метод `MetricaSend(string eventName, Dictionary<string, string> eventData)`
Этот метод позволяет передать данные в виде ключ-значение, где оба элемента — строки. Используется для более простых случаев, когда не требуется передавать сложные вложенные объекты. Пример:

```
var eventData = new Dictionary&ltstring, string&gt
{
    { "action", "click" },
    { "element", "button" }
};
YG2.MetricaSend("ui_interaction", eventData);
```

eventName: строка, представляющая название события.
eventData: словарь, где ключи и значения — строки (например, `{"action": "click",`

`"element": "button"}`).
Метод `MetricaSend(string eventName, Dictionary<string, object> eventData)`
Этот метод более универсален и позволяет передавать данные в виде словаря, где значения могут быть не только строками, но и другими типами данных (например, числами, массивами и даже вложенными словарями).
Пример:

```
var eventData = new Dictionary&ltstring, object&gt
{
    { "action", "click" },
    { "element", "button" },
    { "user", new Dictionary&ltstring, object&gt
        {
            { "level", 5 },
            { "experience", 1020 }
        }
    }
};
YG2.MetricaSend("ui_interaction", eventData);
```

eventName: строка, представляющая название события.
eventData: словарь, где ключи — строки, а значения могут быть любыми типами данных (`int` , `float` , `bool` , другие словари и т. д.).
Метод `MetricaSend(string eventName, string param1, string param2)`
Этот метод предназначен для передачи простых данных, когда нужно отправить имя события и два уровня параметров. Он позволяет удобно передавать двухуровневую информацию.
Пример:
`YG2.MetricaSend("level_up", "level", "5");`
eventName: строка, представляющая название события.
param1: строка, которая будет использована в качестве имени параметра первого уровня.
param2: строка, которая будет использована в качестве значения параметра второго уровня.
Метод `MetricaSend(string eventName, string nestedParam, Dictionary<string, object> subNestedData)`
Этот метод такой же, что и предыдущий, но вложение второго уровня передаётся объектом. Таким образом можно удобно отправить метрики для одного параметра второго уровня с передачей данных этого параметра.

Пример использования:

```
YG2.MetricaSend("levelsFinish", "1", new Dictionary<string, object>
{
    { "stars", 3 },
```

```
        { "time", 10 }
    });
```

eventName: строка, представляющая название события.
param1: строка, которая будет использована в качестве имени параметра первого уровня.
param2: словарь, где ключи — строки, а значения могут быть любыми типами данных (`int` , `float` , `bool` , другие словари и т. д.).

Пример использования

```
using System.Collections.Generic;
using UnityEngine;
using YG;

public class MyClass : MonoBehaviour
{
    // Метод, который будет вызван при старте игры или на определенном событии.
    public void Start()
    {
        // Создаем словарь с дополнительной информацией, которая будет
отправлена в метрику.
        var eventData = new Dictionary&ltstring, object&gt
        {
            { "action", "click" },
            { "element", "button" },
            { "user", new Dictionary&ltstring, object&gt
                {
                    { "level", 5 },
                    { "experience", 1020 }
                }
            }
        };

        // Отправляем событие с названием "ui_interaction" и данными eventData.
        YG2.MetricaSend("ui_interaction", eventData);
    }
}
```

Также примеры можно найти внутри модуля в скрипте `YandexMetricaExample` .

Использование на разных платформах

Яндекс Метрику можно использовать не только на Яндекс Играх, но и на других площадках, которые поддерживают интеграцию Яндекс Метрики. Возможно, для некоторых платформ

потребуется добавить хост metrika.yandex в белый список. Узнайте подробности по необходимым платформам.
Чтобы использовать другой сервис для аналитики, необходимо переопределить скрипты Яндекс Метрики. Это можно сделать самостоятельно, изучив документацию для Платформы > Создание платформы.

Яндекс Метрика активна на всех платформах по умолчанию, если не переопределена. Её можно вырезать для отдельных платформ. Для этого в настройках платформы активируйте опцию ***Use Yandex Metrica*** и уберите флажок для платформ, для которых требуется отключить Яндекс Метрику.
Также можно использовать разные счётчики для разных платформ. Но это может быть излишне, т.к. для просмотра метрик с разных площадок, в сервисе можно просматривать метрики по отдельным сайтам.

Если метрика не работает на определённой платформе, возможно в настройках счётчика включена опция «Принимать данные только с указанных адресов». В таком случае выключите её, или добавьте все необходимые ссылки.



Оглавление раздела:

1. Ссылки на игры и разработчика
2. Работа с кодом
3. Готовые решения

Ссылки на игры и разработчика

Некоторые платформы, вроде Яндекс Игр - предоставляют возможность из текущей игры перейти по ссылкам на другие игры и на страницу разработчика.
Благодаря этому можно расширить аудиторию всех ваших игр.
Можно отображать список всех доступных игр для текущего пользователя с помощью лишь одного префаба.
А если подключить модуль флагов, то отображение игр в списке можно контролировать удаленно.

Работа с кодом

`YG2.OnDeveloperURL()` - переход по ссылке на страницу разработчика.
`YG2.OnGameURL(int gameID)` - переход по ссылке к определённой игре. Требуется передать идентификатор игры. Его можно найти в ссылке игры и в консоли разработчика.
`OnURL(string link)` - переход по любой ссылке.

Эти методы удобно использовать через компоненты **EventsYG2**, **AllGamesYG**, **GameYG**.

`YG2.allGames` - массив всех игр доступных для текущего пользователя. Тип массива - класс **GameInfo**.
`YG.Utils.OpenURL.GameInfo` - класс содержащий данные игры. Для использования класса необходимо подключить пространство имён `YG.Utils.OpenURL`. В классе содержатся следующие данные:
`appID` - идентификатор
`title` - название игры
`url` - адрес игры
`coverURL` - адрес обложки
`iconURL` - адрес иконки

`YG2.GetGameByID(int gameID)` - возвращает класс **GameInfo** игры.

Готовые решения

В папке модуля → Prefabs есть демонстрационные префабы. Используйте их для примера. Префаб **Game YG** имеет одноимённый компонент, который отображает информацию о игре. Он может использоваться как для отображения одной игры, так и в связке со скриптом **All Games YG** для автоматического отображения списка таких объектов со скриптом **Game YG**.

> **Вариант префаба:**
>
> Префаб необходимо поместить в Canvas объект.
>
> - При использовании префабов PluginYG рекомендуется создавать свой вариант префаба для избежания проблем при обновлении плагина (что такое вариант префаба).

Отображение одной игры

Для отображения одной игры - используйте один префаб или скрипт **Game YG**. При этом необходимо включить опцию **Load By App ID** и настроить вытекающие параметры. Обязательно укажите идентификатор игры!

Симуляция

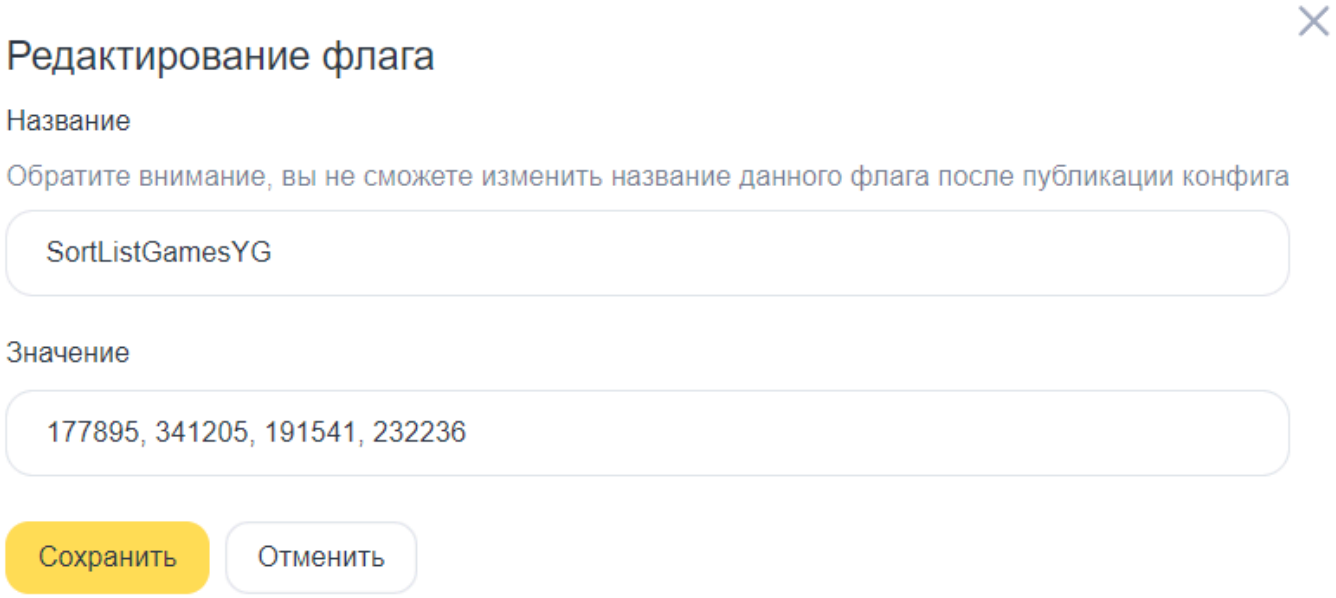
Чтобы игры отображались в Unity Editor, заполните соответствующую информацию в настройках модуля **Open URL**. Массив **All Games** симулирует список доступных игр, который предоставляет платформа. Чтобы отобразить одну игру с помощью компонента **Game YG** недостаточно просто в него записать **ID**. Ещё необходимо создать игру с таким ID в списке симуляции.

Отображение списка игр

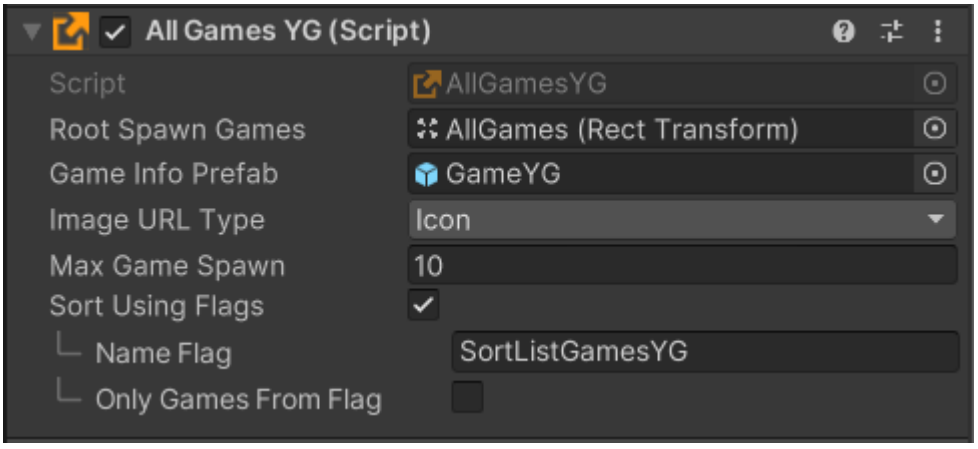
Используйте префаб **All Games**. Он создаёт копии объекта из поля **Game Info Prefab**. В данное поле нужно поместить ссылку на префаб со скриптом **Game YG**.

Чтобы отображать список игр, который можно контролировать в удалённой конфигурации - воспользуйтесь флагами.

1. Флаги (удалённая конфигурация) с названием, например - `SortListGamesYG`.
2. В значение запишите список идентификаторов игр через запятую. В игре список будет отображаться в том же порядке, который будет в значении флага.



3. В компоненте **All Games YG** включите опцию **Sort Using Flags**.
4. Запишите название флага.
5. Когда опция **Only Games From Flag** включена - будет отображаться только список доступных игр из перечисления в значении флага. Когда опция выключена - после отображения игр из значения флага, список будет продолжаться отображением доступных игр в случайном порядке. Таким образом, с помощью флагов можно просто обозначить какие игры будут отображаться в начале.





Оглавление раздела:

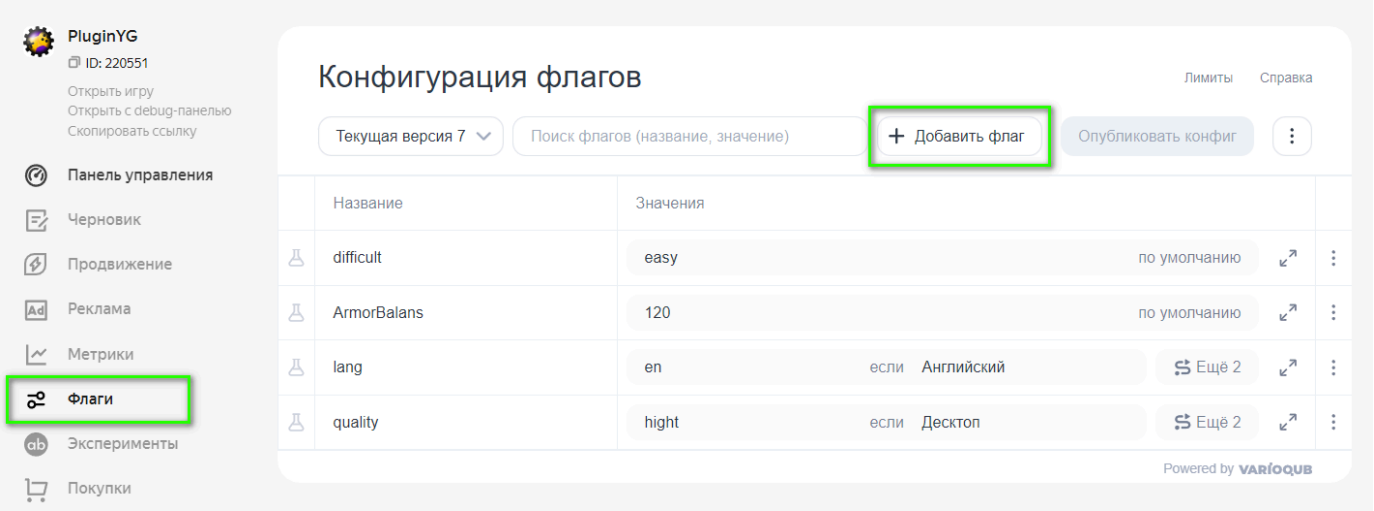
1. Флаги (удалённая конфигурация)
2. Создание флагов в консоли Яндекс Игр
3. Использование в Unity
4. Симуляция в Unity Editor

Флаги (удалённая конфигурация)

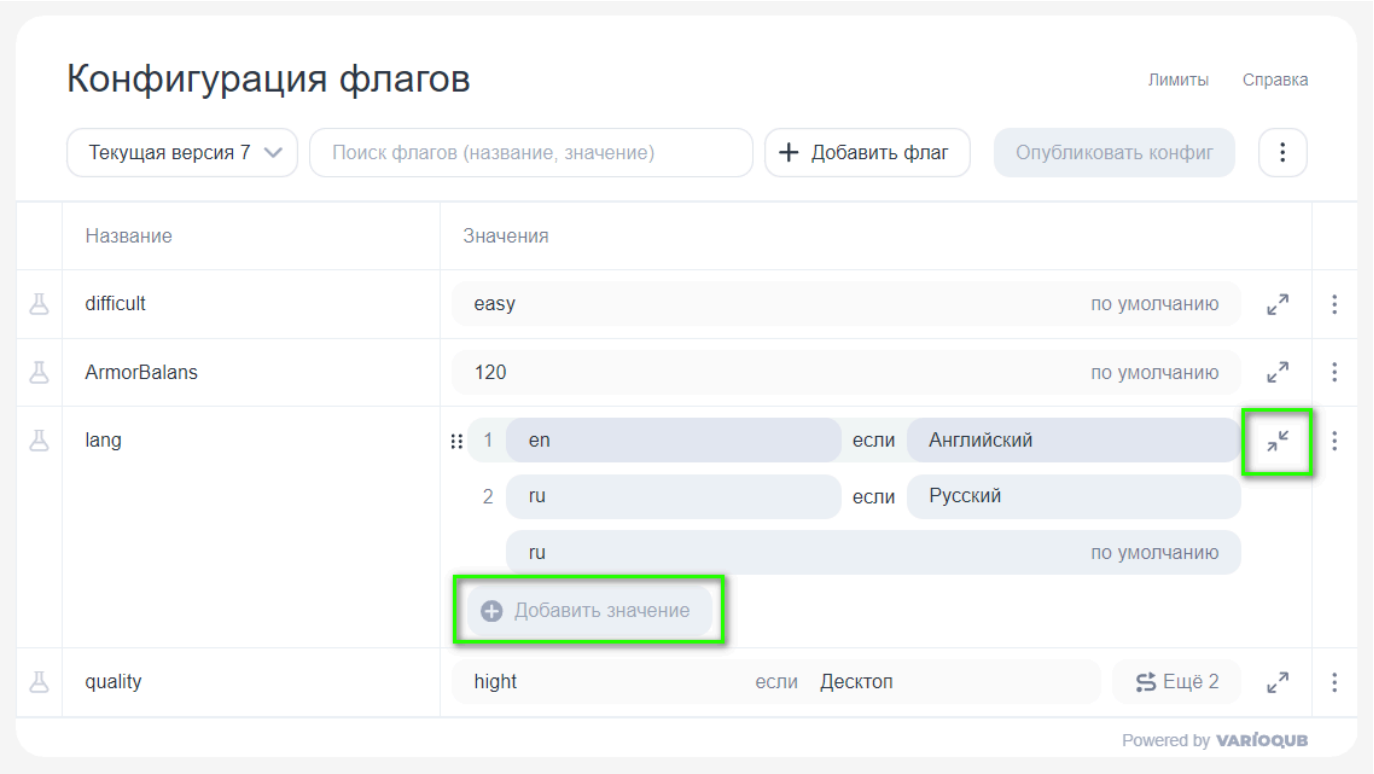
Флаги — это очень мощный инструмент. Вы можете через консоль разработчика в любое время менять какие-либо данные игры.
Представим, что в нашей игре есть переменные отвечающие за какие-то характеристики персонажа, например — сколько у него будет здоровья и силы. Изменить такие переменные в игре можно удалённо, не обновляя игру!
Таким образом, без изменения билда можно настраивать баланс в игре, безопасно внедрять новые обновления, проводить акции.
И в прибавок к этому — в Яндекс Играх есть возможность проводить тестирование с разными параметрами!

Создание флагов в консоли Яндекс Игр

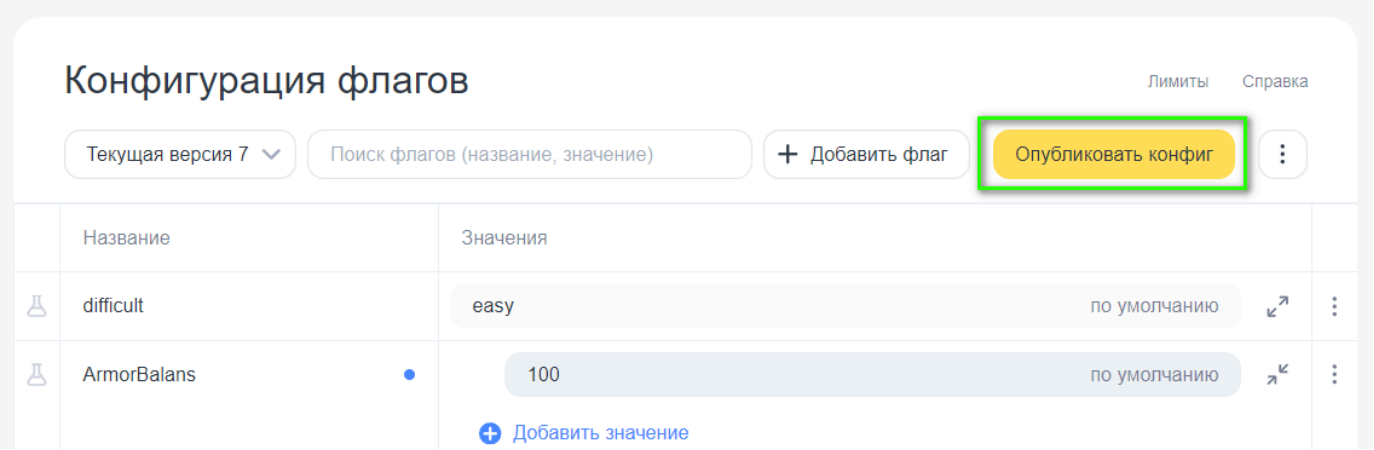
1. В разделе **Флаги**, добавьте флаг:



2. Укажите название и значение флага. В вашем коде, флаг вы будете искать по названию. А значение можно менять через консоль и оно будет передаваться в игру.
3. Также вы можете добавить условия. Например, если язык русский, то значение будет равно ru.



4. Чтобы изменения вступили в силу, необходимо их опубликовать:



Использование в Unity

Получить значение флага можно с помощью метода `YG2.GetFlag(string)`.
Метод `GetFlag` принимает параметр (`string` имя флага), и возвращает значение флага.

Пример получения и обработки флагов:

```
// Допустим, из облака мы получаем уровень сложности

string value = YG2.GetFlag("difficult");  // difficult — имя флага

if (value == "easy")
{
    // Установите лёгкий уровень сложности.
}
else if (value == "hard")
{
    // Установите сложный уровень сложности.
```

```
}
else
{
    // Значение флага не определено, установите дефолтное значение.
    // Если значение не определено, метод GetFlag вернёт null.
}
```

Методы, преобразующие значение флага в другой тип данных:

```
if (YG2.TryGetFlag("difficult", out string difficult))
{
    // Флаг существует, пользуемся им!
    Debug.Log(difficult);
}

if (YG2.TryGetFlagAsInt("intType", out int intType))
{
    // Флаг существует и преобразован в тип int
}

if (YG2.TryGetFlagAsFloat("floatType", out float floatType))
{
    // Флаг существует и преобразован в тип float
}

if (YG2.TryGetFlagAsBool("boolType", out bool boolType))
{
    // Флаг существует и преобразован в тип bool
}
```

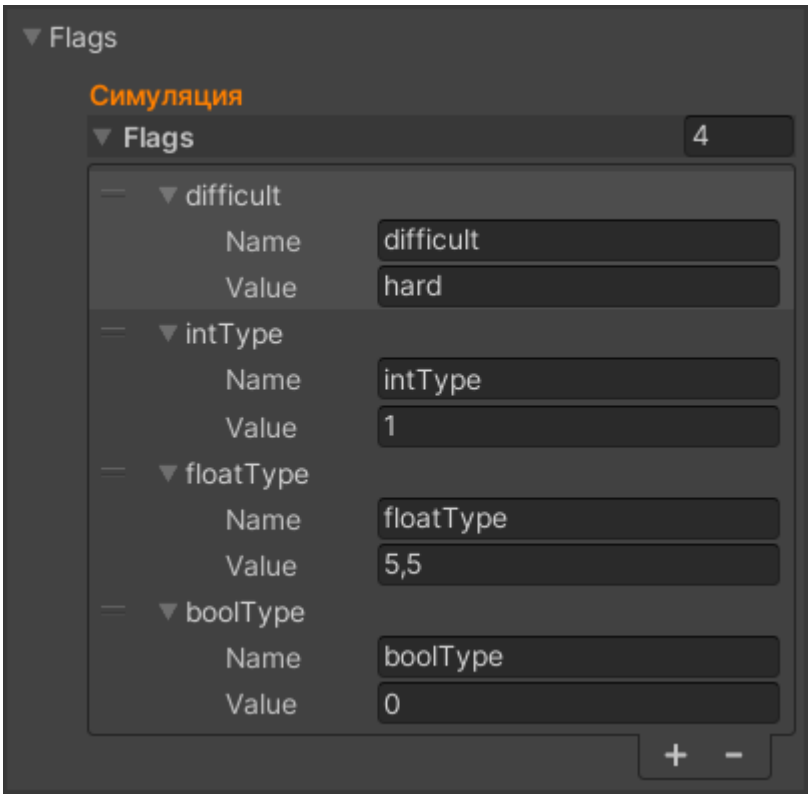
В представленных методах - качественное преобразование. Изначально строковое значение может иметь опечатки в виде лишних символов. Лишние символы будут удалены, благодаря чему преобразование в определённый тип будет произведено.
Для числовых значений можно использовать знак минус.
Для `float` после целого числа можно использовать как точку, так и запятую.
`boolean` поддерживает не только `true` и `false`, а также `0` и `1`.

Все флаги можно получить из словаря `YG2.flagsDictionary`. Доступно только для чтения.
Также есть массив `YG2.flags` — в котором элементы массива состоят из структуры **Flag**. Она имеет два поля: **name** и **value**. Доступно чтение и запись.

Подобный пример можно найти в скрипте **FlagsExample**. Он используется в демо сцене **Flags**.

Симуляция в Unity Editor

В настройках модуля есть массив **Flags**. Это массив из структуры **Flag**, которая описана выше. Создайте флаги подобно созданию конфигурации в консоли разработчика, и в Unity Editor можно будет протестировать работу флагов в вашей игре.





Оглавление раздела:

1. Порт игры на TV
2. Отслеживание нажатий кнопок на пульте
3. Кнопка «назад» и выход из игры
4. Симуляция
5. Навигация по кнопкам
6. Навигация без использования слоёв
7. Динамические кнопки
8. Описание параметров компонента Menu Navigation

Порт игры на TV

Для начала ознакомьтесь с первой главой документации Яндекса для игр под TV.
PluginYG покрывает все требования платформы.

Рекомендую для примера демо сцену, которую можно найти внутри папки с модулем **TV**.

Есть симуляция на ПК, которая работает в Unity Editor и её можно активировать для тестирования в браузере. А еще есть система навигации по кнопкам, которую можно использовать не только для адаптации под TV, но и просто для игры под ПК. Навигация осуществляется с помощью клавиатуры или мыши.

Отслеживание нажатий кнопок на пульте

При нажатии кнопки на пульте — сработает событие `onTVKeyDown<string key>`.

Пример:

```
using YG;

private void OnEnable()
{
    YG2.onTVKeyDown += OnKeyDown;
}

private void OnDisable()
{
    YG2.onTVKeyDown -= OnKeyDown;
}

private void OnKeyDown(string key)
{
    switch(key)
    {
        case "Left":
            // Была нажата кнопка "стрелка налево"
            break;
        case "Right":
            // Была нажата кнопка "стрелка направо"
        break;
    }
}
```

Также можно подписаться на событие отжатия кнопки `onTVKeyUp<string key>`.
События возвращают строку с названием кнопки.

Весь список возвращаемых кнопок:

- `Up`
- `Left`
- `Down`
- `Right`
- `Enter`
- `MediaRewind`
- `MediaPlayPause`
- `MediaFastForward`

Кнопка «назад» и выход из игры

Для кнопки назад используется отдельный ивент `onTVKeyBack`.

Выход из игры осуществляется двойным нажатием на кнопку пульта **Back**. Так должно быть по правилам Яндекс Игр.
Если нажать на пульте кнопку **Back** до того как игра загрузится, то выход из игры будет при одном нажатии кнопки.

Для выхода из игры используйте метод `ExitTVGame();`.

Симуляция

В настройках плагина → **General Simulation** в опции **Device** выберите **TV**.

Для симуляции на сайте в браузере — к ссылке игры добавьте: `&payload=tvtest`. Должно получиться примерно так: `https://yandex.ru/games/app/123456&payload=tvtest`

В симуляции следующие кнопки на клавиатуре будут играть роль пульта: ←↑→↓ или WASD, Enter, Backspace.
Отключить управление WASD можно поправив скрипт `TVKeyController`.

Навигация по кнопкам

Вы можете воспользоваться навигацией от плагина. Плагин в свою очередь использует навигацию Unity. Вы можете настроить её для кнопок в компоненте **Button**:

В папке модуля **Example** → **Prefabs** найдите префаб **Menu Navigation**. На нём прикреплены компоненты Menu Navigation, Visual Select Button и Cursor Select Button.

Menu Navigation

Главный компонент контролирующий навигацию по кнопкам. Скрипт действует по логике слоёв кнопок. При старте сцены все активные кнопки что есть на сцене добавляются в нулевой слой. При открытии окна/нового меню в игре, кнопки из нулевого слоя перестают быть интерактивными, чтобы на них нельзя было нажать и навести фокус. А кнопки содержащиеся в новом открытом окне — добавляются на новый слой. При закрытии окна, последний слой от

этого окна удаляется, а кнопки из предыдущего слоя активируются. При нажатии на кнопку **Back** последний активный слой закрывается.

Использование системы слоёв

Для открытия и закрытия окна/нового слоя - используются методы `OpenWindow(GameObject openLayerObj)` и `CloseWindow`.
Допустим, в игре есть главное меню с кнопками «играть», «настройки» и т.д. Это будет нулевой слой.
На событие клика по кнопке «настройки» повешайте метод **OpenWindow** и укажите объект, который будет являться новым окном/слоем.
Для закрытия последнего открытого слоя используйте метод **CloseWindow**.

Дополнительные методы для добавления и удаления одной кнопки из списка слоёв

Методы `AddButtonList` и `RemoveButtonList` для добавления и удаления из списка слоёв. В метод необходимо передать ссылку на кнопку. Есть перегрузка, в которой можно указать номер слоя. Если не заполнить, то слой будет нулевым.

Навигация без использования слоёв

Необязательно использовать данную систему слоёв. Но нужно понимать, что необходимо грамотно настроить меню, чтобы перемещение по кнопкам было корректным.

Если не использовать слои, то выделение мышью будет выделять кнопки только те, что были активными при старте сцены, то есть кнопки нулевого слоя.

Чтобы новые открытые кнопки так же выделялись мышкой без использования слоёв, можно использовать компонент **Button Dynamic Select**. Подробнее о нём ниже.

Динамические кнопки

Добавьте компонент **Button Dynamic Select** на объект с кнопкой, которая может динамически появляться и исчезать. При активации такая кнопка будет добавляться в последний из слоёв.

Описание параметров компонента Menu Navigation

- **Us Button** — укажите кнопку, на которую будет направлен фокус при старте сцены (необязательно).
- **Exit Game Obj** — укажите объект, который будет становиться активным, при нажатии на кнопку Back. Используйте это для открытия меню выхода из игры. Меню выхода откроется, если не открыты другие слои. В ином случае последний открытый слой закроется.
- **Navigation On Desktop** — использовать ли навигацию на ПК. В таком случае визуальное выделение кнопки будет только в момент наведения мыши на кнопку. Но если на клавиатуре нажать кнопки навигации, то визуальное выделение останется.
- **Navigation On Mobile** — использовать ли навигацию на мобильных устройствах. Визуального выделения не будет, но сохранятся звуки которые вы возможно повесили на

события нажатия кнопки.

- Событие **On Back Button** — привяжите своё событие вместо активации объекта - меню выхода из игры. Подходит для ситуации, когда открыта сцена — например уровень. И надо выйти из уровня в главное меню, а не из игры.
- **Layers** — заполнять не нужно. Показывает слои когда игра запущена.

Остальные компоненты дополнительные, вы можете сделать свои скрипты на их основе.

- **Visual Select Button** — визуализирует обводку над кнопкой. Требуется указать спрайт и размер. Также можно указать — будет ли отрисовка на ПК или мобильных устройствах.
- **Cursor Select Button** — выделяет кнопки при наведении на них мышкой. У данного компонента есть Unity Events для привязки своих событий. Например, вы можете проигрывать звук при наведении и нажатии.

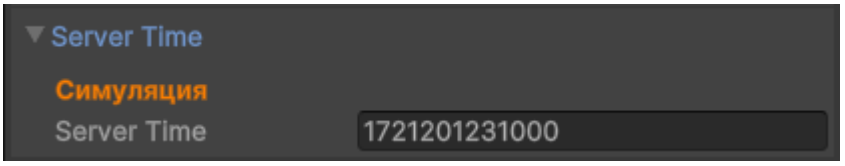


Оглавление раздела:

1. Серверное время
2. Мультиплатформенность

Серверное время

С помощью серверного времени вы можете сделать ежедневную награду за посещение игры, или что то похожее.
Метод `YG2.ServerTime()` возвращает значение типа `long` серверного времени в миллисекундах.
Для симуляции в Unity Editor значение серверного времени используется из поля **Server Time** в настройках модуля:



Мультиплатформенность

Яндекс Игры предоставляют возможность получить время со своего сервера, и PluginYG берёт эти данные.

В большинстве случаев другие платформы не предоставляют API для серверного времени. Плагин имеет стандартную реализацию для получения ServerTime. Это значит, что на любой

платформе которая не предоставляет API, будет использоваться следующий код плагина:
`DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();`



Оглавление раздела:

1. Оценка игры

Оценка игры

Вы можете попросить пользователя оценить игру и написать комментарий во всплывающем окне (появится в момент запроса оценки, закрывая фон приложения). Всплывающее окно не будет показано, если пользователь не авторизован или оценивал игру ранее.

Чтобы вызвать окно для оценки игры используйте метод `YG2.ReviewShow()`. Он откроет окно для оценки игры, если такая опция доступна.
Есть поле `YG2.reviewCanShow` — доступность отзыва.

В папке модуля → Prefabs есть префаб **Review** со скриптом **Review YG** — это готовая кнопка отзыва. Она будет деактивирована, если отзыв не доуступен.

Скрипт имеет четыре события:

1. **Review Available** — выполните действие, которое должно произойти, если отзыв доступен.
2. **Review Not Available** — выполните действие, которое должно произойти, если отзыв не доступен (если `reviewCanShow = false`). Действие может быть, например, скрыть кнопку.
3. **Review Success** — выполните действие, которое должно происходить в случае, когда пользователь оставил отзыв. За это можно выдавать награду. На данный ивент также можно подписаться в коде. Событие `YG2.onReviewSent` возвращает значения `bool` типа, которое равно `true`, если отзыв был успешно оставлен.
4. **Review Failed** — выполните действие, которое должно происходить в случае, если пользователь не оставил отзыв, а закрыл окно для оценки игры.

Также:

Аналогичными событиями можно пользоваться в коде.



Оглавление раздела:

1. Ярлык игры на рабочий стол

Ярлык игры на рабочий стол

С помощью диалогового окна вы можете предложить пользователю добавить на рабочий стол ярлык — это может быть как ссылка на игру, так и ярлык Яндекс Игр — зависит от разных обстоятельств.

В папке модуля → Prefabs есть префаб **Game Label** со скриптом **Game Label YG** — это готовая кнопка для вызова диалогового окна установки ярлыка. Она будет деактивирована, если ярлык не доуступен. Отредактируйте объекты префаба по предпочтениям.

Вы можете создать свою реализацию с помощью следующих объектов:

- Метод `YG2.GameLabelShowDialog` — вызывает диалоговое окно, в котором будет предложено установить ярлык. Доступность опции зависит от платформы, внутренних правил браузера и ограничений платформы.
- Поле `YG2.gameLabelCanShow` — используется для определения доступности установки ярлыка.
- Событие `YG2.onGameLabelSuccess` — вызывается при успешной установки ярлыка. В этом случае можно выдать вознаграждение.
- Событие `YG2.onGameLabelFail` — вызывается при не успешной установке ярлыка.



Полный экран (Fullscreen модуль)

С помощью модуля Fullscreen вы можете развернуть игру на полный экран через код игры. В таком случае скроется баннерная реклама. Для этого используйте метод:
`YG2.SetFullscreen(bool fullscreen)`
Также есть поле `isFullscreen` — выполняет запрос к платформе о состоянии полного экрана и возвращает переменную bolean типа.



Оглавление раздела:

1. Буфер обмена (Clipboard модуль)

Буфер обмена (Clipboard модуль)

В WebGL есть сложности с чтением и записью буфера обмена (CTRL + C, CTRL + V) из-за соображениям безопасности. Яндекс предоставляет только метод для записи:
`YG2.SetClipboardText(string value)`
Метод для чтения `YG2.GetClipboardTextAsync(string value)` — работает на других платформах. Необходимо передать колбек, который выполнится асинхронно.



Оглавление раздела:

1. Событие при выходе из игры (Quit Game Event модуль)

Событие при выходе из игры (Quit Game Event модуль)

Модуль Quit Game Event позволяет выполнить какой-либо метод при закрытии страницы игры (при выключении игры на WebGL).
При закрытии страницы выполнится поиск объекта на сцене с определённым названием, и в этом объекте произведётся поиск метода по названию в скриптах объекта.
В настройках плагина необходимо указать название объекта и название метода.
• Объект должен быть без родителей, то есть объект не должен учавствовать в какой-либо иерархии объектов.
• Метод не должен принимать как-либо значений.



Оглавление раздела:

1. Создание своего модуля

Создание своего модуля

При желании вы можете создать модуль, который будет определяться PluginYG и давать следующие возможности:

- Создание раздела с опциями модуля в настройках плагина;
- Для каждого модуля создаётся его дефайн (подробнее);
- Использование API плагина;

> **Создали модуль?**
> Напишите разработчику плагина: https://t.me/MaxBornysov



Оглавление раздела:

1. Android Wi-Fi ADB Connect (ADB Build)
2. Активация подключения (первое сопряжение)
3. Детали
4. Дополнительно

Android Wi-Fi ADB Connect (ADB Build)

Это инструмент, который позволяет легко подключить Android-устройство к Unity и быстро отправлять билды прямо на телефон. После сборки проекта билд будет автоматически отправлен по локальной сети Wi-Fi, установлен на смартфон и запущен.

Инструмент также позволяет тестировать игру без кабеля и без отдельной сборки через Unity Remote.
Беспроводное соединение удобнее в использовании и избавляет от риска повредить кабель или разъём зарядки телефона из-за постоянного подключения при тестировании.

При первом подключении инструмент автоматически создаёт постоянную связку между ПК и телефоном. Благодаря этому, при следующем подключении не придётся вводить коды — достаточно выбрать в Unity меню **Tools → ADB Wi-Fi → Connect**.

Скачать Android Wi-Fi ADB Connect (бесплатно)

Активация подключения (первое сопряжение)

Выполнить первое подключение можно как по кабелю USB, так и по Wi-Fi.

Через USB провод

1. Подключите USB-кабель.
2. Включите **отладку по USB**.
3. Выберите режим подключения «**Передача файлов**».
4. Введите IP вашего устройства в соответствующее поле.
5. Нажмите кнопку Сопряжения.

Через Wi-Fi

1. На телефоне перейдите в **Настройки** → **Для разработчиков** → **Отладка по Wi-Fi**.
2. Включите беспроводную отладку.
3. На экране появится IP устройства и текущий порт — введите его в поле **Port 1**.
4. Нажмите «**Подключить устройство с помощью кода подключения**».
5. Введите код в поле **Code**. Ниже будет показан новый порт для подключения — укажите его в поле **Port 2**.
6. Нажмите кнопку активации подключения.

Детали

- После перезагрузки телефона, скорее всего, придётся снова выполнить «активацию подключения». Это ограничение механизма ADB.
- При первом подключении телефон может показать уведомление о неудачной попытке, хотя соединение будет установлено. Это нормально: скрипт сначала ищет ранее подключённое устройство, чтобы сделать процесс максимально быстрым.
- Чтобы сделать быстрый билд с установкой на телефон, выберите устройство в **Build Settings** → **Build to Device**. Возможно, понадобится нажать кнопку Refresh справа от этого поля. Если устройство не отображается, но в логе инструмента видно успешное подключение, измените настройку в Unity: **Edit** → **Project Settings** → **Editor** → **Device** → **Any Android Device**.
- Чтобы в настройках телефона появился режим разработчика, его нужно активировать. В интернете есть множество инструкций, как это сделать.

Дополнительно

- Инструмент — это всего лишь один скрипт, который можно разместить в любой папке проекта. Он работает только в Unity Editor и не включается в билд.
- Если Unity не видит телефон или не получается подключиться через USB, возможно, нужно установить драйвер устройства. Самый простой способ — использовать

универсальный драйвер Universal Adb Driver Setup, либо обновить драйвер телефона через «Диспетчер устройств» Windows.

⚠ **Обратите внимание**

- **Поддержка MacOS отсутствует**.